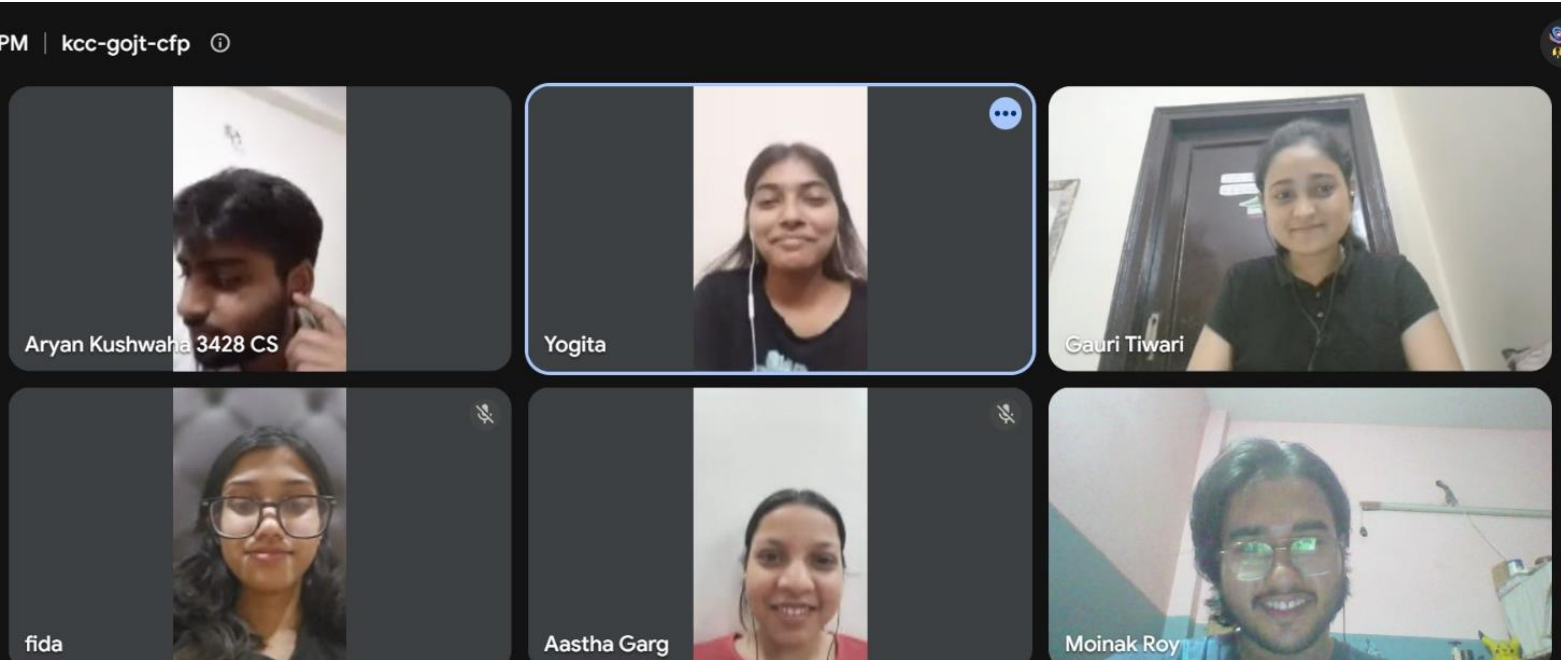
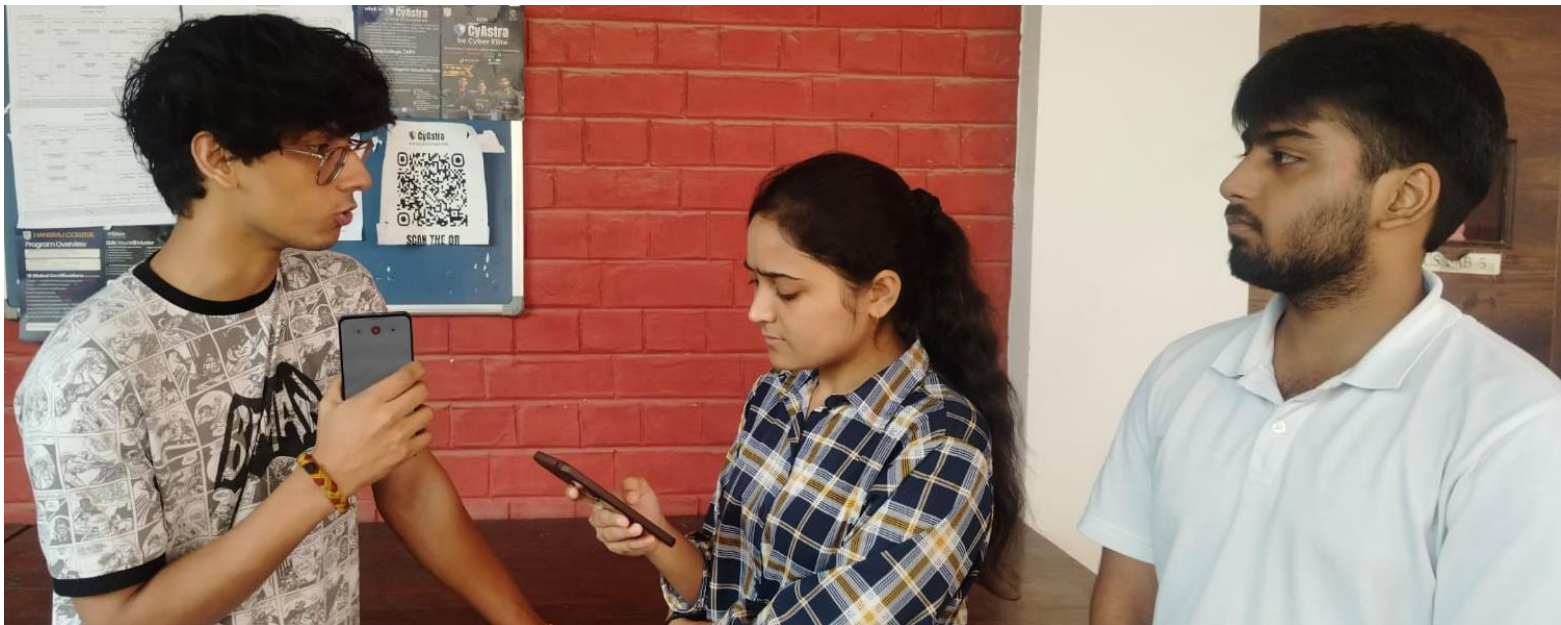
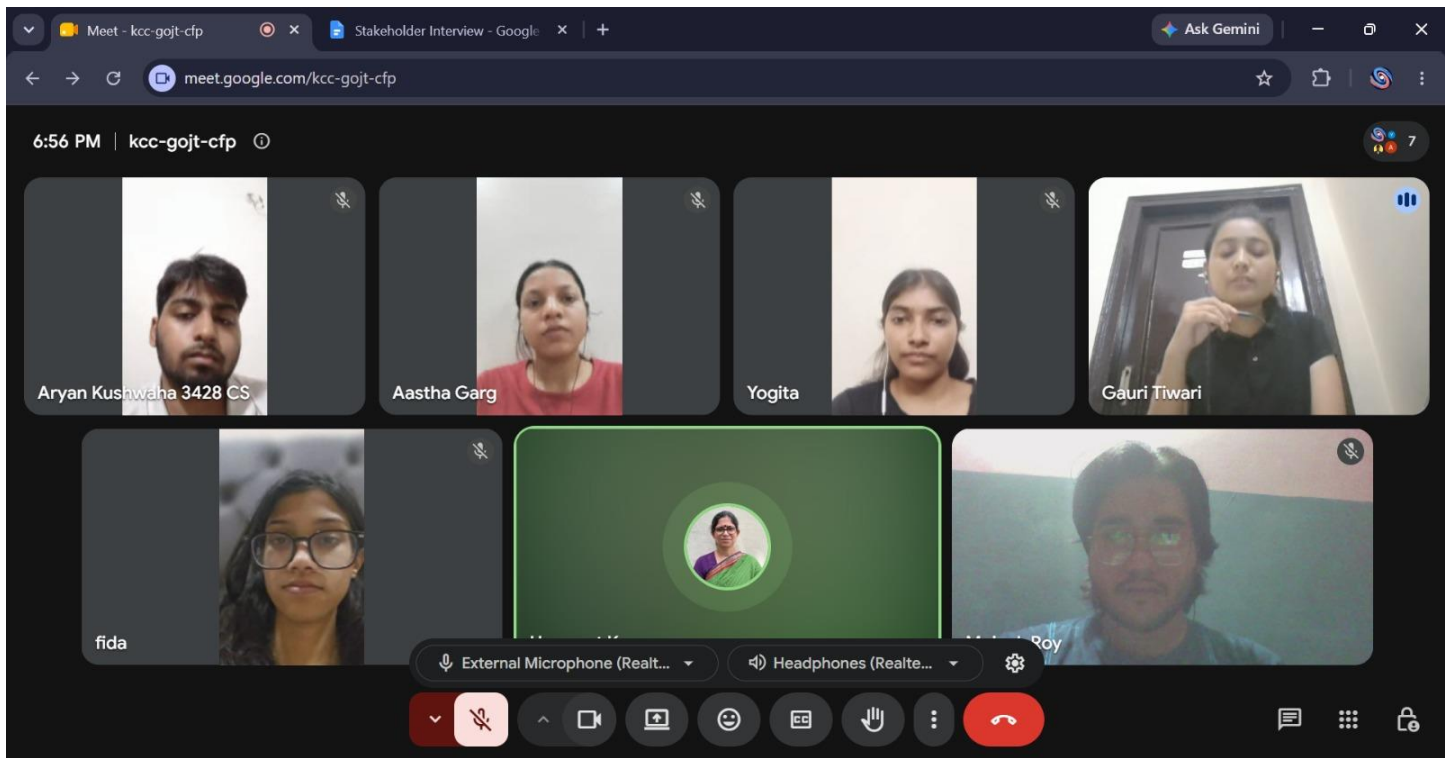




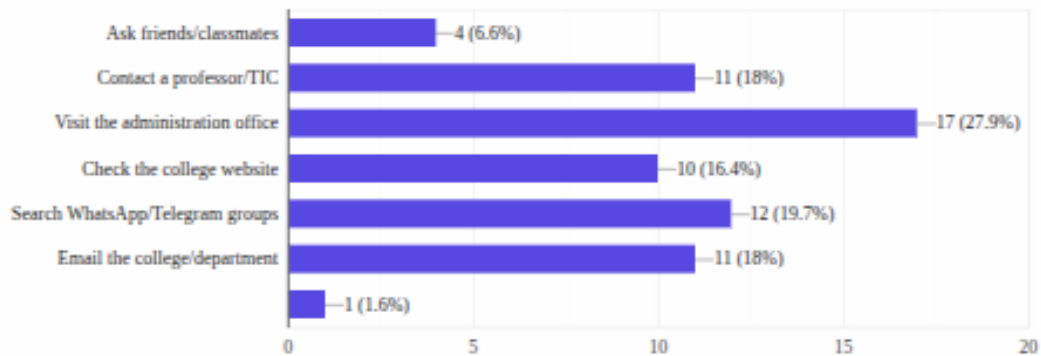


Google Form Responses

How do you usually find information when you have a college-related query?

[Copy chart](#)

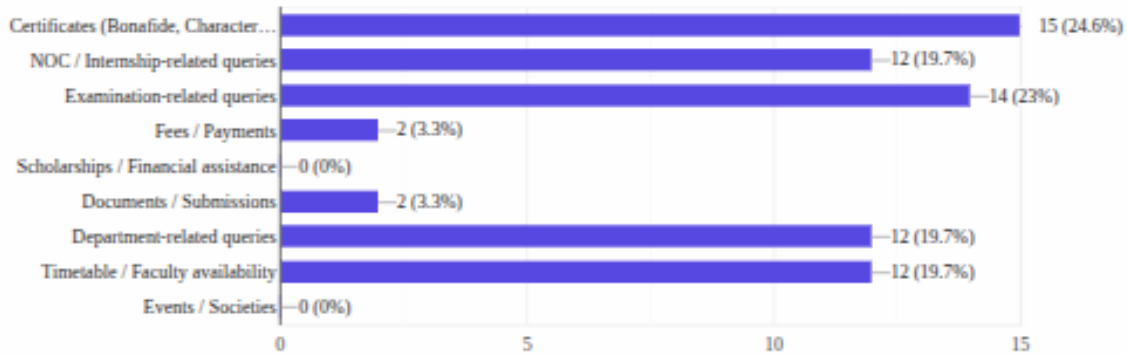
61 responses



What types of college-related queries do you most frequently need help with?

[Copy chart](#)

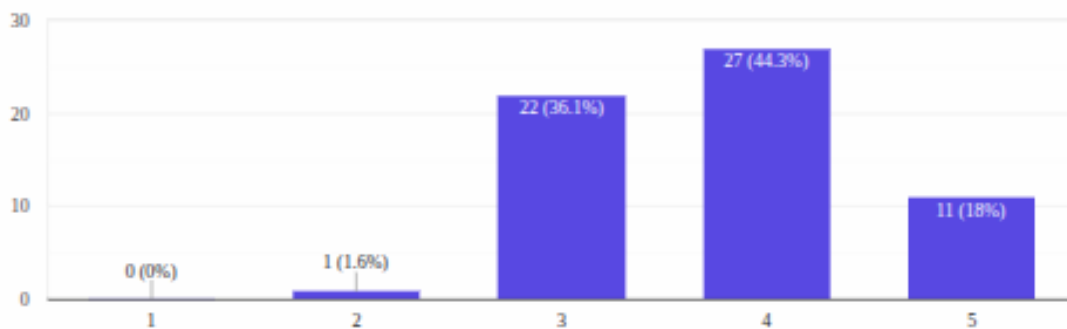
61 responses



How difficult is it to find the **correct person or department** to approach for a query?

[Copy chart](#)

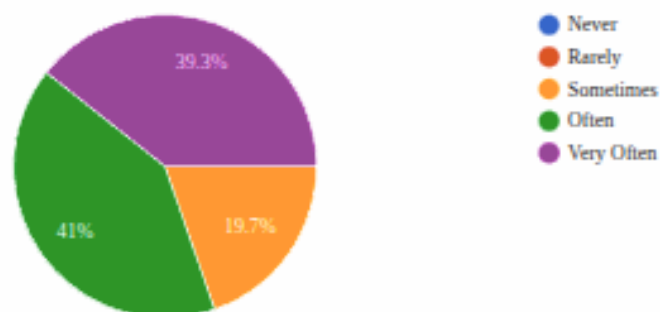
61 responses



How often do you have to contact or visit **more than one person/office** to resolve a single query?

 Copy chart

61 responses



What is the biggest problem you face when trying to get a college-related query resolved?

 Copy chart

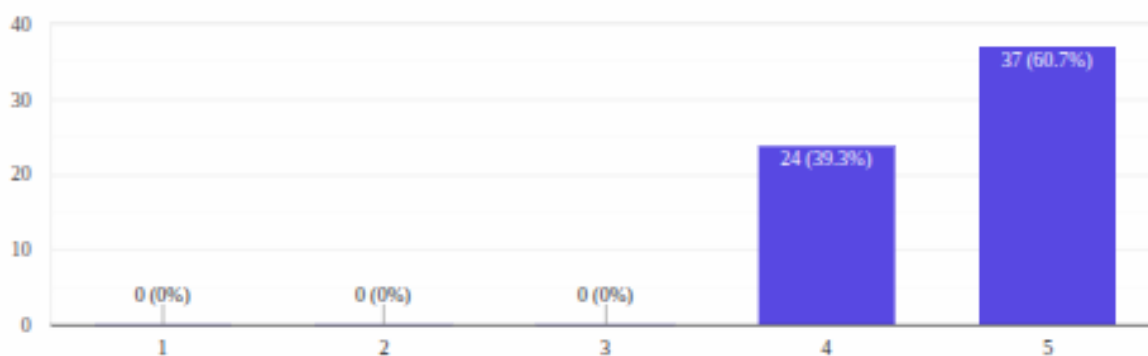
58 responses



How useful would you find a **single platform** where you can ask any college-related query in natural language?

 Copy chart

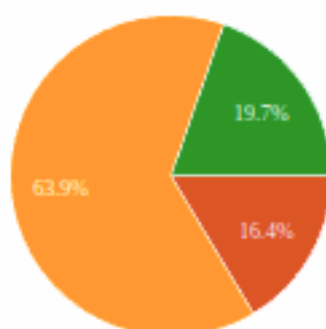
61 responses



If the system does not have a reliable answer to your query, what should it ideally do?

 [Copy chart](#)

61 responses

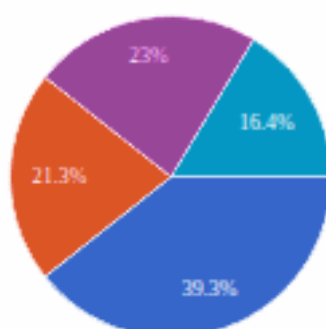


- Give its best possible answer
- Ask me for more information
- Automatically forward the query to the appropriate college authority
- Tell me whom I should contact and provide their details

If your query is forwarded to an Admin/TIC, which information would you want to see?

 [Copy chart](#)

61 responses

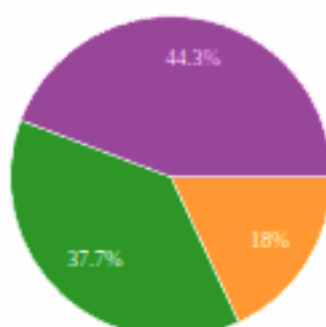


- Current query status
- Person/department handling the query
- Date and time of submission
- Previous responses/communication
- Expected response time
- Required documents

How important is it for you to receive **notifications when your query is updated or resolved?**

 [Copy chart](#)

61 responses

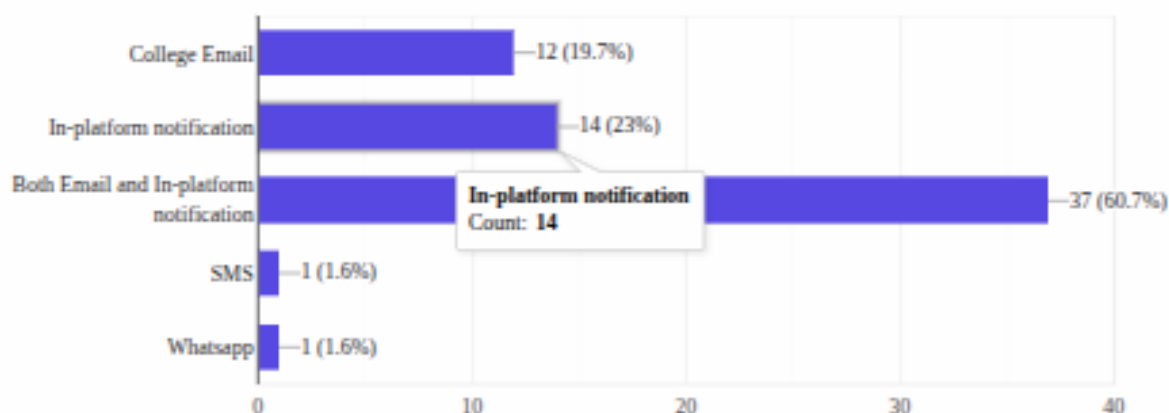


- Not Important
- Slightly Important
- Moderately Important
- Very Important
- Extremely Important

Which notification methods would you prefer?

 Copy chart

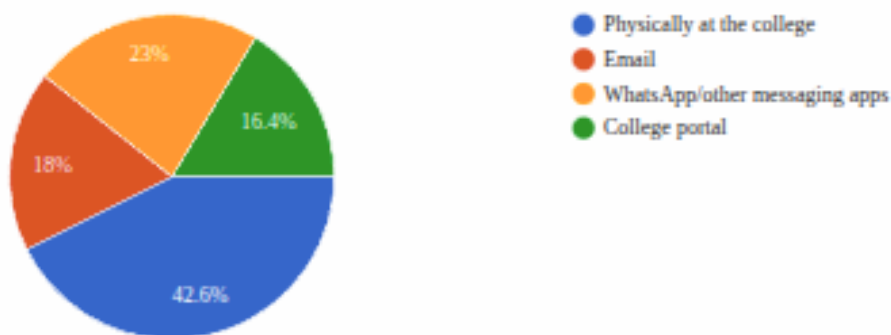
61 responses



How do you currently submit documents when a faculty member or administration asks for them?

 Copy chart

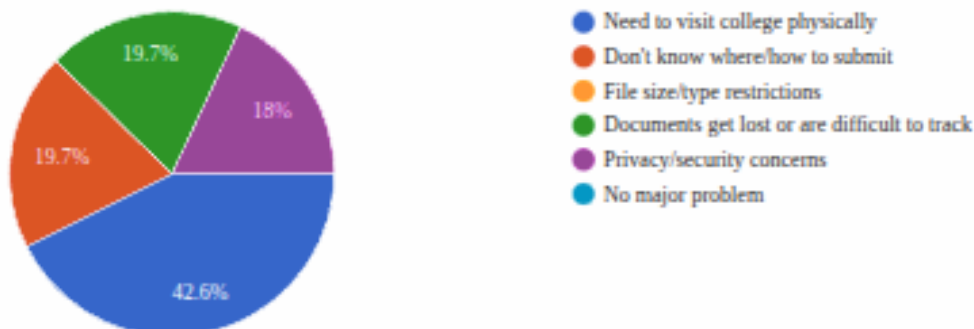
61 responses



What problems have you faced while submitting or sharing documents?

 Copy chart

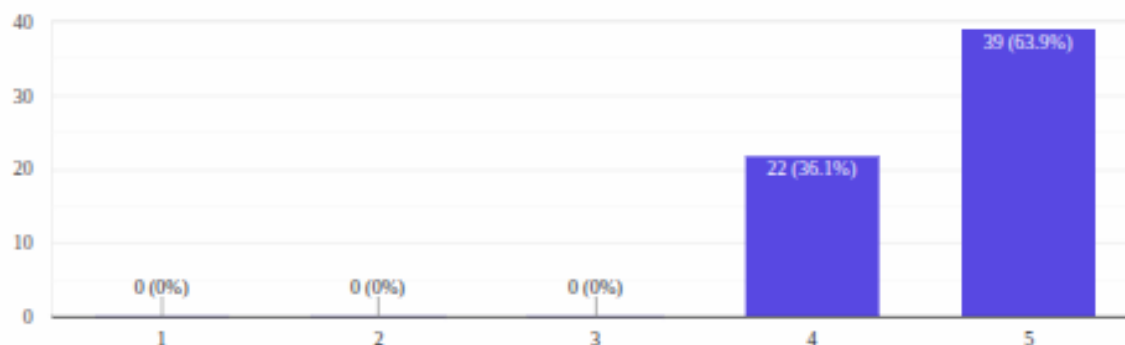
61 responses



How useful would it be to have a **secure document upload feature** directly inside the platform?

 Copy chart

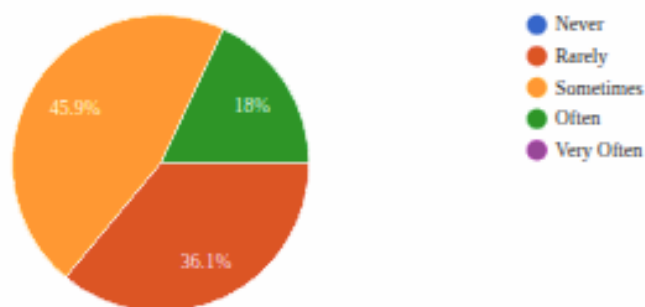
61 responses



How often do you need to meet a faculty member/TIC in person to resolve a query?

 Copy chart

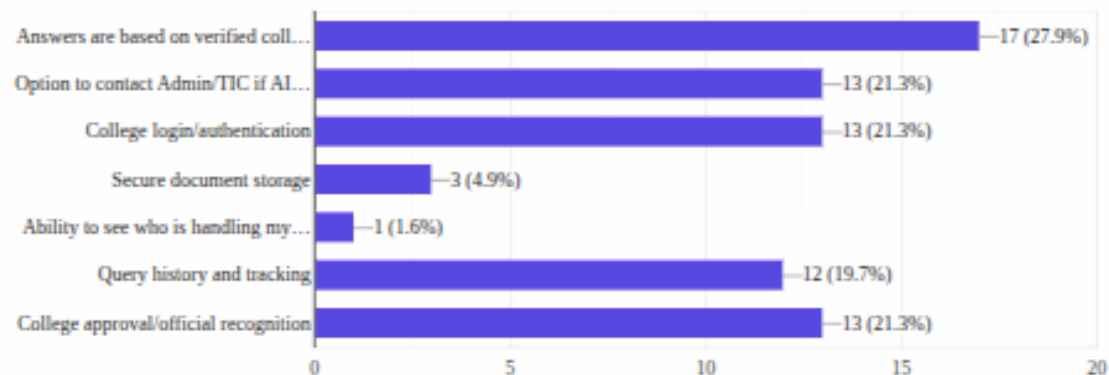
61 responses



What would make you comfortable trusting an AI-based system with your college queries and personal documents?

 Copy chart

61 responses



Inference / Conclusion of this Survey / Form:

Core Administrative Pain Points

- Roughly 80% of student reports having to visit or contact multiple offices to resolve a single query "Often" or "Very Often," with a 62.3% rating on the difficulty of finding the correct authority as high (4 or 5 out of 5).
- Students rely on many channels with no single source of truth—primarily in-person office visits (27.9%), informal WhatsApp/Telegram groups (19.7%), departmental emails (18.0%), and direct faculty reachouts (18.0%).
- Administrative certificates (Bonafide, Character, Transfer), examination-related issues (admit cards, re-evaluation), and department/timetable logistics represent the largest operational volume.

Demand for Centralized AI Platform

- 100% of surveyed students rated a single natural-language query resolution platform as highly useful (scores of 4 or 5 out of 5, average 4.61/5).
- 63.9% (39/61) expect the system to automatically forward unanswered queries to the designated Teacher-in-Charge (TIC) or administrative authority, rather than providing dead-end responses.
- When inquiries are forwarded, students prioritize real-time query status tracking, expected resolution timeframes, and handler identification.

Document Handling & Notification Infrastructure

- 42.6% of students currently submit documents in person, citing repeated college visits and untracked/lost paperwork as major frustrations.
- 100% of respondents rated a built-in, secure document upload feature as highly useful (average 4.64/5).
- 82% consider resolution updates "Very Important" or "Extremely Important," with a majority favoring multi-channel updates (Both College Email and In-Platform notifications).

Key Trust Drivers for Adoption: Student willingness to trust the system is primarily driven by verified college-sourced answers, official institutional authentication/login, and a direct fallback option to human administrators.

Response Authentication

Student Requirements Survey – Hansraj Query Bridge (Responses) ☆ 📁 ☁				
File Edit View Insert Format Data Tools Extensions Help				
🔍 Menus ↶ ↷ 🖨 📄 100% \$ % .0+ .00 123 Roboto - 10 + B I 🔗 A 📌 🏠				
A58	fx 8/31/2026 15:11:26			
	A	B	C	
	Form_Responses ▾ 📅			
1	Timestamp ▾	How do you usually find information when you ▾	What types of college-related queries do you ▾	How difficult is it to ▾
6	8/21/2026 10:27:29	Visit the administration office	Certificates (Bonafide, Character Certificate, etc.)	
7	8/21/2026 11:21:29	Search WhatsApp/Telegram groups	Timetable / Faculty availability	
8	8/21/2026 11:41:40	Search WhatsApp/Telegram groups	Timetable / Faculty availability	
9	8/22/2026 10:54:15	Check the college website	Department-related queries	
10	8/22/2026 11:38:43	Check the college website	Department-related queries	
11	8/22/2026 13:28:55	Email the college/department	Examination-related queries	
12	8/22/2026 13:31:07	Search WhatsApp/Telegram groups	Timetable / Faculty availability	
13	8/22/2026 15:01:42	Visit the administration office	Certificates (Bonafide, Character Certificate, etc.)	
14	8/22/2026 15:34:53	Email the college/department	Examination-related queries	
15	8/22/2026 17:20:28	Contact a professor/TIC	NOC / Internship-related queries	
16	8/22/2026 19:50:15	Contact a professor/TIC	NOC / Internship-related queries	
17	8/22/2026 20:32:30	Email the college/department	Examination-related queries	
18	8/22/2026 20:42:27	Check the college website	Department-related queries	
19	8/23/2026 9:42:59	Visit the administration office	Certificates (Bonafide, Character Certificate, etc.)	
20	8/23/2026 10:51:18	Search WhatsApp/Telegram groups	Timetable / Faculty availability	
21	8/23/2026 18:17:12	Check the college website	Department-related queries	
22	8/24/2026 10:34:24	Visit the administration office	Certificates (Bonafide, Character Certificate, etc.)	
23	8/24/2026 12:00:29	Email the college/department	Examination-related queries	
24	8/24/2026 12:45:10	Visit the administration office	Certificates (Bonafide, Character Certificate, etc.)	
25	8/25/2026 11:00:58	Contact a professor/TIC	NOC / Internship-related queries	
26	8/25/2026 13:49:52	Search WhatsApp/Telegram groups	Timetable / Faculty availability	
+ ☰ 📄 Form Responses 1 ▾				

19:43



19:43



CS(Hons) 2024-28 Unofficial
~ Lalit Kumar, Deepika, Josue, Kshiti, Liter...



SEC - Front End Web Design and...
Anchal, Deepika, Literally, Yogitaa, ~ Ans...



📎 Photo

Please answer honestly based on your own experience.
Your responses will really help us with the project. Thanks! 🙌

19:18



Student Requirements Survey – Hansraj Query Bridge
This survey aims to understand t...
[forms.gle](https://forms.gle/1G1NjcReTLmW2jf38)

Hello, guys!
We're working on our Software Engineering project, "Hansraj Query Bridge" and need your help with our survey.
It's about the day-to-day difficulties.

This is the questionnaire: <https://forms.gle/1G1NjcReTLmW2jf38>

Please answer honestly based on your own experience.
Your responses will really help us with the project. Thanks! 🙌

19:36 ✓

How do you usually find information when you have a college-related query?

- ☐ Ask friends/classmates
- ☐ Contact a professor/TIC
- ☐ Visit the administration office
- ☐ Check the college website
- ☐ Search WhatsApp/Telegram groups
- ☐ Email the college department

Student Requirements Survey – Hansraj Query Bridge

This survey aims to understand the difficulties students face while obtaining college-related information and resolving administrative or departmental queries. Your responses will help us identify the key requirements for HANS Connect – Intelligent Query Resolution System, a proposed platform for fa

[forms.gle](https://forms.gle/1G1NjcReTLmW2jf38)

Hello, guys!
We're working on our Software Engineering project, "Hansraj Query Bridge" and need your help with our survey.
It's about the day-to-day difficulties.

This is the questionnaire: <https://forms.gle/1G1NjcReTLmW2jf38>

Please answer honestly based on your own experience.
Your responses will really help us with the project. Thanks! 🙌

19:42 ✓

19:42



19:42



SEC NETWORK FLOW SEM 5 (El...
Anchal, Deepika, Literally, ~ Aditya Yadav,...



GE_Understanding Development
5 online



📎 Photo

How do you usually find information when you have a college-related query?

- ☐ Ask friends/classmates
- ☐ Contact a professor/TIC
- ☐ Visit the administration office
- ☐ Check the college website
- ☐ Search WhatsApp/Telegram groups
- ☐ Email the college department

Student Requirements Survey – Hansraj Query Bridge
This survey aims to understand the difficulties students face while obtaining college-related information and resolving administrative or departmental queries. Your responses will help us identify the key requirements for HANS Connect – Intelligent Query Resolution System, a proposed platform for fa

[forms.gle](https://forms.gle/1G1NjcReTLmW2jf38)

Hello, guys!
We're working on our Software Engineering project, "Hansraj Query Bridge" and need your help with our survey.
It's about the day-to-day difficulties.

This is the questionnaire: <https://forms.gle/1G1NjcReTLmW2jf38>

Please answer honestly based on your own experience.
Your responses will really help us with the project. Thanks! 🙌

19:41 ✓

How do you usually find information when you have a college-related query?

- ☐ Ask friends/classmates
- ☐ Contact a professor/TIC
- ☐ Visit the administration office
- ☐ Check the college website
- ☐ Search WhatsApp/Telegram groups
- ☐ Email the college department

Student Requirements Survey – Hansraj Query Bridge
This survey aims to understand the difficulties students face while obtaining college-related information and resolving administrative or departmental queries. Your responses will help us identify the key requirements for HANS Connect – Intelligent Query Resolution System, a proposed platform for fa

[forms.gle](https://forms.gle/1G1NjcReTLmW2jf38)

Hello, guys!
We're working on our Software Engineering project, "Hansraj Query Bridge" and need your help with our survey.
It's about the day-to-day difficulties.

This is the questionnaire: <https://forms.gle/1G1NjcReTLmW2jf38>

Please answer honestly based on your own experience.
Your responses will really help us with the project. Thanks! 🙌

19:42 ✓

Citations:

Form / Survey: <https://forms.gle/x3ig1QsofwBGMWk77>

Form Response:

<https://docs.google.com/spreadsheets/d/138uHrVNEkXFALfKgoRLipFpqY94rzYlXX2HhatuwwNY/edit?usp=sharing>

Student Stakeholder Interview: <https://drive.google.com/file/d/1k-QxRilq6HxMCPPaPao0DDQko0STv7Dq/view>

TIC / Admin Stakeholder Interview:

https://drive.google.com/file/d/1CUlazTDABFfScUUmDh2amhrDcHCqxBL/view?usp=drive_link

Research Paper 1: [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)

Research Paper 2: [2019 Conference on Empirical Methods in Natural Language Processing and 9th International Joint Conference on Natural Language Processing](#)

Research Paper 3: [2020 Conference on Empirical Methods in Natural Language Processing](#)

Research Paper 4: [International Conference on Learning Representations \(ICLR 2024\)](#)

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; ^{*}New York University;
plewis@fb.com

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory can overcome this issue, but have so far been only investigated for extractive downstream tasks. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) — models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We compare two RAG formulations, one which conditions on the same retrieved passages across the whole generated sequence, and another which can use different passages per token. We fine-tune and evaluate our models on a wide range of knowledge-intensive NLP tasks and set the state of the art on three open domain QA tasks, outperforming parametric seq2seq models and task-specific retrieve-and-extract architectures. For language generation tasks, we find that RAG models generate more specific, diverse and factual language than a state-of-the-art parametric-only seq2seq baseline.

1 Introduction

Pre-trained neural language models have been shown to learn a substantial amount of in-depth knowledge from data [47]. They can do so without any access to an external memory, as a parameterized implicit knowledge base [51, 52]. While this development is exciting, such models do have downsides: They cannot easily expand or revise their memory, can’t straightforwardly provide insight into their predictions, and may produce “hallucinations” [38]. Hybrid models that combine parametric memory with non-parametric (i.e., retrieval-based) memories [20, 26, 48] can address some of these issues because knowledge can be directly revised and expanded, and accessed knowledge can be inspected and interpreted. REALM [20] and ORQA [31], two recently introduced models that combine masked language models [8] with a differentiable retriever, have shown promising results,

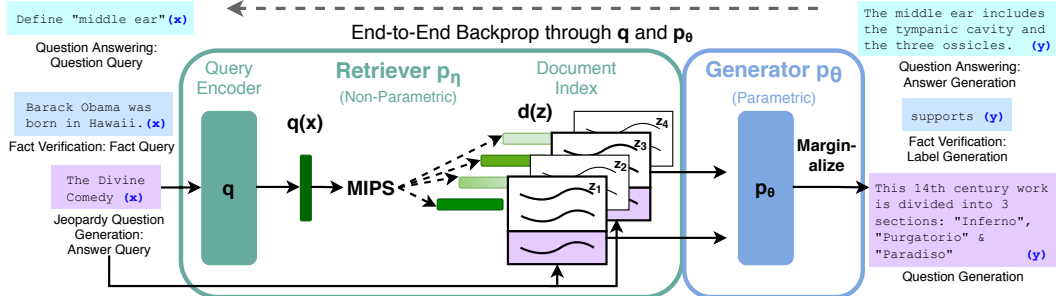


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i . For final prediction y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

but have only explored open-domain extractive question answering. Here, we bring hybrid parametric and non-parametric memory to the “workhorse of NLP,” i.e. sequence-to-sequence (seq2seq) models.

We endow pre-trained, parametric-memory generation models with a non-parametric memory through a general-purpose fine-tuning approach which we refer to as retrieval-augmented generation (RAG). We build RAG models where the parametric memory is a pre-trained seq2seq transformer, and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We combine these components in a probabilistic model trained end-to-end (Fig. 1). The retriever (Dense Passage Retriever [26], henceforth DPR) provides latent documents conditioned on the input, and the seq2seq model (BART [32]) then conditions on these latent documents together with the input to generate the output. We marginalize the latent documents with a top-K approximation, either on a per-output basis (assuming the same document is responsible for all tokens) or a per-token basis (where different documents are responsible for different tokens). Like T5 [51] or BART, RAG can be fine-tuned on any seq2seq task, whereby both the generator and retriever are jointly learned.

There has been extensive previous work proposing architectures to enrich systems with non-parametric memory which are trained from scratch for specific tasks, e.g. memory networks [64, 55], stack-augmented networks [25] and memory layers [30]. In contrast, we explore a setting where both parametric and non-parametric memory components are pre-trained and pre-loaded with extensive knowledge. Crucially, by using pre-trained access mechanisms, the ability to access knowledge is present without additional training.

Our results highlight the benefits of combining parametric and non-parametric memory with generation for *knowledge-intensive tasks*—tasks that humans could not reasonably be expected to perform without access to an external knowledge source. Our RAG models achieve state-of-the-art results on open Natural Questions [29], WebQuestions [3] and CuratedTrec [2] and strongly outperform recent approaches that use specialised pre-training objectives on TriviaQA [24]. Despite these being extractive tasks, we find that unconstrained generation outperforms previous extractive approaches. For knowledge-intensive generation, we experiment with MS-MARCO [1] and Jeopardy question generation, and we find that our models generate responses that are more factual, specific, and diverse than a BART baseline. For FEVER [56] fact verification, we achieve results within 4.3% of state-of-the-art pipeline models which use strong retrieval supervision. Finally, we demonstrate that the non-parametric memory can be replaced to update the models’ knowledge as the world changes.¹

2 Methods

We explore RAG models, which use the input sequence x to retrieve text documents z and use them as additional context when generating the target sequence y . As shown in Figure 1, our models leverage two components: (i) a retriever $p_\eta(z|x)$ with parameters η that returns (top-K truncated) distributions over text passages given a query x and (ii) a generator $p_\theta(y_i|x, z, y_{1:i-1})$ parametrized

¹Code to run experiments with RAG has been open-sourced as part of the HuggingFace Transformers Library [66] and can be found at <https://github.com/huggingface/transformers/blob/master/examples/rag/>. An interactive demo of RAG models can be found at <https://huggingface.co/rag/>

by θ that generates a current token based on a context of the previous $i - 1$ tokens $y_{1:i-1}$, the original input x and a retrieved passage z .

To train the retriever and generator end-to-end, we treat the retrieved document as a latent variable. We propose two models that marginalize over the latent documents in different ways to produce a distribution over generated text. In one approach, *RAG-Sequence*, the model uses the same document to predict each target token. The second approach, *RAG-Token*, can predict each target token based on a different document. In the following, we formally introduce both models and then describe the p_η and p_θ components, as well as the training and decoding procedure.

2.1 Models

RAG-Sequence Model The RAG-Sequence model uses the same retrieved document to generate the complete *sequence*. Technically, it treats the retrieved document as a single latent variable that is marginalized to get the seq2seq probability $p(y|x)$ via a top-K approximation. Concretely, the top K documents are retrieved using the retriever, and the generator produces the output sequence probability for each document, which are then marginalized,

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) \prod_i^N p_\theta(y_i|x, z, y_{1:i-1})$$

RAG-Token Model In the RAG-Token model we can draw a different latent document for each target *token* and marginalize accordingly. This allows the generator to choose content from several documents when producing an answer. Concretely, the top K documents are retrieved using the retriever, and then the generator produces a distribution for the next output token for each document, before marginalizing, and repeating the process with the following output token. Formally, we define:

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z_i, y_{1:i-1})$$

Finally, we note that RAG can be used for sequence classification tasks by considering the target class as a target sequence of length one, in which case RAG-Sequence and RAG-Token are equivalent.

2.2 Retriever: DPR

The retrieval component $p_\eta(z|x)$ is based on DPR [26]. DPR follows a bi-encoder architecture:

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{BERT}_d(z), \quad \mathbf{q}(x) = \text{BERT}_q(x)$$

where $\mathbf{d}(z)$ is a dense representation of a document produced by a $\text{BERT}_{\text{BASE}}$ *document encoder* [8], and $\mathbf{q}(x)$ a query representation produced by a *query encoder*, also based on $\text{BERT}_{\text{BASE}}$. Calculating $\text{top-}k(p_\eta(\cdot|x))$, the list of k documents z with highest prior probability $p_\eta(z|x)$, is a Maximum Inner Product Search (MIPS) problem, which can be approximately solved in sub-linear time [23]. We use a pre-trained bi-encoder from DPR to initialize our retriever and to build the document index. This retriever was trained to retrieve documents which contain answers to TriviaQA [24] questions and Natural Questions [29]. We refer to the document index as the *non-parametric memory*.

2.3 Generator: BART

The generator component $p_\theta(y_i|x, z, y_{1:i-1})$ could be modelled using any encoder-decoder. We use BART-large [32], a pre-trained seq2seq transformer [58] with 400M parameters. To combine the input x with the retrieved content z when generating from BART, we simply concatenate them. BART was pre-trained using a denoising objective and a variety of different noising functions. It has obtained state-of-the-art results on a diverse set of generation tasks and outperforms comparably-sized T5 models [32]. We refer to the BART generator parameters θ as the *parametric memory* henceforth.

2.4 Training

We jointly train the retriever and generator components without any direct supervision on what document should be retrieved. Given a fine-tuning training corpus of input/output pairs (x_j, y_j) , we

minimize the negative marginal log-likelihood of each target, $\sum_j -\log p(y_j|x_j)$ using stochastic gradient descent with Adam [28]. Updating the document encoder BERT_d during training is costly as it requires the document index to be periodically updated as REALM does during pre-training [20]. We do not find this step necessary for strong performance, and keep the document encoder (and index) fixed, only fine-tuning the query encoder BERT_q and the BART generator.

2.5 Decoding

At test time, RAG-Sequence and RAG-Token require different ways to approximate $\arg \max_y p(y|x)$.

RAG-Token The RAG-Token model can be seen as a standard, autoregressive seq2seq generator with transition probability: $p'_\theta(y_i|x, y_{1:i-1}) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z_i|x) p_\theta(y_i|x, z_i, y_{1:i-1})$. To decode, we can plug $p'_\theta(y_i|x, y_{1:i-1})$ into a standard beam decoder.

RAG-Sequence For RAG-Sequence, the likelihood $p(y|x)$ does not break into a conventional per-token likelihood, hence we cannot solve it with a single beam search. Instead, we run beam search for each document z , scoring each hypothesis using $p_\theta(y_i|x, z, y_{1:i-1})$. This yields a set of hypotheses Y , some of which may not have appeared in the beams of all documents. To estimate the probability of an hypothesis y we run an additional forward pass for each document z for which y does not appear in the beam, multiply generator probability with $p_\eta(z|x)$ and then sum the probabilities across beams for the marginals. We refer to this decoding procedure as “Thorough Decoding.” For longer output sequences, $|Y|$ can become large, requiring many forward passes. For more efficient decoding, we can make a further approximation that $p_\theta(y|x, z_i) \approx 0$ where y was not generated during beam search from x, z_i . This avoids the need to run additional forward passes once the candidate set Y has been generated. We refer to this decoding procedure as “Fast Decoding.”

3 Experiments

We experiment with RAG in a wide range of knowledge-intensive tasks. For all experiments, we use a single Wikipedia dump for our non-parametric knowledge source. Following Lee et al. [31] and Karpukhin et al. [26], we use the December 2018 dump. Each Wikipedia article is split into disjoint 100-word chunks, to make a total of 21M documents. We use the document encoder to compute an embedding for each document, and build a single MIPS index using FAISS [23] with a Hierarchical Navigable Small World approximation for fast retrieval [37]. During training, we retrieve the top k documents for each query. We consider $k \in \{5, 10\}$ for training and set k for test time using dev data. We now discuss experimental details for each task.

3.1 Open-domain Question Answering

Open-domain question answering (QA) is an important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output text pairs (x, y) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA paradigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also compare to “Closed-Book QA” approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24], WebQuestions (WQ) [3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on the TQA Wiki test set.

3.2 Abstractive Question Answering

RAG models can go beyond simple extractive QA and answer questions with free-form, abstractive text generation. To test RAG’s natural language generation (NLG) in a knowledge-intensive setting, we use the MSMARCO NLG task v2.1 [43]. The task consists of questions, ten gold passages retrieved from a search engine for each question, and a full sentence answer annotated from the retrieved passages. We do not use the supplied passages, only the questions and answers, to treat

Applying BERT to Document Retrieval with Birch

Zeynep Akkalyoncu Yilmaz, Shengjin Wang, Wei Yang, Haotian Zhang, and Jimmy Lin

David R. Cheriton School of Computer Science
University of Waterloo

Abstract

We present Birch, a system that applies BERT to document retrieval via integration with the open-source Anserini information retrieval toolkit to demonstrate end-to-end search over large document collections. Birch implements simple ranking models that achieve state-of-the-art effectiveness on standard TREC newswire and social media test collections. This demonstration focuses on technical challenges in the integration of NLP and IR capabilities, along with the design rationale behind our approach to tightly-coupled integration between Python (to support neural networks) and the Java Virtual Machine (to support document retrieval using the open-source Lucene search library). We demonstrate integration of Birch with an existing search interface as well as interactive notebooks that highlight its capabilities in an easy-to-understand manner.

1 Introduction

The information retrieval community, much like the natural language processing community, has witnessed the growing dominance of approaches based on neural networks. Applications of neural networks to document ranking usually involve multi-stage architectures, beginning with a traditional term-matching technique (e.g., BM25) over a standard inverted index, followed by a reranker that rescores the candidate list of documents (Asadi and Lin, 2013).

Researchers have developed a panoply of neural ranking models—see Mitra and Craswell (2019) for a recent overview—but there is emerging evidence that BERT (Devlin et al., 2019) outperforms previous approaches to document retrieval (Yang et al., 2019c; MacAvaney et al., 2019) as well as search-related tasks such as question answering (Nogueira and Cho, 2019; Yang et al., 2019b).

We share with the community Birch,¹ which integrates the Anserini information retrieval toolkit² with a BERT-based document ranking model that provides an end-to-end open-source search engine. Birch allows the community to replicate the state-of-the-art document ranking results presented in Yilmaz et al. (2019) and Yang et al. (2019c). Here we summarize those results, but our focus is on system architecture and the rationale behind a number of implementation design decisions, as opposed to the ranking model itself.

2 Integration Challenges

The problem we are trying to solve, and the focus of this work, is how to bridge the worlds of information retrieval and natural language processing from a software engineering perspective, for applications to document retrieval. Following the standard formulation, we assume a (potentially large) corpus D that users wish to search. For a keyword query Q , the system’s task is to return a ranked list of documents that maximizes a retrieval metric such as average precision (AP). This stands in contrast to reading comprehension tasks such as SQuAD (Rajpurkar et al., 2016) and many formulations of question answering today such as WikiQA (Yang et al., 2015) and the MS MARCO QA Task (Bajaj et al., 2018), where there is no (or minimal) retrieval component. These are better characterized as “selection” tasks on (pre-determined) text passages.

Within the information retrieval community, there exists a disconnect between academic researchers and industry practitioners. Outside of a few large organizations that deploy custom infrastructure (mostly commercial search engine companies), Lucene (along with the closely-related

¹<http://birchir.io/>

²<http://anserini.io/>

projects Solr and Elasticsearch) has become the de facto platform for building real-world search applications, deployed at Twitter, Netflix, eBay, and numerous other organizations. However, many researchers still rely on academic systems such as Indri³ and Terrier,⁴ which are mostly unknown in real-world production environments. This gap hinders technology transfer and the potential impact of research results.

Even assuming Lucene as a “common denominator” that academic researchers learn to adopt, there is still one technical hurdle: Lucene is implemented in Java, and hence runs on the Java Virtual Machine (JVM). However, most deep learning toolkits today, including TensorFlow and PyTorch, are written in Python with a C++ backend. Bridging Python and the JVM presents a technical challenge for NLP/IR integration.

3 Birch

3.1 Anserini

Anserini (Yang et al., 2017, 2018) represents an attempt to better align academic researchers with industry practitioners by building a research-focused toolkit on top of the open-source Lucene search library. Further standardizing on a common platform within the academic community can foster greater replicability and reproducibility, a growing concern in the community (Lin et al., 2016).

Already, Anserini has proven to be effective and has gained some traction: For example, Nogueira and Cho (2019) used Anserini for generating candidate documents before applying BERT to ranking passages in the TREC Complex Answer Retrieval (CAR) task (Dietz et al., 2017), which led to a large increase in effectiveness. Yang et al. (2019b) also combined Anserini and BERT to demonstrate large improvements in open-domain question answering directly on Wikipedia.

3.2 Design Decisions

The architecture of Birch is shown in Figure 1, which codifies a two-stage pipeline architecture where Anserini is responsible for retrieval, the output of which is passed to a BERT-based reranker. Since our research group has standardized on PyTorch, the central challenge we tackle is: How do we integrate the deep learning toolkit with Anserini?

³<https://www.lemurproject.org/>

⁴<http://terrier.org/>

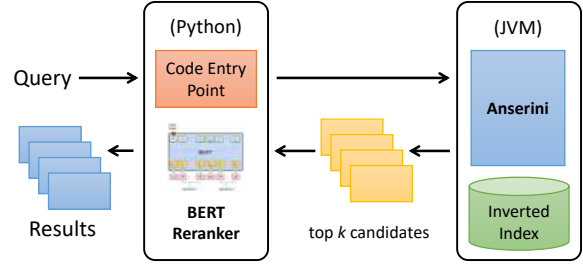


Figure 1: Architecture of Birch, illustrating a tight integration between Python and the Java Virtual Machine. The main code entry point is in Python, which calls Anserini for retrieval; candidate documents from Anserini are then reranked by our BERT models.

At the outset, we ruled out “loosely-coupled” integration approaches: For example, passing intermediate text files is not a sustainable solution in the long term. It is not only inefficient, but interchange formats frequently change (whether intentionally or accidentally), breaking code between multiple components. We also ruled out integration via REST APIs for similar reasons: efficiency (overhead of HTTP calls) and stability (imperfect solutions for enforcing API contracts, particularly in a research environment).

There are a few options for the “tightly-coupled” integration we desired. In principle, we could adopt the Java Virtual Machine (JVM) as the primary code entry point, with integration to the Torch backend via JNI, but this was ruled out because it would create two separate code paths (JVM to C++ for execution and Python to C++ for model development), which presents maintainability issues. After some exploration, we decided on Python as the primary development environment, integrating Anserini using the Pyjnius Python library⁵ for accessing Java classes. The library was originally developed to facilitate Android development in Python, and allows Python code to directly manipulate Java classes and objects. Thus, Birch supports Python as the main development language (and code entry point, as shown in Figure 1), connecting to the backend JVM to access retrieval capabilities.

3.3 Models

Our document ranking approach is detailed in Yilmaz et al. (2019) and Yang et al. (2019c). We follow Nogueira and Cho (2019) in adapting BERT for binary (specifically, relevance) classification over text. Candidate documents from Anserini

⁵<https://pyjnius.readthedocs.io/>

Model	2011		2012		2013		2014	
	AP	P@30	AP	P@30	AP	P@30	AP	P@30
QL	0.3576	0.4000	0.2091	0.3311	0.2532	0.4450	0.3924	0.6182
RM3	0.3824	0.4211	0.2342	0.3452	0.2766	0.4733	0.4480	0.6339
MP-HCNN (Rao et al., 2019)	0.4043	0.4293	0.2460	0.3791	0.2896	0.5294	0.4420	0.6394
BiCNN (Shi et al., 2018)	0.4293	0.4728	0.2621	0.4147	0.2990	0.5367	0.4563	0.6806
Birch	0.4697	0.5040	0.3073	0.4356	0.3357	0.5656	0.5176	0.7006

Table 1: Results on test collections from the TREC Microblog Tracks, comparing BERT with selected neural ranking models. The first two blocks of the table contain results copied from Rao et al. (2019).

are processed individually. As model input, we concatenate the query Q and document D into a text sequence $[[CLS], Q, [SEP], D, [SEP]]$, and then pad each text sequence in a mini-batch to N tokens, where N is the maximum length in the batch. The $[CLS]$ vector is then taken as input to a single layer neural network. Starting from a pre-trained BERT model, we fine-tune with existing relevance judgments using cross-entropy loss. BERT inference scores are then combined with the original retrieval scores, in the simplest case, using linear interpolation.

In this simple approach, long documents pose a problem since BERT wasn’t specifically designed to perform inference on long input texts. We present a simple solution: inference is applied over each sentence in a candidate document and sentence-level evidence is aggregated for ranking documents as follows:

$$S_f = a \cdot S_{doc} + (1 - a) \cdot \sum_{i=1}^n w_i \cdot S_i \quad (1)$$

where S_{doc} is the original document score and S_i is the i -th top-scoring sentence according to BERT; a and w_i ’s are parameters that need to be learned. In practice, we only consider up to the three top-scoring sentences in each document.

The intuition behind this approach comes from Zhang et al. (2018b,a), who found that the “best” sentence or paragraph in a document provides a good proxy for document relevance. From a different perspective, we are essentially implementing a form of passage retrieval.

4 Retrieval Results

4.1 TREC 2011–2014 Microblog Tracks

As originally reported in Yang et al. (2019c), Birch was evaluated on tweet test collections from the TREC Microblog Tracks, 2011 to 2014 (Lin et al.,

2014). Since tweets are short, relevance judgments can be directly used to fine-tune the BERT model (Section 3.3). For evaluation on each year’s dataset, we used the remaining years for fine-tuning, e.g., tuning on 2011–2013 data, testing on 2014 data. Additional details on the fine-tuning strategy and experimental settings are described in Yang et al. (2019c).

At retrieval (inference) time, query likelihood (QL) with RM3 relevance feedback (Nasreen et al., 2004) was used to provide the initial pool of candidates (to depth 1000). Since tweets are short, we can apply inference over each candidate document in its entirety. The interpolation weight between the BERT scores and the retrieval scores was tuned on the validation data.

Experimental results are shown in Table 1, where we present average precision (AP) and precision at rank 30 (P@30), the two official metrics of the evaluation (Ounis et al., 2011). The first two blocks of the table are copied from Rao et al. (2019), who compared bag-of-words baselines (QL and RM3) to several popular neural ranking models as well as MP-HCNN, the model they introduced. The results of Rao et al. (2019) were further improved in Shi et al. (2018); in all cases, the neural models include interpolation with the original document scores. We see that Birch yields a large jump in effectiveness across all Microblog collections.

4.2 TREC 2004 Robust Track

In addition to searching short social media posts, we also examined a “traditional” document retrieval task over newswire articles. For this, we used the test collection from the TREC 2004 Robust Track (Voorhees, 2004), which comprises 250 topics over a newswire corpus of around 500K documents. Here, we provide a summary of Yilmaz et al. (2019), which contains more detailed

Model	AP	P@20	NDCG@20
BM25+RM3	0.2903	0.3821	0.4407
1S: BERT	0.3676	0.4610	0.5239
2S: BERT	0.3697	0.4657	0.5324
3S: BERT	0.3691	0.4669	0.5325

Table 2: Results on Robust04, where n S denotes combining scores from the top n sentences in a document.

descriptions of our approach and presents experiments on more test collections.

The additional challenge with ranking newswire articles is the lack of training data to fine-tune the BERT models, since relevance judgments are provided at the document level. That is, in the standard formulation of document ranking, a document is considered relevant if *any* part of it is relevant—but documents are typically longer than the lengths of text BERT was designed to handle. The surprising finding of Yilmaz et al. (2019) is that BERT models fine-tuned with the Microblog test collections in Section 4.1 can be *directly* applied to rank newswire documents, despite the differences in domain (social media posts vs. news articles). Furthermore, it appears that out-of-domain passage-level relevance judgments *fortuitously* available, such as the MS MARCO passage dataset (Bajaj et al., 2018) and the TREC CAR dataset (Dietz et al., 2017), are also beneficial.

Thus, it appears that BERT is able to learn *cross-domain, sentence-level* notions of relevance that can be exploited for ranking newswire documents. Table 2 presents an extract of results from Yilmaz et al. (2019) for Robust04, where we find that the best results are achieved by first fine-tuning on MS MARCO and then on the Microblog data. Scores from BERT are then combined with document scores (BM25+RM3) based on Eq (1). The notation “1S”, “2S”, and “3S” refer to aggregating scores from the top one, two, and three sentences, respectively. Including more sentences doesn’t help and ranking is already quite good if we just consider the top-scoring sentence. This result, surprisingly, suggests that document ranking can be distilled into relevance prediction primarily at the sentence level. Based on the meta-analysis by Yang et al. (2019a), this is not only the highest known AP reported on the Robust04 dataset for neural models, but also exceeds the previous best known AP score of 0.3686, which is a non-neural method based on ensembles.

5 Demonstration

We demonstrate the integration of Birch with the search frontend from HiCAL (Abualsaud et al., 2018b) and interactive Google Colab notebooks.

5.1 HiCAL

The goal of the HiCAL system⁶ is to help human assessors efficiently find as many relevant documents as possible in a large document collection to achieve high recall on a search task. The system comprises two main components: a Continuous Active Learning (CAL) model (Cormack and Grossman, 2014) and a search model. In the CAL model, a machine-learned classifier selects the most likely relevant document or paragraph for the assessor to judge; judgments are then fed back to retrain the classifier. In the current HiCAL implementation, Anserini provides the back-end search capabilities.

For the TREC Common Core Tracks in 2017 and 2018, a small group of researchers used HiCAL to find and judge relevant documents. The runs generated based on their assessments achieved the highest AP scores among all the submitted runs for two consecutive years (Zhang et al., 2017; Abualsaud et al., 2018a). The effectiveness of the system was further demonstrated in Zhang et al. (2018a).

We further augment the Anserini backend for HiCAL with Birch in two ways: First, HiCAL can directly take advantage of improved rankings provided by BERT. Second, the top-scoring sentences can be highlighted in the document to aid in assessment. A sample screenshot is shown in Figure 2. For query 336 “black bear attacks” from Robust04, we show part of the highest-scoring document LA081689-0039 with one of the top three sentences (according to BERT) highlighted.

5.2 Interactive Colab Notebooks

We present Google Colab⁷ notebooks that make it possible for anyone to reproduce our end-to-end document retrieval pipeline in an interactive manner.⁸ We make all our data and pre-trained models available, although users may also opt to rebuild them from scratch; the Colab GPU backend enables fine-tuning BERT models directly in the notebook environment.

⁶<https://github.com/hical>

⁷<https://colab.research.google.com/>

⁸To ensure long-term availability, sample notebooks are linked from the main Birch repository.



Figure 2: Screenshot of HiCAL using Birch to identify the most relevant sentences in a document retrieved for the query “black bear attacks”.

Our notebooks are set up to allow relevance scores to be computed for an entire test collection in batch, and also to support interactive querying. When the user issues a query through the interactive notebook, candidate documents from the corpus are first retrieved using Anserini. A sentence-level dataset is created on the fly from the initial ranking by splitting each document into its constituent sentences. Each sentence is fed into our BERT model to obtain a relevance score. These relevance scores are then aggregated with document scores to rerank the candidate documents, per Eq (1). The notebook setting allows a user to step through each part of the process and examine intermediate results to gain a better understanding of our approach.

In addition, we have implemented two methods to visualize the relevant documents for a given query from a test collection, hopefully conveying even more insights. First, we generate a table that displays the document scores juxtaposed with the BERT scores of constituent sentences. Sentences with low document scores but high BERT scores (and vice versa) are highlighted, allowing the user to examine the relative contributions of exact term matching and semantic matching, as contributed by BERT. Second, we incorporate *bertviz*,⁹ an open-source tool for visualizing attention in transformer models, to explore the interaction between multiple attention heads. In Figure 3, we show a sentence with a high BERT score in a document retrieved for query 322 “international art crime” from Robust04. Note that this sentence does *not*

⁹<https://github.com/jessevig/bertviz>

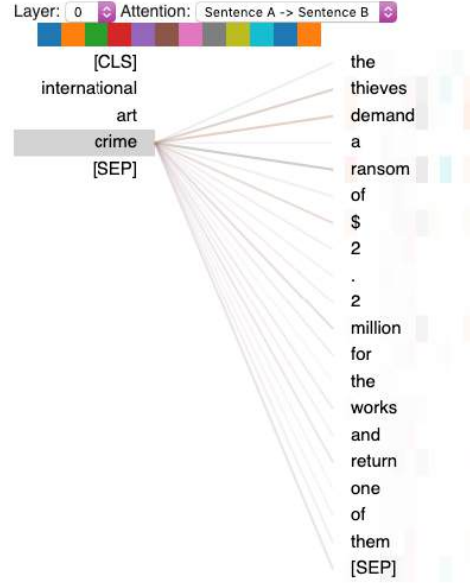


Figure 3: Screenshot of attention visualization for a sentence with a high BERT score from a document retrieved for the query “international art crime”. Note the lack of exact matches with query terms.

contain any of the query terms, but yet appears to be relevant. If we examine the attention visualization for the query term “crime”, we see that the model attends to obviously-related terms like “thieves”, “demand”, and “ransom”, illustrating the semantic knowledge that is captured in the BERT model.

6 Conclusions

This paper describes the system architecture and motivation behind a straightforward application of BERT to document ranking, via sentence-level inference and simple score aggregation. With the implementation of this system, we have also overcome the technical challenge of integrating a Lucene-based backend on the JVM with PyTorch to enable development in an environment NLP researchers and practitioners are already familiar with. The fruits of our labor are released in an open-source system for the community to continue explorations in search-related tasks with BERT.

Acknowledgments

This research was supported by the Natural Sciences and Engineering Research Council (NSERC) of Canada, and enabled by computational resources provided by Compute Ontario and Compute Canada.

CFO: A Framework for Building Production NLP Systems

Rishav Chakravarti¹ Cezar Pendus² Andrzej Sakrajda² Anthony Ferritto¹
Lin Pan¹ Michael Glass² Vittorio Castelli² J. William Murdock¹
Radu Florian² Salim Roukos² Avirup Sil^{2*}

¹IBM Watson, ²IBM Research AI

{rchakravarti, cpendus, ansa, panl, mrglass, vittorio, murdockj, raduf, roukos, avi}@us.ibm.com
aferritto@ibm.com

Abstract

This paper introduces a novel orchestration framework, called CFO (COMPUTATION FLOW ORCHESTRATOR), for building, experimenting with, and deploying interactive NLP (Natural Language Processing) and IR (Information Retrieval) systems to production environments. We then demonstrate a question answering system built using this framework which incorporates state-of-the-art BERT based MRC (Machine Reading Comprehension) with IR components to enable end-to-end answer retrieval. Results from the demo system are shown to be high quality in both academic and industry domain specific settings. Finally, we discuss best practices when (pre-)training BERT based MRC models for production systems.

1 Introduction

Production NLP (Natural Language Processing) and IR (information retrieval) applications often rely on a system flow consisting of multiple components that need to be woven together to build an end-to-end system (A. Ferrucci et al., 2010; Yang et al., 2019). This paper presents a novel approach for defining flow graphs and a toolkit for compiling those definitions into deploy-able production grade systems¹.

Though the framework, which we refer to as CFO (COMPUTATION FLOW ORCHESTRATOR), is well suited to a variety of use cases, we demonstrate it by creating an interactive QA (Question Answering) system that can be used both for academic benchmarking as well as industry specific use cases. The interactive system integrates SOTA (state-of-the-art) BERT-based MRC (Machine Reading Comprehension), an Elasticsearch

based document retrieval component, and a deduplication & sorting component to provide end-to-end answer retrieval. We will refer to this demonstration system as GAAMA (Go Ahead, Ask Me Anything). The key contributions of this work, therefore, are to (1) introduce a novel framework for stitching together deployable NLP components, (2) demonstrate the framework with an end-to-end QA system, and (3) discuss the training steps necessary for adapting a SOTA MRC model to a data set before plugging it into the QA system.

Section 2 provides the motivations and details of the CFO framework, section 3 discusses the specific model components integrated into GAAMA, section 4 discusses experimentation to adapt the BERT-based MRC component to this system, section 5 discusses related work, and, finally, section 6 provides a conclusion and discussion of future work.

In addition, a (private) screencast video demonstration of GAAMA has been uploaded at http://ibm.biz/gaama_demo (along with a supplementary presentation of the CFO framework at http://ibm.biz/gaama_cfo_demo).

2 CFO Architecture

The CFO framework relies on two sets of system specifications to define the computation flow graph. First, each node within the graph defines its service name, input message data fields, and output message data fields using Google’s Protocol Buffer Interface Definition Language². Second, the orchestrator is described using a custom specification format allowing the user to declare:

1. The set of nodes (provided as containerized gRPC³ microservices). Each node will have

* Corresponding author.

¹We are actively seeking to open source the toolkit and flow definition language. If successful, we will be releasing the code to http://ibm.biz/cfo_framework

²<https://developers.google.com/protocol-buffers/>

³<https://grpc.io/>

implemented a microservice according to the node’s declared interface specification.

2. The set of nodes to treat as entry points to the flow graph.
3. A mapping within the flow for each *data element* comprising the input and output message interfaces for nodes.
4. For ease of deployment, the specification also provides the ability to describe deployment specific configuration settings like service ports, docker registry location etc.

Given these specification files, CFO provides a compiler to auto-generate an orchestrator node which implements the computation flow graph, a (dockerized) launch script, and a simple REST interface / GUI which provide access to the defined entry points and debug information.

The resulting design allows data flows and dependencies to be described both concisely and transparently. The flow specification allows for easy debugging and modification of how data fields enter the computation flow, get transformed, and finally outputted. This is a significant departure from traditional orchestration systems which would require parsing through source code to determine and change the route taken by data fields through the computation flow. Furthermore, the use of Protocol Buffers to describe these data fields ensures a language and platform agnostic representation that does not compromise the system’s speed or ability to ensure data type correctness at compile time (as opposed to traditional JSON/XML representations which need to encode/decode from strings at run time).

Another benefit of the CFO toolkit is to auto-generate the serialization and connectivity code for each node as well as exposing the entry points via REST interfaces. Relinquishing this responsibility to the CFO toolkit frees the developer up from writing a significant amount of boilerplate code and worrying about distributed system best practices such as enforcing time outs, error propagation, latency logging, and parallelization through asynchronous calls. As an illustrative example, the auto-generated code for the GAAMA demo consists of 2,800 lines of C++ orchestration code.

Finally, the CFO toolkit generates a set of shell scripts, docker images, and config files for deploying and running the flow graph using either

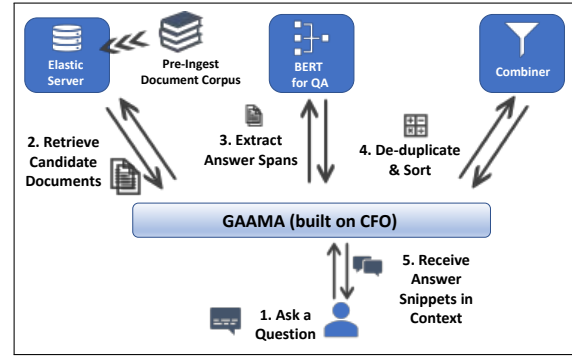


Figure 1: GAAMA System Architecture

docker-compose⁴ or kubernetes⁵. This allows the generated project to be deployed both locally for debugging as well as on modern cloud infrastructure.

3 GAAMA Architecture

As a simple case study for CFO, we create a demonstration QA system consisting of four nodes: (1) an Elasticsearch⁶ based IR node (2) a BERT based MRC node (3) an answer “de-duplication” node and (4) a final answer combiner node. See fig. 1 for an overview of the QA system.

As described in section 2, each component starts by declaring an interface specification allowing the underlying implementation to be swapped out without the need to modify the rest of the QA system. Next, a gRPC server is implemented with the core business logic. Auto-generated gRPC code stubs provide the core communication/serialization logic for the service layer of the server. We describe the IR and MRC nodes in further detail in the following two sections.

3.1 IR with Elasticsearch

The core business logic of the IR node uses Elasticsearch APIs to retrieve the top k documents from the appropriate corpus based on BM25, a popular variant of term frequency overlap between the query and document text (Robertson et al., 1976). Two document corpora are ingested using the standard English analyzer. The first corpus consists of Wikipedia paragraphs used for academic benchmarking and the second corpus consists of an industry dataset made up of IBM Technical Support Documentation. The user interface allows us to choose either corpus when asking

⁴<https://docs.docker.com/compose/>

⁵<https://kubernetes.io/>

⁶<https://www.elastic.co/products/elasticsearch>

questions to evaluate the system. We use “paragraphs” as the base unit of ingestion in line with (Yang et al., 2019) which shows this as an optimal pre-processing step for consumption by a MRC component.

The node’s input interface, therefore, accepts query text, a hyperparameter k , and a target corpus identifier. Its output interface produces a list of document texts accompanied by retrieval scores. As discussed earlier, these interfaces are defined using protocol buffer definitions, so we follow standard steps⁷ for auto-generating a gRPC server with placeholders for the custom business logic of retrieving documents. Similarly, with the server logic in place, we follow standard steps⁸ to create a docker image that CFO will need to launch the gRPC server. Though we are wrapping Elasticsearch’s index based retrieval implementation here, we can swap our implementation for more recent Neural IR based techniques (Craswell et al., 2017) without changing the exposed interface or the orchestration code.

3.2 MRC with BERT

The MRC node similarly wraps a BERT-for-QA model in a dockerized gRPC server which accepts a single query-document pair as its input and produces a span from the document along with a prediction score as its output. Note that CFO’s orchestrator automatically realizes that the IR node produces a list of documents, while the MRC node accepts a single document at a time. So CFO will take care of calling the MRC node for each of the k retrieved documents (using asynchronous calls to parallelize requests if a configuration flag is set).

The underlying BERT-for-QA model is based on (Alberti et al., 2019). BERT (Devlin et al., 2018) is one of a series of pre-trained neural models that can be fine tuned to provide state-of-the-art results in NLP (Peters et al., 2018; Howard and Ruder, 2018; Radford et al., 2019) including on the SQuAD (Rajpurkar et al., 2018) and NQ (Kwiatkowski et al., 2019) tasks that align with our MRC based QA.

We use the Huggingface PyTorch implementation of BERT⁹ which supports starting from a Base (a 12 layer, 768 hidden dimension, 12 atten-

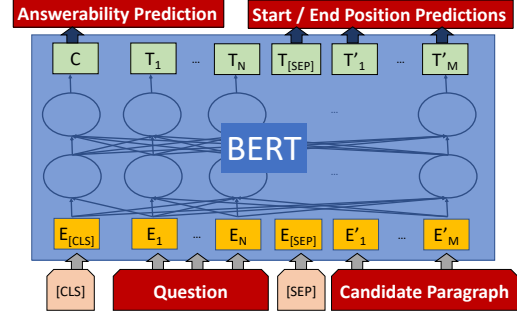


Figure 2: BERT for QA (Devlin et al., 2018)

tion head, 110M parameter transformer network) or a Large (a 24 layer, 1024 hidden dimension, 16 attention head, 340M parameter transformer network) model. An output feed forward layer is added on top of this to produce 3 sets of scores: (1) scores at each token offset marking the likelihood of an answer chunk starting at this offset (2) scores at each token offset marking the likelihood of an answer chunk ending at this offset (3) a score for the entire sequence marking the likelihood of the question being answerable given the current context.

The parameters of the entire network are fine tuned using a set of question and document pairs where annotators provide the correct start and end offsets or have marked the question-document pair as having no correct answer. Refer to fig. 2 for a visual depiction of the model and to (Alberti et al., 2019) for additional details about model architecture and implementation.

Section 4 includes practical pre-training, fine tuning, and hyperparameter optimization steps for building the final model deployed as part of GAAMA. At the time of writing, our BERT-based MRC model¹⁰ was the best performing submission (dated 7/31/2019), outperforming the next best system by 1% on F1 on the Natural Questions public leaderboard¹¹.

4 Experiments

Prior to integration of the MRC node into GAAMA, we first experiment with data preparation and training of the BERT MRC model as a standalone component using dev sets provided with the NQ and SQuAD data sets (see 5 for more on these data sets). NQ is preferred for evaluat-

⁷<https://grpc.io/docs/tutorials/>

⁸<https://github.com/grpc/grpc-docker-library/>

⁹<https://github.com/huggingface/pytorch-transformers>

¹⁰The best performing submission to the leaderboard uses an ensemble of models using different hyperparameters rather than a single model

¹¹<https://ai.google.com/research/NaturalQuestions/leaderboard>

Conundrums in Entity Coreference Resolution: Making Sense of the State of the Art

Jing Lu and Vincent Ng

Human Language Technology Research Institute

University of Texas at Dallas

Richardson, TX 75083-0688

{ljwinnie, vince}@hlt.utdallas.edu

Abstract

Despite the significant progress on entity coreference resolution observed in recent years, there is a general lack of understanding of what has been improved. We present an empirical analysis of state-of-the-art resolvers with the goal of providing the general NLP audience with a better understanding of the state of the art and coreference researchers with directions for future research.

1 Introduction

The advent of the neural NLP era has revolutionized virtually all areas of NLP research. For entity coreference, many issues that were once thought to be important no longer appear to be particularly relevant to the current research agenda. Specifically, while a decade ago coreference researchers have focused on developing computational models that are complex (e.g., structured models (Fernandes et al., 2012; Björkelund and Kuhn, 2014; Martschat and Strube, 2015)) and knowledge-rich (e.g., those that encode world knowledge (Ponzetto and Strube, 2007; Rahman and Ng, 2011a; Hajishirzi et al., 2013)), nowadays virtually all state-of-the-art resolvers employ a simple model (i.e., the mention-ranking model, which was developed more than a decade ago (Denis and Baldridge, 2008))¹ and a fairly simple input representation (i.e., contextualized word embeddings) in conjunction with a mechanism for learning representations of entity mention spans such that coreferent mentions have similar representations (Lee et al., 2017, 2018; Kantor and Globerson, 2019; Joshi et al., 2019).

Despite significant progress in the past few years in terms of performance numbers, what seems to be missing is an understanding of what has been

improved. The lack of understanding has long been a concern shared by coreference researchers, even before the neural revolution in NLP. This has led to several attempts to analyze coreference resolvers over the years (Stoyanov et al., 2009; Kummerfeld and Klein, 2013). With the development of neural resolvers, however, this concern has become more serious than ever: the fact that significant progress can be made via learning mention representations with a simple neural mention-ranking model that employs a fairly simple input representation for a task as challenging as coreference resolution (CR) is somewhat contrary to common wisdom.

In light of this apparent conundrum, we present an empirical analysis of state-of-the-art entity coreference resolvers through four major sets of experiments in this paper, with the goal of gaining insights into their behaviors. We believe that our analysis will not only provide the general NLP audience with a better understanding of the state of the art, but also provide coreference researchers with directions for future research.

2 Evaluation Setup

In this section, we describe the datasets, the evaluation metrics, the state-of-the-art resolvers and the hyperparameters used in our experiments.

Datasets. We report results on three coreference datasets. The NIST-sponsored ACE evaluations resulted in several datasets. We use ACE 2005 (Walker et al., 2006), the last one in the series. The ACE 2005 organizers have only made the official training set (but not the official test set) publicly available, so previous work defined different train-test splits over the official training set. We employ the same train-test split as Bansal and Klein (2012).

KBP is another series of NIST-sponsored evaluations in the mid 2010s. KBP does not have any evaluations on entity CR, but to support high-level

¹The first learning-based resolver is a pairwise ranker (Connolly et al., 1994), which was extended by Denis and Baldridge (2008) to rank more than two candidate antecedents at a time.

	ACE			OntoNotes			KBP		
	Train	Dev	Test	Train	Dev	Test	Train	Dev	Test
#docs	365	117	117	2802	343	348	360	97	168
#mentions	34481	11126	9261	155558	19155	19764	40628	10983	13860
#chains	11963	3798	3050	35142	4545	4532	14332	3942	5482

Table 1: Dataset statistics in terms of the number of documents, mentions, and coreference chains.

information extraction tasks (e.g., event extraction, event CR), the organizers have made available several corpora that include entity coreference annotations. For training, we use three such corpora (LDC2015E29, LDC2015E68, and LDC2016E64). For evaluation, the KBP 2017 organizers have made available the official test set for the event CR task (LDC2017E51), which also include entity coreference annotations. We use it as our test set.

OntoNotes (Hovy et al., 2006), which was developed circa 2006, is the most widely-used dataset for entity coreference evaluations. It has a standard train-dev-test split. Unlike in ACE and KBP, singleton clusters are not annotated in OntoNotes.

The key difference between these three corpora is that OntoNotes supports “unrestricted” CR, meaning that coreference links are annotated between entity mentions without regard to their entity types. In contrast, coreference links are only annotated between mentions belonging to one of the seven entity types in ACE and one of the five entity types in KBP. Statistics on these corpora are shown in Table 1.

Evaluation metrics. Following the convention established in the CoNLL 2011 and 2012 shared tasks (Pradhan et al., 2011, 2012), we use as our primary coreference evaluation measure the CoNLL score, which is the unweighted average of the F-scores provided by three popular metrics, the link-based MUC metric (Vilain et al., 1995), the mention-based B³ metric (Bagga and Baldwin, 1998), and the entity-based CEAF_e metric (Luo, 2005). We obtain these scores using the official CoNLL scorer (Pradhan et al., 2014).²

Mention detection (MD) is the task of extracting the mentions in a text needed for entity CR. A key observation made in the CoNLL shared tasks was that the performance of resolvers was limited by MD, so it is important to examine the extent to which MD performance has improved over the

years. We report performance in terms of recall, precision, and F-score, considering that a system mention is correctly detected if and only if it has an exact match in boundary with a gold mention.

Systems. We evaluate five variants of three state-of-the-art neural resolvers, all of which employ a ranking model where all candidate antecedents are ranked against each other for a given anaphor.

The first resolver, the Stanford neural resolver (Clark and Manning, 2016)³, takes as input a set of entity mentions identified for a given document by a rule-based MD system and trains using reinforcement learning a simple mention ranker consisting of three hidden layers of ReLU units and a final layer that is fully-connected.

The other two resolvers are developed by Lee et al. (2018)⁴ and Joshi et al. (2019)⁵. Both are *span-based* models, which have two key characteristics. First, mention spans are identified as part of CR, so this mitigates the propagation of errors from MD to CR. Second, *representations* of entity mention spans are learned so that coreferent mentions have similar representations. The key differences between these resolvers are: (1) in Lee et al. the input instances correspond to the sentences in the given document, whereas in Joshi et al. the input instances correspond to fixed-length non-overlapping segments of the input document⁶; (2) Lee et al. use a LSTM, whereas Joshi et al. use a transformer; and (3) the pretrained embeddings are different.

For each of these resolvers, we derive two variants. Specifically, Lee et al. (2018) employ GloVe+ELMo embeddings, but to better understand the effect of the contextual information provided by ELMo embeddings (Peters et al., 2018) on CR performance, we evaluate a version of Lee et al. using only GloVe embeddings (Pennington et al., 2014). We will henceforth refer to these two versions of Lee et al. as ELMo (i.e., ELMo+GloVe) and GloVe, respectively. Joshi

²LEA (Moosavi and Strube, 2016) is a coreference evaluation metric recently designed to address the shortcomings associated with B³ and CEAF_e, but we found no difference in the performance trends in our experiments according to CoNLL and LEA. See the Appendix for the LEA results.

³<https://github.com/clarkkev/deep-coref>

⁴<https://github.com/kentonl/e2e-coref>

⁵<https://github.com/mandarjoshi90/coref>

⁶This is the *independent* version in Joshi et al. (2019).

Hyperparameter	ACE			OntoNotes			KBP		
	GloVe/ELMo	SpanB-b	SpanB-l	GloVe/ELMo	SpanB-b	SpanB-l	GloVe/ELMo	SpanB-b	SpanB-l
Max span width	30	30	30	30	30	30	20	20	10
Max top antecedents	50	50	50	50	50	50	50	50	35
Max training segs/sents	50 sents	3 segs	3 segs	50 sents	3 segs	3 segs	50 sents	3 segs	3 segs
Top span ratio	0.4	0.35	0.4	0.4	0.4	0.4	0.35	0.35	0.35
Max segment length	—	384	512	—	384	512	—	384	512
SpanBERT learning rate	—	2e-5	1e-5	—	2e-5	1e-5	—	2e-5	2e-5
Task learning rate	0.001	1e-4	3e-4	0.001	1e-4	3e-4	0.001	1e-4	2e-4

Table 2: Best hyperparameters obtained on the development sets for each span-based resolver.

et al. (2019) employ embeddings pretrained using a new method called SpanBERT (Joshi et al., 2020), which is designed to better represent text *spans* than BERT. The two variants of Joshi et al. differ in the transformer. Specifically, SpanBERT-base (henceforth SpanBERT-b) employs a simple transformer while SpanBERT-large (henceforth SpanBERT-l) employs a more complex transformer.

We use the publicly-available implementation of each of these resolvers. There is one caveat, however. Recall that the span-based resolvers were all evaluated on OntoNotes. Since singleton clusters are not annotated in OntoNotes, all singleton clusters predicted by a resolver are removed from its output before it is sent to the scoring program. In contrast, singleton clusters that contain mentions belonging to one of the ACE/KBP entity types are annotated in ACE/KBP, so these mentions should not be removed from a resolver’s output. However, span-based resolvers cannot distinguish between spans that correspond to entity mentions and those that do not. To address this problem, we extend the span-based models so that they are jointly trained to predict entity mention spans and coreference links. Specifically, the feedforward neural network that is responsible for scoring a span in these models currently do not receive *direct* feedback on whether a span corresponds to an entity mention. We first turn it into a mention detector by training it in a supervised manner using the negative cross entropy loss, so that it predicts a positive mention score for a span if and only if the span corresponds to an entity mention. Then, to jointly learn MD and CR in the span-based resolvers, we employ a loss function that is the unweighted sum of the coreference loss and the MD loss.

Hyperparameter tuning. To ensure a fair comparison of the resolvers, we tune their hyperparameters to maximize the CoNLL score on development data. Note, however, that the authors of Stanford, ELMo, SpanBERT-b, and SpanBERT-l reported the best hyperparameter settings on OntoNotes in

the original papers (Lee et al., 2018; Joshi et al., 2019), so we simply use them in our experiments and focus on tuning the hyperparameters for the remaining cases. We adopt the set of hyperparameters to be tuned from the original papers.

For Stanford, there are three hyperparameters to tune: α_{WL} , α_{FA} , and α_{FN} . These are the weights associated with three different types of mistakes made by the coreference model. Following Clark and Manning (2016), we fix $\alpha_{WL} = 1.0$ and search for α_{FA} and α_{FN} out of $\{0.1, 0.2, \dots, 1.5\}$ using a variant of grid search. For ACE, $(\alpha_{WL}, \alpha_{FA}, \alpha_{FN}) = (1.0, 0.5, 1.0)$ is the best configuration, and for KBP, the best configuration is $(1.0, 0.5, 0.8)$. For OntoNotes, we use the configuration found by Clark and Manning, which is $(1.0, 0.5, 0.8)$.

For GloVe and ELMo, we have five hyperparameters to tune. Specifically, we search for: (1) max span width (i.e., maximum number of words in a candidate span) out of $\{10, 20, 30\}$; (2) max top antecedents (i.e., maximum number of candidate antecedents) out of $\{35, 40, 45, 50\}$; (3) max training sentences out of $\{25, 50, 75, 100\}$; (4) task learning rate out of $\{5e-4, 1e-3, 2e-3\}$; and (5) top span ratio (i.e., the fraction of top spans that survive the filtering) out of $\{0.3, 0.35, 0.4, 0.45, 0.5\}$. For the two SpanBERT resolvers, we have seven hyperparameters to tune. For three of the hyperparameters (max span width, max top antecedents, and top span ratio), the ranges are the same as those used in GloVe and ELMo. For the remaining four, we search for: (1) max training segments out of $\{3, 4, 5\}$; (2) max segment length out of $\{128, 256, 384, 512\}$; (3) SpanBERT learning rate out of $\{1e-5, 2e-5\}$; and (4) task learning rate out of $\{1e-4, 2e-4, 3e-4\}$. Table 2 shows the best hyperparameter setting of each span-based model on each dataset.

3 Performance across Datasets

We first provide the reader with a high-level understanding of the state of the art by analyzing the five

	System	ACE				OntoNotes				KBP			
		CoNLL MUC	Sing.	MD		CoNLL MUC	Sing.	MD		CoNLL MUC	Sing.	MD	
1	Stanford	38.6	57.9	56.8	50.6	65.5	74.3	—	80.1	26.2	48.8	42.8	26.4
2	GloVe	68.0	76.6	67.0	87.4	68.1	76.7	—	82.0	65.9	71.3	63.2	82.4
3	ELMo	71.8	79.5	70.1	90.4	73.0	80.5	—	85.1	69.7	73.8	67.5	85.3
4	SpanBERT-b	75.7	83.1	71.5	91.6	77.4	83.7	—	87.1	71.3	75.6	67.5	86.3
5	SpanBERT-l	78.9	85.4	75.1	92.3	79.6	85.3	—	88.2	75.8	80.1	71.1	88.5

Table 3: Results of the resolvers on the three coreference datasets.

resolvers’ performance on the three datasets.

Performance across datasets. Results on the three datasets, which are reported in terms of the CoNLL score, are shown in the CoNLL column in Table 3.⁷ Although the five resolvers have been evaluated solely on OntoNotes, their relative performances are consistent across the datasets. In particular, the use of ELMo embeddings enables ELMo to outperform GloVe by 3.8–4.9% points. SpanBERT-b outperforms ELMo by 1.6–4.4% points, and SpanBERT-l further outperforms SpanBERT-b by 2.2–4.5% points.

Source of performance improvements. Do the above improvements stem from improved recognition of coreference links, or improved recognition of singleton clusters, or both? To understand whether these resolvers have improved in terms of *link* prediction, we examine the MUC F-scores (see the MUC column), which are computed solely on coreference links. As we can see, the MUC scores are consistently increasing down the table across all datasets, meaning that later systems are indeed doing better at identifying coreference links. To understand whether later resolvers are also better at identifying singleton clusters, we show in the Singleton column the percentage of singleton clusters that are correctly *recalled*. Again, the scores are increasing down the table, and the degree of improvement is particularly large from GloVe to ELMo and from SpanBERT-b to SpanBERT-l.

Mention detection performance. First, MD performance has improved significantly over the years. SpanBERT-l achieves an F-score of 88.2 in MD on OntoNotes, which is significantly higher than the best MD F-score achieved in the CoNLL-2012 shared task (77.7). Note that Stanford’s mention detector performs substantially worse than those of the other resolvers, especially on ACE and KBP. The reason is that Stanford employs a rule-based MD system that was initially developed when the

Stanford NLP Group participated in the CoNLL-2011 shared task, whereas in the other resolvers MD is jointly trained with CR. Overall, MD performance appears to have a significant impact on CR performance. In particular, joint MD and CR in the span-based resolvers seems to be a driving force behind the rapid coreference performance improvements we have seen in recent years.

4 Using Oracles

Can the performance of coreference resolvers be further improved if we improve MD? Being able to answer this kind of questions is important: if further improvements in MD can result in significant gains in coreference performance, then future research efforts should perhaps be focused on MD.

To answer this kind of questions, we perform oracle experiments. Specifically, we provide a resolver with a particular type of perfect information (e.g., using gold mentions as input) and see how much performance improvement can be obtained.

4.1 Gold Mention Boundaries

Our first oracle experiment concerns training and testing our resolvers on gold mention boundaries. While this experiment has been conducted over the years by numerous researchers (e.g., Peng et al. (2015), Zhang et al. (2018)), we are primarily interested in understanding whether further improving an MD component that already has an F-score of more than 85% can improve coreference performance. For the four span-based models, we disable the component in the span representation layer that is responsible for proposing spans (i.e., mention boundaries) and instruct them to use gold mention spans instead. Note, however, that the representation of a span will be learned during training. In other words, although all resolvers are given gold mention spans, the span representations that will be used during resolution will still be different for different span-based resolvers.

Results, expressed in terms of the CoNLL score, are shown in the Gold Mention Boundaries col-

⁷Owing to space limitations, we show only the most important scores in Table 3. The detailed results (e.g., B³ and CEAF_s results) can be found in the Appendix.

	System	Gold Mention Boundaries			Perfect Anaphoricity			Gold Entity Types		
		ACE	Onto.	KBP	ACE	Onto.	KBP	ACE	Onto.	KBP
1	Stanford	76.3	83.7	80.0	40.3	73.4	29.2	39.5	71.8	27.7
2	GloVe	79.3	86.1	81.6	71.5	76.2	69.8	69.6	71.4	67.9
3	ELMo	81.8	87.2	83.5	76.1	82.0	73.8	73.3	76.7	71.6
4	SpanBERT-b	84.9	90.5	85.7	79.3	84.4	76.3	77.4	79.3	73.8
5	SpanBERT-l	87.3	91.9	88.0	82.8	86.9	80.1	80.4	81.4	77.6

Table 4: CoNLL scores of the resolvers using gold mention boundaries, perfect anaphoricity, and gold entity types.

umn in Table 4. First, despite recent significant improvement in MD, these results suggest that coreference performance can still be significantly improved just by improving MD: for the best resolver (SpanBERT-l), the CoNLL score can be improved by 8.4–12.3% points. Second, the relative performances of the resolvers are consistent across the three datasets: the CoNLL scores increase as we go down the table. Since the four span-based resolvers use essentially the same (mention-ranking) model for resolution and the same algorithm for weight updates, their performance differences can be attributed largely to differences in the pretrained embeddings and the encoder. In addition, these results suggest that the coreference performance improvements we observed in recent years can be attributed to not only improved mention (boundary) detection but also improved resolution accuracy presumably as a result of better span representations.

4.2 Perfect Anaphoricity

Anaphoricity determination, a.k.a. discourse-new detection (Poesio et al., 2004), is the task of determining whether a mention is coreferent with another mention that appears earlier in the text. Being able to identify non-anaphoric mentions could improve the precision of coreference resolvers, as any antecedent chosen for them is erroneous.

In this oracle experiment, we provide a resolver with perfect anaphoricity information, meaning that we know for every entity mention whether it is anaphoric or not. We use this perfect anaphoricity information during resolution: we will resolve all and only those mentions that are anaphoric.

Results are shown in the Perfect Anaphoricity column of Table 4. A few points deserve mention. First, all resolvers improved on all datasets when provided with perfect anaphoricity information. These results imply that anaphoricity determination remains an important issue in CR research, and further improvements in anaphoricity can improve CR. However, the gains that state-of-the-art resolvers can achieve by improving anaphoricity

determination are generally smaller than those by improving MD: the CoNLL scores of the span-based resolvers increase by 3.5–4.3% points on ACE, 4.4–9% points on OntoNotes, and 3.9–5% points on KBP. This is understandable, as MD is likely to improve both coreference precision and recall, whereas anaphoricity determination can only improve precision. Note that Stanford’s poor performance on ACE and KBP is due to poor MD.

4.3 Gold Entity Types

In this experiment, we assume that a resolver is given gold entity types (i.e., semantic classes) such as PERSON, ORGANIZATION, and LOCATION. The set of entity types to be provided is corpus-dependent. As mentioned before, ACE and KBP only have seven and five entity types respectively. In OntoNotes, however, only named entities are annotated with (one of 18) entity types. Consequently, we automatically derive entity types for pronouns and nominals using gold coreference chains: if a pronoun or a nominal appears in a coreference cluster that contains a name, we derive its entity type from that of the name. This method allows us to derive the entity type of 36.4% of the nominals and 70% of the pronouns. Any pronoun or nominal whose entity type cannot be derived using this method will be assigned the entity type UNKNOWN. While this method does not provide full coverage, we will still be able to examine whether having access to perfect entity types on a subset of the mentions will enable us to improve the performance of a resolver on OntoNotes.

We use entity types during resolution. We disallow a candidate antecedent to be selected as the antecedent for a given anaphor if they have different entity types. Results are shown in the Gold Entity Types column in Table 4. As we can see, all resolvers improved on all datasets when provided with gold entity types. Compared with the gains achieved using gold mention spans or perfect anaphoricity, the gains that come with the use of gold entity types are smaller: for the span-based

resolvers, the CoNLL scores increase by 1.5–1.7% points on ACE, 1.8–3.7% points on OntoNotes, and 1.9–2.5% points on KBP. In other words, state-of-the-art resolvers can be improved by improving the determination of entity types.

These results are particularly interesting in light of a conundrum in entity CR: while some researchers have reported successes with improving entity CR using automatically computed semantic information (Ng, 2007), there have also been numerous failed attempts (Kehler et al., 2004; Durrett and Klein, 2013; Sapena et al., 2013). Although the semantic information we use in this paper is restricted to gold entity types, our results suggest that hand-annotated semantic information is indeed useful, and the (non-)utility of semantics for CR reported in earlier work could be attributed to the noise inherent in computing semantic information.

5 Results on Resolution Classes

To gain additional insights into the state-of-the-art resolvers, we analyze their performance on different types of entity mentions. More specifically, motivated by Stoyanov et al. (2009), we partition the *gold* mentions into different *resolution classes*. While previous work has focused mainly on three coarse-grained resolution classes (namely, pronouns, names, and nominal mentions), we employ the 13 fine-grained resolution classes defined by Rahman and Ng (2011b), as discussed below.

Names. Four classes are defined for gold names. (1) **e**: a name is assigned to this *exact string match* class if there is a preceding mention such that the two are coreferent and are the same string; (2) **p**: a name is assigned to this *partial string match* class if there is a preceding mention such that the two are coreferent and have some content words in common; (3) **n**: a name is assigned to this *no string match* class if there is no preceding mention such that the two are coreferent and have some content words in common; and (4) **na**: a name is assigned to this *non-anaphor* class if it is not coreferent with any preceding mention.

Nominal mentions. Four analogous resolution classes are defined for gold mentions whose head is a nominal: (5) **e**; (6) **p**; (7) **n**; and (8) **na**.

Pronouns. We have three pronoun classes. (9) **1/2**: 1st and 2nd person pronouns; (10) **G3**: gendered 3rd person pronouns (e.g., *she*); (11) **U3**: ungendered 3rd person pronouns; (12) **oa**: any anaphoric pronouns that do not belong to (9), (10),

and (11) (e.g., relative pronouns); and (13) **na**: non-anaphoric pronouns (e.g., pleonastic pronouns).

Table 5 shows the performance of each resolver on each resolution class. To avoid overwhelming the reader, we only show the results of ELMo and SpanBERT-l, which will allow us to gain insights into what made SpanBERT-l better. Specifically, for each resolution class C , we show each resolver’s MD recall (percentage of gold mentions in C that are correctly recalled) under **MD** and its resolution accuracy (percentage of correctly identified anaphors in C that are correctly resolved)⁸ under **RA**. Under **Size** we show the percentage of gold mentions belonging to each resolution class.

First, if we consider only the three coarse-grained resolution classes, the results are perhaps not surprising: name resolution is the easiest and nominal resolution is the hardest.

Second, consider the 13 fine-grained resolution classes. By design, the names and the nominals in the ‘e’ class should be easier to resolve than those in ‘p’, which in turn should be easier to resolve than those in ‘n’. The results are consistent with this intuition. Results on the anaphoric pronoun classes are also consistent with our intuition: 3rd person gendered pronouns are the easiest to resolve, followed by 1st/2nd person gendered pronouns and then ungendered 3rd person pronouns.

Third, these results reveal that the difficulty of anaphoricity determination stems primarily from pronouns: while resolution accuracies on non-anaphoric names and nominal mentions are above 89%, those on non-anaphoric pronouns are only between 65.9% and 77.6%. Note that we consider a non-anaphoric mention correctly “resolved” if it is resolved to the dummy antecedent.

Finally, SpanBERT-l has better resolution accuracies than ELMo for all resolution classes on all datasets. Encouragingly, the harder a resolution class is, the bigger the improvement is. These results clearly show that we are making progress on resolving anaphors that are traditionally considered difficult to resolve. Note that part of this improvement can be attributed to improved MD, which increases the likelihood that the correct antecedent of an anaphor is present in its list of candidate antecedents. Additional experiments are needed to determine the impact of improved MD on improvement in resolution accuracies, however.

⁸In other words, the resolution accuracy does not depend on anaphor recall and precision.

	Class	ACE					OntoNotes					KBP				
		Size %	ELMo		SpanB-1		Size %	ELMo		SpanB-1		Size %	ELMo		SpanB-1	
			RA	MD	RA	MD		RA	MD	RA	MD		RA	MD	RA	MD
1	NAM-e	16.8	94.6	96.0	97.4	96.6	14.0	95.3	93.2	95.2	91.8	16.5	94.0	95.0	95.3	96.5
2	NAM-p	4.2	71.5	86.6	81.5	88.2	7.3	82.7	83.2	88.5	83.7	2.0	66.1	90.8	74.6	93.8
3	NAM-n	2.0	52.5	87.6	69.8	91.4	1.9	56.6	60.6	73.6	68.1	2.9	56.8	88.4	64.2	92.0
4	NAM-na	10.8	93.3	88.4	94.8	91.0	11.5	93.5	74.8	94.4	79.0	16.8	95.8	91.6	96.1	92.8
5	NOM-e	3.4	77.0	91.0	84.3	92.3	3.7	92.9	89.6	95.7	90.0	3.4	86.2	85.7	88.1	86.7
6	NOM-p	4.3	51.1	82.8	61.5	89.4	5.3	78.1	78.6	84.3	83.1	4.2	43.7	83.3	52.5	80.3
7	NOM-n	4.2	48.5	84.0	66.3	89.4	3.3	58.6	61.9	78.4	71.6	5.8	42.1	77.0	50.6	76.5
8	NOM-na	16.8	92.0	83.1	93.1	86.1	8.4	89.2	66.7	92.5	77.1	19.9	92.8	78.2	93.4	78.3
9	PRO-I/2	16.4	81.8	99.8	88.6	99.8	16.1	90.1	93.2	93.6	96.1	13.5	82.1	100	88.2	99.9
10	PRO-G3	5.7	88.5	99.8	94.1	100	10.7	91.6	99.6	95.9	99.3	6.5	88.3	100	94.7	99.7
11	PRO-U3	6.2	68.8	91.5	85.2	96.8	13.8	84.1	93.4	91.2	96.0	4.6	70.5	93.4	79.5	97.8
12	PRO-oa	3.9	52.9	80.3	70.3	86.1	2.2	57.0	57.3	69.1	70.0	1.2	68.3	69.8	72.0	72.7
13	PRO-na	5.3	73.9	87.3	77.6	90.6	1.8	65.9	80.5	69.6	84.3	2.8	66.5	93.5	72.6	92.7

Table 5: Results on resolution classes.

6 Sensitivity to Perturbed Inputs

Next, we conduct a series of experiments that involve perturbing the input. In each experiment, we (1) replace a certain kind of words/phrases in each training document with other words/phrases, (2) train a coreference model on these perturbed training documents, and (3) evaluate the output. Our goal is to gain insights into the behavior of state-of-the-art resolvers by examining how sensitive their performance is to perturbations in the input. Specifically, if performance drops significantly when a particular kind of words/phrases is replaced, that means the replaced words/phrases are important in the model learning process. Note that perturbations are only applied to the training documents; no changes are made to the test documents.

We divide the different kinds of perturbations into two broad categories, mention-internal perturbations and mention-external perturbations.

6.1 Mention-internal Perturbations

Mention-internal perturbations involve making changes to the words within an entity mention.

6.1.1 Perturbations to Names

We consider two kinds of perturbations to names.

Unseen names. We replace each name in a training document with a name that will highly unlikely appear in any test set. With this replacement, all the names in the test set will be unseen w.r.t. the training set. When trained on this perturbed training set, we can determine the extent to which the algorithms for learning coreference resolvers rely on memorizing seen names (as opposed to generalizing from their contexts) when performing MD and CR. Specifically, if a learner memorizes a lot,

Perturbation Type	Example
Unseen names	Mr. Smith → Mr. Htims
Names of a different type	John Smith → New York
Unseen nominals	activist → tsivitca
Nominals of a different type	actor → plane
Nominals of the same type	wife → grandmother
Unseen verbs	support → troppus
Seen verbs	acquire → believe
Unseen adj/adv	directly → yltcerid
Seen adj/adv	organic → shredded

Table 6: Perturbation examples.

it will likely perform poorly on MD (i.e., its recall will suffer) and subsequently CR.

We perform name replacement in a deterministic manner: we replace each word in a name with another word in which the order of its characters is reversed. Note that person prefixes (e.g., “Mr.”), organization words and suffixes (“Airlines”, “Inc.”), and location nouns (e.g., “River”) will not be replaced, as the goal is to introduce unseen names rather than change the type of a name.⁹ In addition, any word in a name that appears in a nominal mention in the training set will not be replaced. For instance, the word “Church” in “Baptist Church” appears in a nominal in the training set and therefore will not be replaced. This is done to ensure that only the “name” part of a mention will be changed to something that is not previously seen.¹⁰

Names of a different type. In this experiment, we replace each name, ne_1 , in a training document with another name, ne_2 , that satisfies two conditions. First, ne_2 , like ne_1 , should appear in the training set. This ensures that the number of names in the test set that will be unseen w.r.t. the train-

⁹These lists are available in the Appendix.

¹⁰Examples of this and other kinds of perturbations are shown in Table 6.

ing set will not change. Second, the two names should have different entity types. Importantly, the replacement is deterministic, meaning that (1) all occurrences of ne_1 will be replaced with the same name (i.e., ne_2), and (2) any name coreferent with ne_1 (but are not lexically identical to ne_1 , such as “Trump” and “President Trump”) will be replaced with a name coreferent with ne_2 . These conditions together ensure that only the names and their types will change, but their coreference relationships will not. Note that the choice of ne_2 is random subject to these conditions. Due to the randomness involved in the selection of ne_2 , we repeat the experiment three times and report the average result.

With this replacement, the resulting training documents may no longer make sense to a human reader, as a PERSON name may appear in a context for an ORGANIZATION name. In particular, the contexts in which a certain type of names (e.g., PERSON) appear in the training set will be different from those in which these names appear in the test set. This experiment will allow us to determine the extent to which a resolver makes use of contextual information when identifying coreference links involving names: if it makes heavy use of contextual information, we should see a considerable drop in resolver performance.

6.1.2 Perturbations to Nominal Mentions

We consider three kinds of perturbations to nominal mentions to determine the roles they play.

Unseen nominals. This experiment has the same setup as the “Unseen names” experiment above, except that we replace each nominal mention in the training set with another mention in which we reverse the order of the characters of each of its words. Note that this is a mention-internal perturbation, meaning that we replace all and only those nominals that are annotated as entity mentions, not all nominals in the training set.

Nominals of a different type. This experiment has the same setup as the “Names of a different type” experiment above, except that we replace nominal mentions rather than names. As in the previous experiment, we replace each nominal that is annotated as an entity mention in the training set.

Nominals of the same type. This experiment has the same setup as the “Nominals of a different type” experiment above, except that the nominal mention being replaced must have the same entity type as its replacement. This kind of perturbation

is “milder” than the previous kind of perturbation, as a PERSON mention will continue to appear in a PERSON context after the replacement. In other words, if a machine learner does not pay attention to the semantic compatibility between a nominal mention and its context, then we should see little performance difference when a resolver is trained on this training set vs. the previous training set (i.e., the one from “Nominals of a different type”).

6.2 Mention-external Perturbations

Mention-external perturbations involve making changes to the words outside a mention.

6.2.1 Perturbations to Verbs

We consider two kinds of perturbations to verbs to determine the role they play in resolution.

Unseen verbs. This experiment has the same setup as the two “Unseen” experiments above, except that we replace each verb in the training set that is not part of an entity mention.

Seen verbs. This experiment has the same setup as the “Names of a different type” experiment above, except that we replace verbs outside of entity mentions rather than names. In particular, the new verb is not constrained to have the same type as the verb being replaced: it can be any verb taken from the training set. Nevertheless, the replacement is deterministic: all occurrences of a given verb will be replaced with the same verb.

6.2.2 Perturbations to Adjectives & Adverbs

We consider two kinds of perturbations to adjectives and adverbs to determine the roles they play.

Unseen adjectives and adverbs. This experiment has the same setup as the three “Unseen” experiments above, except that we replace each adjective and adverb in the training set that is not part of an entity mention.

Seen adjectives and adverbs. This experiment has the same setup as the “Seen verbs” experiment above, except that we replace adjectives and adverbs outside of entity mentions rather than verbs.

6.3 Perturbation Results

Results of these experiments on the three datasets are shown in Table 7. As in Table 5, we only show the results of ELMo and SpanBERT-1. For each resolver, we show its CR CoNLL score and its MD F-score. To facilitate comparison, we show in row 1 the performance of the resolvers when the input is not perturbed.

Perturbation Type	ACE				OntoNotes				KBP			
	ELMo		SpanB-l		ELMo		SpanB-l		ELMo		SpanB-l	
	CR	MD	CR	MD	CR	MD	CR	MD	CR	MD	CR	MD
1 No Perturbation	71.8	90.4	78.9	92.3	73.0	85.1	79.6	88.2	69.7	85.3	75.8	88.5
2 NAM-Unseen	68.4	88.6	74.9	90.8	46.2	63.3	65.7	77.2	62.1	79.7	65.7	79.7
3 NAM-DiffType	69.5	89.5	75.7	91.1	58.8	75.2	72.7	83.5	64.3	82.5	67.2	81.4
4 NOM-Unseen	67.6	86.5	73.8	88.7	57.4	73.6	70.7	81.9	60.6	76.4	67.8	81.7
5 NOM-DiffType	69.7	88.6	74.7	89.6	59.0	75.4	71.2	82.1	66.0	82.5	70.7	83.6
6 NOM-SameType	69.9	89.2	75.3	90.1	62.5	79.2	72.8	83.4	66.8	83.4	72.1	84.9
7 Verb-Unseen	70.6	89.4	76.6	91.1	64.7	79.7	75.3	86.2	68.2	84.2	74.0	86.9
8 Verb-Seen	71.6	90.1	77.1	91.1	67.2	81.1	77.0	87.0	68.8	84.7	74.4	87.1
9 Adj/Adv-Unseen	71.7	89.9	76.8	91.0	68.9	82.3	77.4	87.4	69.3	85.3	73.5	86.4
10 Adj/Adv-Seen	69.8	88.8	76.9	91.3	70.1	83.0	76.3	86.7	68.5	84.8	74.0	86.6

Table 7: Perturbation results.

A few points deserve mention. First, mention-internal perturbations (rows 2–6) triggered larger deterioration in CR performance than mention-external perturbations. These results suggest that the resolvers rely more on the mentions themselves than their contexts for resolution, which should not be surprising. Among the mention-internal perturbations, the biggest CR performance drops occur with the Unseen perturbations (rows 2 and 4), particularly those involving unseen names, followed by perturbations involving the replacement of a seen name or nominal with a different type. A closer inspection of the results reveals that there is a strong correlation between CR performance and MD performance: larger drops in CR performance are always accompanied by larger drops in MD performance. This sheds light on why the Unseen perturbations triggered the largest drop in CR performance: when all the names or nominal mentions in the test set are not seen in the training set, the mention detector is likely to perform poorly on the test set. In contrast, when they are replaced by mentions of a different entity type, the percentage of unseen mentions in the test set doesn’t change, thus posing fewer problems for the mention detector.

As for the mention-external perturbations, no clear patterns emerged: while verb replacement (rows 7–8) has a greater impact than adjective and adverb replacement (rows 9–10) for OntoNotes, the same observation cannot be made for the other datasets. Moreover, while replacing a word with another seen word is generally expected to cause less harm to MD (and thus CR) performance than replacing a word with an unseen word, these experiments show that this is not necessarily the case. These results seem to suggest that the mention span learner is not particularly sensitive to the verbs, adjectives and adverbs that appear in the context.

Third, it is not easy to conclude which resolver

is more robust to perturbations. While the drops in CR performance on ACE are fairly mild for both resolvers, we see bigger CR performance drops on the other two datasets. In particular, ELMo suffers from a bigger drop in performance than SpanBERT-l on OntoNotes, whereas the reverse is true on KBP.

Finally, while the two resolvers’ MD performances are similar, SpanBERT-l’s CR performance is always superior to ELMo’s. These results reveal once again that the mention representations learned by SpanBERT-l are indeed better than those by ELMo as far as resolution is concerned.

7 Conclusions

While space limitations preclude a reiteration of all the observations we have made, we believe the key conclusions are: (1) the relative performances of the resolvers are consistent across datasets; (2) for each resolver, higher mention detection performance always yields better coreference performance; (3) the newest resolvers perform better because of not only improved mention detection, but also improved mention span representations, and they improved the resolution of both easy-and difficult-to-resolve anaphors; (4) all resolvers can be improved by improving mention detection, anaphoricity determination, and entity type detection; and (5) our perturbation results suggest that coreference performance is most sensitive to those words/phrases in the input that have the greatest impact on mention detection performance.

Acknowledgments

We thank the three anonymous reviewers for their comments. The paper’s title was motivated by that of Stoyanov et al.’s ACL 2009 paper — the credit goes to them. This work was supported in part by NSF Grants IIS-1528037 and CCF-1848608.

MUSTARD: MASTERING UNIFORM SYNTHESIS OF THEOREM AND PROOF DATA

Yinya Huang^{1,6} Xiaohan Lin² Zhengying Liu^{3†} Qingxing Cao^{2†} Huajian Xin²

Haiming Wang² Zhenguo Li³ Linqi Song^{1,6†} Xiaodan Liang^{2,4,5†}

¹City University of Hong Kong ²Shenzhen Campus of Sun Yat-sen University

³Huawei Noah’s Ark Lab ⁴DarkMatter AI Research ⁵MBZUAI ⁶CityUSRI

yinya.huang@hotmail.com, linxh55@mail2.sysu.edu.cn,
{liuzhengying2, Li.Zhenguo}@huawei.com, caoqx8@sysu.edu.cn,
linqi.song@cityu.edu.hk, xdliang328@gmail.com

ABSTRACT

Recent large language models (LLMs) have witnessed significant advancement in various tasks, including mathematical reasoning and theorem proving. As these two tasks require strict and formal multi-step inference, they are appealing domains for exploring the reasoning ability of LLMs but still face important challenges. Previous studies such as Chain-of-Thought (CoT) have revealed the effectiveness of intermediate steps guidance. However, such step-wise annotation requires heavy labor, leading to insufficient training steps for current benchmarks. To fill this gap, this work introduces MUSTARD, a data generation framework that masters uniform synthesis of theorem and proof data of high quality and diversity. MUSTARD synthesizes data in three stages: (1) It samples a few mathematical concept seeds as the problem category. (2) Then, it prompts a generative language model with the sampled concepts to obtain both the problems and their step-wise formal solutions. (3) Lastly, the framework utilizes a proof assistant (e.g., Lean Prover) to filter the valid proofs. With the proposed MUSTARD, we present a theorem-and-proof benchmark MUSTARDSAUCE with 5,866 valid data points. Each data point contains an informal statement, an informal proof, and a translated formal proof that passes the prover validation. We perform extensive analysis and demonstrate that MUSTARD generates validated high-quality step-by-step data. We further apply the MUSTARDSAUCE for fine-tuning smaller language models. The fine-tuned Llama 2-7B achieves a 15.41% average relative performance gain in automated theorem proving, and 8.18% in math word problems¹.

1 INTRODUCTION

Large language models (LLMs) (OpenAI, 2023; 2022) have shown promising reasoning capabilities in various domains, including math word problem and theorem proving (Cobbe et al., 2021; Hendrycks et al., 2021; Zheng et al., 2022; Wu et al., 2021). These two tasks, which require strictly and successively multi-step inference, have become appeal domains to evaluate and develop LLMs’ ability in complex reasoning. Recent works progress LLMs in solving math problems mainly through two techniques. The first is the chain-of-thoughts (CoT) prompting (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023b), which provides step-by-step solutions to the LLMs. The second is to leverage the LLMs’ ability in code generation to generate formalized languages and utilize external solvers to obtain strict inference results (Wu et al., 2022; Jiang et al., 2023; Polu & Sutskever, 2020; Han et al., 2022; Polu et al., 2023). Both techniques rely on step-wise annotation to improve LLMs’ performance and interpretability on the math problem.

Correct intermediate steps are crucial for LLMs to perform complex reasoning. However, high-quality step-wise annotations are hard to obtain, and Figure 1 demonstrates a few representative

[†]Corresponding author.

¹Codes and data are available at: <https://github.com/Eleanor-H/MUSTARD>

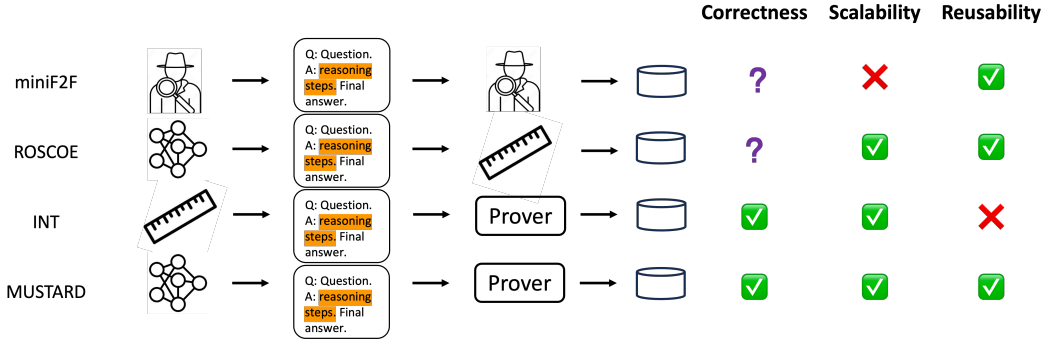


Figure 1: A comparison of methods of synthesizing and validating intermediate reasoning steps.

works. Previous works such as miniF2F (Zheng et al., 2022) resorts to manual annotation and validation to obtain high-quality step-wise labels. However, manual annotation requires heavy labor of knowledgeable experts, resulting in an extremely small-scale dataset. Manual checking also does not guarantee the correctness of data as the labelers would make mistakes in labeling. On the other hand, generating data with rule-based checking such as ROSCOE (Golovneva et al., 2023) can produce large-scale reasoning data. Given that the generated data are more friendly and readable for human beings, the correctness of the reasoning is not guaranteed by those rules. Moreover, another line of work such as INT (Wu et al., 2021) performs rule-based synthesis to generate validated proofs, which are both correct and large-scale. However, the data are brutally synthesized so that many generated proofs lack actual meaning. Therefore, we need a more efficient way to generate mathematical data that are large-scale, with accurate intermediate steps, and also meaningful mathematical knowledge to human beings.

To fill this gap, we propose MUSTARD, a data generation framework that uniformly synthesizes large-scale and high-quality mathematical data by combining the advantages of LLMs in verbalization and formal theorem provers in rigorous data validation. Specifically, MUSTARD first samples a few mathematical concepts from a predefined list and prompts an LLM to generate a related question described in natural language. Then, it applies the LLM to generate the corresponding solution in both natural and formal language. Given the generated solution, MUSTARD further validates them using a theorem prover. The passed one is considered to be correct and is a high-quality data point. The invalid one on the other hand is considered to be a challenging sample, which will be further combined with the error messages to prompt the LLM for a solution revision, and added as a challenging data point.

By applying the proposed MUSTARD one can obtain large amounts of problems and theorems with desired mathematical concepts and domains. Eventually, we build a mathematical dataset with validated informal and formal solutions, named MUSTARDSAUCE (MUSTARD resource).

We conduct extensive data analysis and experiments on the generated MUSTARDSAUCE. Through deep inspection of the data, we find that MUSTARD generates interesting and reasonable math problems by creatively combining two mathematical concepts, and MUSTARDSAUCE is diverse and has a high proportion of difficult data. We also observe that the prover is consistent with human evaluation, where humans usually consider a validated solution to have a higher quality than those without a formal validation process. Lastly, we fine-tune smaller-scale language models on MUSTARDSAUCE. The fine-tuned Llama 2-7B achieves improvements by 20.9% on zero-shot inference on GSM8K and achieves 8.7 of pass@1 on mathlib. These results demonstrate the effectiveness of MUSTARDSAUCE in improving the mathematical reasoning capabilities of language models.

The contributions of this paper are summarized as follows:

1. We propose a novel framework MUSTARD that can generate high-quality mathematical data (both informal and formal) with an interplay between generative language model and theorem prover assistants.
2. We release the MUSTARDSAUCE, which contains both math word problems and theorem-proving problems spanning over four educational levels. Each sample has corresponding informal and formal solutions.
3. We conduct extensive analysis and experiments on the generated data, demonstrating their quality, diversity, and effectiveness in improving language models’ mathematical reasoning performance.

2 RELATED WORKS

Large Language Models for Mathematical Reasoning The growing generative language models (Brown et al., 2020; OpenAI, 2022, 2023) show compelling potential for solving mathematical problems both in natural language proofs (OpenAI, 2023; Zheng et al., 2023a; Xiong et al., 2024) and in formal languages with theorem provers (Polu & Sutskever, 2020; Han et al., 2022; Zheng et al., 2023b; Polu et al., 2023). On the other hand, some works explore using language models to automatically translate natural language proofs into formal ones given few-shot demonstrations (Wu et al., 2022; Jiang et al., 2023; Liu et al., 2023; Wang et al., 2024). Chain-of-thought reasoning (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023b) is demonstrated beneficial for the LLMs to derive correct answers. However, some recent works (Saparov & He, 2023; Golovneva et al., 2023) observe that the intermediate reasoning steps can be inconsistent. This paper proposes a data generation framework that taps the comprehensive mathematical reasoning capabilities of large language models. It generates mathematical reasoning problems with informal and formal solutions that are step-wise validated by a formal theorem prover. With the framework, we obtain high-quality mathematical data.

Synthesizing Mathematical Data Obtaining large-scale high-quality mathematical data is a long-standing challenge. Previous data relies on well-trained annotators to hand-craft and review the formal proofs (Zheng et al., 2022), which is time and labour-consuming and results in a small data scale. Wang & Deng (2020) and Xiong et al. (2023) construct a neural generator for data synthesis, but it still requires the intervention of human-written data. Besides, Wu et al. (2021) explore using a theorem generator to automatically generate formal proofs with rules. However, the rule-based generation depends on given axioms in specified orders. As a result, the generated data is restricted to a few domains. On the other hand, recent works demonstrate the effectiveness of distilling knowledge from large language models (West et al., 2022; Yuan et al., 2023; Li et al., 2023), and some of them (Wang et al., 2023c; Xu et al., 2023) explore data evolution by properly prompting the language models. The proposed framework explores eliciting mathematical knowledge from large language models to achieve diverse and large-scale mathematical data. In this framework, an interplay between the language model and a formal proof assistant controls the quality and difficulties of data. Using the proposed framework, we collect a large-scale mathematical dataset that contains diverse and multiple-difficulty math questions with high-quality solutions.

3 MUSTARD

In this work, we aim to obtain large-scale mathematical data with multi-step annotations and propose MUSTARD to generate diverse and high-quality math and theorem-proving problems with multi-step informal and formal solutions. As shown in Figure 2, MUSTARD consists of three stages. In the first concept seeding stage, MUSTARD samples a set of math concepts as the problem domain. Then in the second solution generation stage, it generates the concept-related problem and solution by prompting an LLM. In the third stage, a theorem prover is used to validate the generated solution. If the solution can not pass the prover, the error message is returned to the second stage for another turn of solution generation. Through interaction between the LLM and a formal proof assistant, MUSTARD can generate diverse and high-quality data that contains both informal and formal solutions. We describe the details of each stage in this section.

3.1 CONCEPT SEEDING

We first define and build a mathematical concept pool that covers as complete sub-subjects in mathematics and educational levels as possible. Specifically, we collect all math courses on the Khan Academy website², the large-scale online educational platform. The resulting pool includes concepts in four educational levels: elementary school, middle school, high school, and higher education. Each educational level has 5 to 9 math domains, covering different types of math problems such as algebra and geometry. Each domain contains subdivided mathematical concepts to inspect different mathematical abilities like polynomial arithmetic or factorization. Concept statistics and detailed concepts in each domain are demonstrated in Appendix B.

Given the concept pool, for each educational level, MUSTARD uniformly samples 1 or 2 concepts from all domains as seeds, and then generates mathematical problems that cover the concepts. In particular, given an educational level, taking 2 concepts from different subjects challenges the model to generate problems that join diverse domains while keeping the problems reasonable.

²<https://www.khanacademy.org/math>

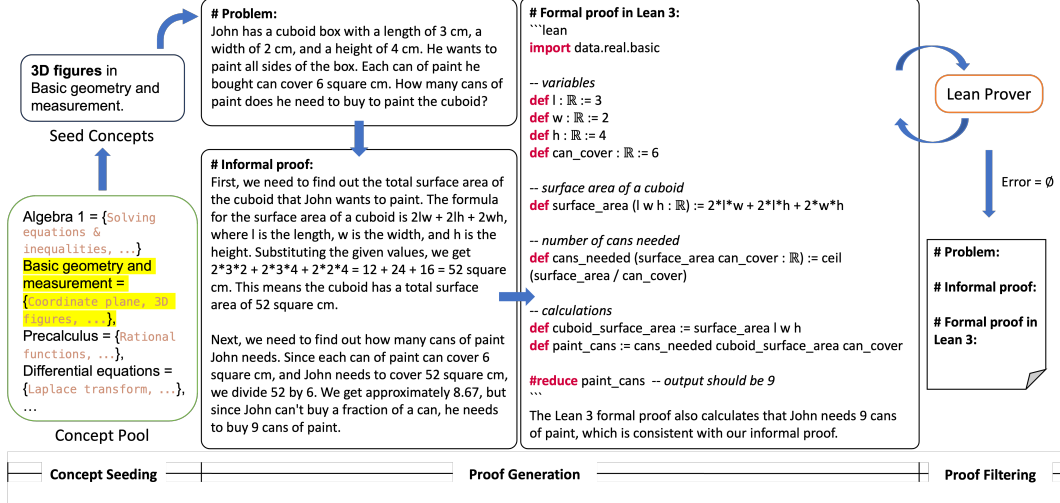


Figure 2: Overview of MUSTARD. The mathematical theorem (indicated by “# Problem:”) is created according to the randomly selected seed concepts. Following the theorem, a corresponding informal proof is generated, which is then translated into a formal proof in Lean 3. The formal proof is passed into Lean Prover for quality feedback, and according to the error messages the formal proof is revised and improved. The proofs that pass the Lean Prover are regarded as high-quality data points and collected.

3.2 PROOF GENERATION

Given the sampled mathematical concepts, MUSTARD generates math problems and their corresponding solutions. Specifically, MUSTARD leverages the capability of LLMs in generating natural language and code and prompts an LLM to generate the problem statement, its natural language solution, and a formal solution written in Lean. As a result, the LLM needs to complete the following three tasks: (T1) Generating a math problem that relates to the given concepts; (T2) Solving the math problem with a natural language proof; (T3) Performing auto-formalization to translate the written natural language proof into a formalized proof. In this work, we use GPT-4 [OpenAI \(2023\)](#) as the LLM for proof generation.

We intend to generate a problem based on educational level, math domains, and concepts. Considering that mathematical problems include proof and calculation, we also introduce the question types into the prompt for generating theorem proving and word problems respectively. Moreover, we do not include any exemplars or other manual interventions except for the sampled concepts. We intend to avoid potential biases brought by the concepts inside the exemplars and achieve a more diverse generation. The prompt template is shown as the following:

You are a math expert. Now please come up with a math problem according to the following requirements. The math problem should contain a question part (indicated by “# Problem:”), a corresponding solution in natural language (indicated by “# Informal proof:”), and a translated formal solution in Lean 3 (indicated by “# Formal proof in Lean 3:”). Please note that the informal proof and the formal proof need to be identical.

Please create a [QUESTION TYPE] in the level of [EDUCATIONAL LEVEL] based on the following knowledge point(s): [CONCEPT] in [DOMAIN]; [CONCEPT] in [DOMAIN].

You must respond in the following format:

```
# Problem: ...  
# Informal proof: ...  
# Formal proof in Lean 3: ...
```

The “[]” indicates the placeholders for the corresponding question type, educational level, concepts, and domains. Multiple concepts are separated by “;”. We retrieved the text after the “Problem:”, “Informal proof:” and “Formal proof in Lean 3:” as the generated sample.

3.3 PROOF FILTERING

In the proof-filtering stage, MUSTARD interacts with the Lean Prover (de Moura et al., 2015) to obtain validation messages of the proof steps, which guides data revision and filtering. Specifically, after a formal solution is passed to the Lean Prover and if the prover returns no error message, the corresponding data point is collected into the valid dataset. Otherwise, MUSTARD collects the error messages from the prover and prompts the language model to revise the invalid solution. To help the language model locate the incorrect lines described in the error messages, we also add a line number at the beginning of each line in the formal solutions. The verification and self-refinement are performed in multiple rounds until LLM generates a valid solution. We use the number of rounds to measure the difficulty of the generated sample, assuming a difficult problem is hard to solve by an LLM and requires more rounds of correction. The prompt template of a single round of correction is demonstrated as follows, and the complete prompt template is shown in Table I3 in Appendix C.1:

```
# Formal proof (c) in Lean 3:
```lean
line 1 <code>
line 2 <code>
line 3 <code>
...
```

# Error messages for Formal proof (c) from Lean Prover:
<error messages>
```

4 EXPERIMENTS

4.1 CASE STUDY

We first inspect the data points generated by MUSTARD. Table 1 shows a generated math problem in which MUSTARD creatively combines two mathematical concepts and constructs a reasonable question. The generated question includes knowledge from both concepts. It is suggested that MUSTARD can join the concepts and construct a reasonable question. Furthermore, Table 2 demonstrates a case that MUSTARD provides solid and comprehensive solutions in both natural language and Lean. Although the formal proof is long, it is consistent and passes the prover’s validation. It is demonstrated that MUSTARD can generate long valid solutions.

4.2 HUMAN EVALUATION

To further explore the quality of the data generated by MUSTARD, we recruit professionals who have expertise in mathematics and the Lean language to perform a sanity check on the data points. We randomly select 200 data points from the generated data, 100 of which pass the Lean Prover (Group Valid) and 100 of which do not (Group Invalid). The sanity check covers the four sections in each data point (i.e., informal statement, informal proof, formal statement, and formal proof), and includes factuality check and consistency check. Specifically, a high-quality data point should have a factually correct informal statement (D1) and a correct solution (D4). The formal statement and proof should be aligned with the informal descriptions (D5, D6). Moreover, the desired data point should meet the specified seed concepts (D2) and question type (D3). The six inspection dimensions and their requirements are demonstrated in Table 3. A data point is scored 1 in a dimension if it meets the requirement, otherwise, it gets 0.

The accuracies of Group Valid and Group Invalid in each dimension are demonstrated in Table 3. We also report the corresponding p -value in each dimension. (D4) and (D6) show significant differences in accuracy between the two groups. The results indicate that high-quality data points have significantly better auto-formalization results. As a result, given the validation of formal proofs by the Lean Prover, the data points have guaranteed high-quality informal proofs. Moreover, (D1) also shows significance with the inspected data scaled up. The differences in statement alignment (D5) and informal statement relevance (D2) of the two groups are less significant. Furthermore, no significant differences are observed in question type classification (D3), which indicates that Lean Prover validation in MUSTARD does not significantly influence the classification. Overall, the human evaluation results suggest that formally validated data have significantly higher quality.

4.3 DATA QUALITY BY DOWNSTREAM APPLICATION

To evaluate the impact of MUSTARDSAUCE on enhancing mathematical reasoning abilities, we use the data to fine-tune smaller-scale language models and evaluate them on math word problems (MWP) and automated theorem proving (ATP). Specifically, given all the generated data, we first

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; ^{*}New York University;
plewis@fb.com

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory can overcome this issue, but have so far been only investigated for extractive downstream tasks. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) — models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We compare two RAG formulations, one which conditions on the same retrieved passages across the whole generated sequence, and another which can use different passages per token. We fine-tune and evaluate our models on a wide range of knowledge-intensive NLP tasks and set the state of the art on three open domain QA tasks, outperforming parametric seq2seq models and task-specific retrieve-and-extract architectures. For language generation tasks, we find that RAG models generate more specific, diverse and factual language than a state-of-the-art parametric-only seq2seq baseline.

1 Introduction

Pre-trained neural language models have been shown to learn a substantial amount of in-depth knowledge from data [47]. They can do so without any access to an external memory, as a parameterized implicit knowledge base [51, 52]. While this development is exciting, such models do have downsides: They cannot easily expand or revise their memory, can’t straightforwardly provide insight into their predictions, and may produce “hallucinations” [38]. Hybrid models that combine parametric memory with non-parametric (i.e., retrieval-based) memories [20, 26, 48] can address some of these issues because knowledge can be directly revised and expanded, and accessed knowledge can be inspected and interpreted. REALM [20] and ORQA [31], two recently introduced models that combine masked language models [8] with a differentiable retriever, have shown promising results,

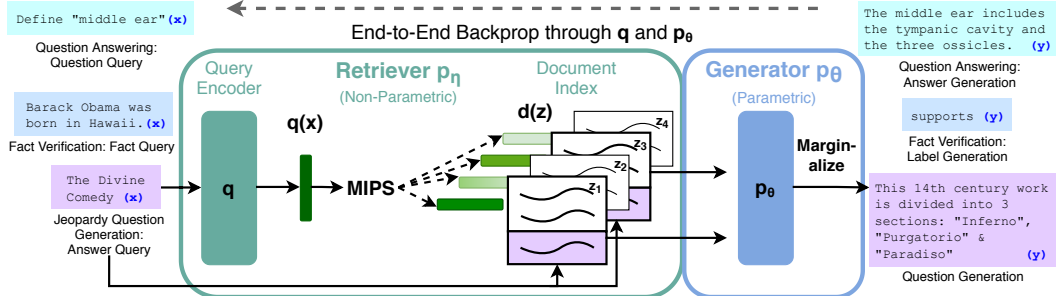


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i . For final prediction y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

but have only explored open-domain extractive question answering. Here, we bring hybrid parametric and non-parametric memory to the “workhorse of NLP,” i.e. sequence-to-sequence (seq2seq) models.

We endow pre-trained, parametric-memory generation models with a non-parametric memory through a general-purpose fine-tuning approach which we refer to as retrieval-augmented generation (RAG). We build RAG models where the parametric memory is a pre-trained seq2seq transformer, and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We combine these components in a probabilistic model trained end-to-end (Fig. 1). The retriever (Dense Passage Retriever [26], henceforth DPR) provides latent documents conditioned on the input, and the seq2seq model (BART [32]) then conditions on these latent documents together with the input to generate the output. We marginalize the latent documents with a top-K approximation, either on a per-output basis (assuming the same document is responsible for all tokens) or a per-token basis (where different documents are responsible for different tokens). Like T5 [51] or BART, RAG can be fine-tuned on any seq2seq task, whereby both the generator and retriever are jointly learned.

There has been extensive previous work proposing architectures to enrich systems with non-parametric memory which are trained from scratch for specific tasks, e.g. memory networks [64, 55], stack-augmented networks [25] and memory layers [30]. In contrast, we explore a setting where both parametric and non-parametric memory components are pre-trained and pre-loaded with extensive knowledge. Crucially, by using pre-trained access mechanisms, the ability to access knowledge is present without additional training.

Our results highlight the benefits of combining parametric and non-parametric memory with generation for *knowledge-intensive tasks*—tasks that humans could not reasonably be expected to perform without access to an external knowledge source. Our RAG models achieve state-of-the-art results on open Natural Questions [29], WebQuestions [3] and CuratedTrec [2] and strongly outperform recent approaches that use specialised pre-training objectives on TriviaQA [24]. Despite these being extractive tasks, we find that unconstrained generation outperforms previous extractive approaches. For knowledge-intensive generation, we experiment with MS-MARCO [1] and Jeopardy question generation, and we find that our models generate responses that are more factual, specific, and diverse than a BART baseline. For FEVER [56] fact verification, we achieve results within 4.3% of state-of-the-art pipeline models which use strong retrieval supervision. Finally, we demonstrate that the non-parametric memory can be replaced to update the models’ knowledge as the world changes.¹

2 Methods

We explore RAG models, which use the input sequence x to retrieve text documents z and use them as additional context when generating the target sequence y . As shown in Figure 1, our models leverage two components: (i) a retriever $p_\eta(z|x)$ with parameters η that returns (top-K truncated) distributions over text passages given a query x and (ii) a generator $p_\theta(y_i|x, z, y_{1:i-1})$ parametrized

¹Code to run experiments with RAG has been open-sourced as part of the HuggingFace Transformers Library [66] and can be found at <https://github.com/huggingface/transformers/blob/master/examples/rag/>. An interactive demo of RAG models can be found at <https://huggingface.co/rag/>

by θ that generates a current token based on a context of the previous $i - 1$ tokens $y_{1:i-1}$, the original input x and a retrieved passage z .

To train the retriever and generator end-to-end, we treat the retrieved document as a latent variable. We propose two models that marginalize over the latent documents in different ways to produce a distribution over generated text. In one approach, *RAG-Sequence*, the model uses the same document to predict each target token. The second approach, *RAG-Token*, can predict each target token based on a different document. In the following, we formally introduce both models and then describe the p_η and p_θ components, as well as the training and decoding procedure.

2.1 Models

RAG-Sequence Model The RAG-Sequence model uses the same retrieved document to generate the complete *sequence*. Technically, it treats the retrieved document as a single latent variable that is marginalized to get the seq2seq probability $p(y|x)$ via a top-K approximation. Concretely, the top K documents are retrieved using the retriever, and the generator produces the output sequence probability for each document, which are then marginalized,

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) \prod_i^N p_\theta(y_i|x, z, y_{1:i-1})$$

RAG-Token Model In the RAG-Token model we can draw a different latent document for each target *token* and marginalize accordingly. This allows the generator to choose content from several documents when producing an answer. Concretely, the top K documents are retrieved using the retriever, and then the generator produces a distribution for the next output token for each document, before marginalizing, and repeating the process with the following output token. Formally, we define:

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z_i, y_{1:i-1})$$

Finally, we note that RAG can be used for sequence classification tasks by considering the target class as a target sequence of length one, in which case RAG-Sequence and RAG-Token are equivalent.

2.2 Retriever: DPR

The retrieval component $p_\eta(z|x)$ is based on DPR [26]. DPR follows a bi-encoder architecture:

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{BERT}_d(z), \quad \mathbf{q}(x) = \text{BERT}_q(x)$$

where $\mathbf{d}(z)$ is a dense representation of a document produced by a $\text{BERT}_{\text{BASE}}$ *document encoder* [8], and $\mathbf{q}(x)$ a query representation produced by a *query encoder*, also based on $\text{BERT}_{\text{BASE}}$. Calculating $\text{top-}k(p_\eta(\cdot|x))$, the list of k documents z with highest prior probability $p_\eta(z|x)$, is a Maximum Inner Product Search (MIPS) problem, which can be approximately solved in sub-linear time [23]. We use a pre-trained bi-encoder from DPR to initialize our retriever and to build the document index. This retriever was trained to retrieve documents which contain answers to TriviaQA [24] questions and Natural Questions [29]. We refer to the document index as the *non-parametric memory*.

2.3 Generator: BART

The generator component $p_\theta(y_i|x, z, y_{1:i-1})$ could be modelled using any encoder-decoder. We use BART-large [32], a pre-trained seq2seq transformer [58] with 400M parameters. To combine the input x with the retrieved content z when generating from BART, we simply concatenate them. BART was pre-trained using a denoising objective and a variety of different noising functions. It has obtained state-of-the-art results on a diverse set of generation tasks and outperforms comparably-sized T5 models [32]. We refer to the BART generator parameters θ as the *parametric memory* henceforth.

2.4 Training

We jointly train the retriever and generator components without any direct supervision on what document should be retrieved. Given a fine-tuning training corpus of input/output pairs (x_j, y_j) , we

minimize the negative marginal log-likelihood of each target, $\sum_j -\log p(y_j|x_j)$ using stochastic gradient descent with Adam [28]. Updating the document encoder BERT_d during training is costly as it requires the document index to be periodically updated as REALM does during pre-training [20]. We do not find this step necessary for strong performance, and keep the document encoder (and index) fixed, only fine-tuning the query encoder BERT_q and the BART generator.

2.5 Decoding

At test time, RAG-Sequence and RAG-Token require different ways to approximate $\arg \max_y p(y|x)$.

RAG-Token The RAG-Token model can be seen as a standard, autoregressive seq2seq generator with transition probability: $p'_\theta(y_i|x, y_{1:i-1}) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z_i|x) p_\theta(y_i|x, z_i, y_{1:i-1})$. To decode, we can plug $p'_\theta(y_i|x, y_{1:i-1})$ into a standard beam decoder.

RAG-Sequence For RAG-Sequence, the likelihood $p(y|x)$ does not break into a conventional per-token likelihood, hence we cannot solve it with a single beam search. Instead, we run beam search for each document z , scoring each hypothesis using $p_\theta(y_i|x, z, y_{1:i-1})$. This yields a set of hypotheses Y , some of which may not have appeared in the beams of all documents. To estimate the probability of an hypothesis y we run an additional forward pass for each document z for which y does not appear in the beam, multiply generator probability with $p_\eta(z|x)$ and then sum the probabilities across beams for the marginals. We refer to this decoding procedure as “Thorough Decoding.” For longer output sequences, $|Y|$ can become large, requiring many forward passes. For more efficient decoding, we can make a further approximation that $p_\theta(y|x, z_i) \approx 0$ where y was not generated during beam search from x, z_i . This avoids the need to run additional forward passes once the candidate set Y has been generated. We refer to this decoding procedure as “Fast Decoding.”

3 Experiments

We experiment with RAG in a wide range of knowledge-intensive tasks. For all experiments, we use a single Wikipedia dump for our non-parametric knowledge source. Following Lee et al. [31] and Karpukhin et al. [26], we use the December 2018 dump. Each Wikipedia article is split into disjoint 100-word chunks, to make a total of 21M documents. We use the document encoder to compute an embedding for each document, and build a single MIPS index using FAISS [23] with a Hierarchical Navigable Small World approximation for fast retrieval [37]. During training, we retrieve the top k documents for each query. We consider $k \in \{5, 10\}$ for training and set k for test time using dev data. We now discuss experimental details for each task.

3.1 Open-domain Question Answering

Open-domain question answering (QA) is an important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output text pairs (x, y) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA paradigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also compare to “Closed-Book QA” approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24], WebQuestions (WQ) [3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on the TQA Wiki test set.

3.2 Abstractive Question Answering

RAG models can go beyond simple extractive QA and answer questions with free-form, abstractive text generation. To test RAG’s natural language generation (NLG) in a knowledge-intensive setting, we use the MSMARCO NLG task v2.1 [43]. The task consists of questions, ten gold passages retrieved from a search engine for each question, and a full sentence answer annotated from the retrieved passages. We do not use the supplied passages, only the questions and answers, to treat

Applying BERT to Document Retrieval with Birch

Zeynep Akkalyoncu Yilmaz, Shengjin Wang, Wei Yang, Haotian Zhang, and Jimmy Lin

David R. Cheriton School of Computer Science
University of Waterloo

Abstract

We present Birch, a system that applies BERT to document retrieval via integration with the open-source Anserini information retrieval toolkit to demonstrate end-to-end search over large document collections. Birch implements simple ranking models that achieve state-of-the-art effectiveness on standard TREC newswire and social media test collections. This demonstration focuses on technical challenges in the integration of NLP and IR capabilities, along with the design rationale behind our approach to tightly-coupled integration between Python (to support neural networks) and the Java Virtual Machine (to support document retrieval using the open-source Lucene search library). We demonstrate integration of Birch with an existing search interface as well as interactive notebooks that highlight its capabilities in an easy-to-understand manner.

1 Introduction

The information retrieval community, much like the natural language processing community, has witnessed the growing dominance of approaches based on neural networks. Applications of neural networks to document ranking usually involve multi-stage architectures, beginning with a traditional term-matching technique (e.g., BM25) over a standard inverted index, followed by a reranker that rescores the candidate list of documents (Asadi and Lin, 2013).

Researchers have developed a panoply of neural ranking models—see Mitra and Craswell (2019) for a recent overview—but there is emerging evidence that BERT (Devlin et al., 2019) outperforms previous approaches to document retrieval (Yang et al., 2019c; MacAvaney et al., 2019) as well as search-related tasks such as question answering (Nogueira and Cho, 2019; Yang et al., 2019b).

We share with the community Birch,¹ which integrates the Anserini information retrieval toolkit² with a BERT-based document ranking model that provides an end-to-end open-source search engine. Birch allows the community to replicate the state-of-the-art document ranking results presented in Yilmaz et al. (2019) and Yang et al. (2019c). Here we summarize those results, but our focus is on system architecture and the rationale behind a number of implementation design decisions, as opposed to the ranking model itself.

2 Integration Challenges

The problem we are trying to solve, and the focus of this work, is how to bridge the worlds of information retrieval and natural language processing from a software engineering perspective, for applications to document retrieval. Following the standard formulation, we assume a (potentially large) corpus D that users wish to search. For a keyword query Q , the system’s task is to return a ranked list of documents that maximizes a retrieval metric such as average precision (AP). This stands in contrast to reading comprehension tasks such as SQuAD (Rajpurkar et al., 2016) and many formulations of question answering today such as WikiQA (Yang et al., 2015) and the MS MARCO QA Task (Bajaj et al., 2018), where there is no (or minimal) retrieval component. These are better characterized as “selection” tasks on (pre-determined) text passages.

Within the information retrieval community, there exists a disconnect between academic researchers and industry practitioners. Outside of a few large organizations that deploy custom infrastructure (mostly commercial search engine companies), Lucene (along with the closely-related

¹<http://birchir.io/>

²<http://anserini.io/>

projects Solr and Elasticsearch) has become the de facto platform for building real-world search applications, deployed at Twitter, Netflix, eBay, and numerous other organizations. However, many researchers still rely on academic systems such as Indri³ and Terrier,⁴ which are mostly unknown in real-world production environments. This gap hinders technology transfer and the potential impact of research results.

Even assuming Lucene as a “common denominator” that academic researchers learn to adopt, there is still one technical hurdle: Lucene is implemented in Java, and hence runs on the Java Virtual Machine (JVM). However, most deep learning toolkits today, including TensorFlow and PyTorch, are written in Python with a C++ backend. Bridging Python and the JVM presents a technical challenge for NLP/IR integration.

3 Birch

3.1 Anserini

Anserini (Yang et al., 2017, 2018) represents an attempt to better align academic researchers with industry practitioners by building a research-focused toolkit on top of the open-source Lucene search library. Further standardizing on a common platform within the academic community can foster greater replicability and reproducibility, a growing concern in the community (Lin et al., 2016).

Already, Anserini has proven to be effective and has gained some traction: For example, Nogueira and Cho (2019) used Anserini for generating candidate documents before applying BERT to ranking passages in the TREC Complex Answer Retrieval (CAR) task (Dietz et al., 2017), which led to a large increase in effectiveness. Yang et al. (2019b) also combined Anserini and BERT to demonstrate large improvements in open-domain question answering directly on Wikipedia.

3.2 Design Decisions

The architecture of Birch is shown in Figure 1, which codifies a two-stage pipeline architecture where Anserini is responsible for retrieval, the output of which is passed to a BERT-based reranker. Since our research group has standardized on PyTorch, the central challenge we tackle is: How do we integrate the deep learning toolkit with Anserini?

³<https://www.lemurproject.org/>

⁴<http://terrier.org/>

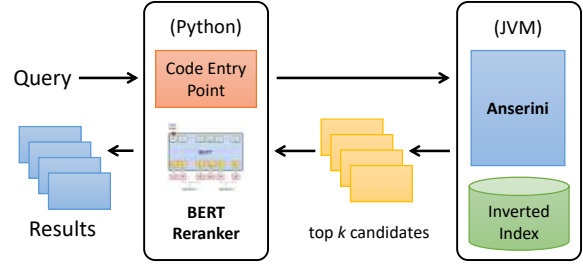


Figure 1: Architecture of Birch, illustrating a tight integration between Python and the Java Virtual Machine. The main code entry point is in Python, which calls Anserini for retrieval; candidate documents from Anserini are then reranked by our BERT models.

At the outset, we ruled out “loosely-coupled” integration approaches: For example, passing intermediate text files is not a sustainable solution in the long term. It is not only inefficient, but interchange formats frequently change (whether intentionally or accidentally), breaking code between multiple components. We also ruled out integration via REST APIs for similar reasons: efficiency (overhead of HTTP calls) and stability (imperfect solutions for enforcing API contracts, particularly in a research environment).

There are a few options for the “tightly-coupled” integration we desired. In principle, we could adopt the Java Virtual Machine (JVM) as the primary code entry point, with integration to the Torch backend via JNI, but this was ruled out because it would create two separate code paths (JVM to C++ for execution and Python to C++ for model development), which presents maintainability issues. After some exploration, we decided on Python as the primary development environment, integrating Anserini using the Pyjnius Python library⁵ for accessing Java classes. The library was originally developed to facilitate Android development in Python, and allows Python code to directly manipulate Java classes and objects. Thus, Birch supports Python as the main development language (and code entry point, as shown in Figure 1), connecting to the backend JVM to access retrieval capabilities.

3.3 Models

Our document ranking approach is detailed in Yilmaz et al. (2019) and Yang et al. (2019c). We follow Nogueira and Cho (2019) in adapting BERT for binary (specifically, relevance) classification over text. Candidate documents from Anserini

⁵<https://pyjnius.readthedocs.io/>

Model	2011		2012		2013		2014	
	AP	P@30	AP	P@30	AP	P@30	AP	P@30
QL	0.3576	0.4000	0.2091	0.3311	0.2532	0.4450	0.3924	0.6182
RM3	0.3824	0.4211	0.2342	0.3452	0.2766	0.4733	0.4480	0.6339
MP-HCNN (Rao et al., 2019)	0.4043	0.4293	0.2460	0.3791	0.2896	0.5294	0.4420	0.6394
BiCNN (Shi et al., 2018)	0.4293	0.4728	0.2621	0.4147	0.2990	0.5367	0.4563	0.6806
Birch	0.4697	0.5040	0.3073	0.4356	0.3357	0.5656	0.5176	0.7006

Table 1: Results on test collections from the TREC Microblog Tracks, comparing BERT with selected neural ranking models. The first two blocks of the table contain results copied from Rao et al. (2019).

are processed individually. As model input, we concatenate the query Q and document D into a text sequence $[[CLS], Q, [SEP], D, [SEP]]$, and then pad each text sequence in a mini-batch to N tokens, where N is the maximum length in the batch. The $[CLS]$ vector is then taken as input to a single layer neural network. Starting from a pre-trained BERT model, we fine-tune with existing relevance judgments using cross-entropy loss. BERT inference scores are then combined with the original retrieval scores, in the simplest case, using linear interpolation.

In this simple approach, long documents pose a problem since BERT wasn’t specifically designed to perform inference on long input texts. We present a simple solution: inference is applied over each sentence in a candidate document and sentence-level evidence is aggregated for ranking documents as follows:

$$S_f = a \cdot S_{doc} + (1 - a) \cdot \sum_{i=1}^n w_i \cdot S_i \quad (1)$$

where S_{doc} is the original document score and S_i is the i -th top-scoring sentence according to BERT; a and w_i ’s are parameters that need to be learned. In practice, we only consider up to the three top-scoring sentences in each document.

The intuition behind this approach comes from Zhang et al. (2018b,a), who found that the “best” sentence or paragraph in a document provides a good proxy for document relevance. From a different perspective, we are essentially implementing a form of passage retrieval.

4 Retrieval Results

4.1 TREC 2011–2014 Microblog Tracks

As originally reported in Yang et al. (2019c), Birch was evaluated on tweet test collections from the TREC Microblog Tracks, 2011 to 2014 (Lin et al.,

2014). Since tweets are short, relevance judgments can be directly used to fine-tune the BERT model (Section 3.3). For evaluation on each year’s dataset, we used the remaining years for fine-tuning, e.g., tuning on 2011–2013 data, testing on 2014 data. Additional details on the fine-tuning strategy and experimental settings are described in Yang et al. (2019c).

At retrieval (inference) time, query likelihood (QL) with RM3 relevance feedback (Nasreen et al., 2004) was used to provide the initial pool of candidates (to depth 1000). Since tweets are short, we can apply inference over each candidate document in its entirety. The interpolation weight between the BERT scores and the retrieval scores was tuned on the validation data.

Experimental results are shown in Table 1, where we present average precision (AP) and precision at rank 30 (P@30), the two official metrics of the evaluation (Ounis et al., 2011). The first two blocks of the table are copied from Rao et al. (2019), who compared bag-of-words baselines (QL and RM3) to several popular neural ranking models as well as MP-HCNN, the model they introduced. The results of Rao et al. (2019) were further improved in Shi et al. (2018); in all cases, the neural models include interpolation with the original document scores. We see that Birch yields a large jump in effectiveness across all Microblog collections.

4.2 TREC 2004 Robust Track

In addition to searching short social media posts, we also examined a “traditional” document retrieval task over newswire articles. For this, we used the test collection from the TREC 2004 Robust Track (Voorhees, 2004), which comprises 250 topics over a newswire corpus of around 500K documents. Here, we provide a summary of Yilmaz et al. (2019), which contains more detailed

Model	AP	P@20	NDCG@20
BM25+RM3	0.2903	0.3821	0.4407
1S: BERT	0.3676	0.4610	0.5239
2S: BERT	0.3697	0.4657	0.5324
3S: BERT	0.3691	0.4669	0.5325

Table 2: Results on Robust04, where n S denotes combining scores from the top n sentences in a document.

descriptions of our approach and presents experiments on more test collections.

The additional challenge with ranking newswire articles is the lack of training data to fine-tune the BERT models, since relevance judgments are provided at the document level. That is, in the standard formulation of document ranking, a document is considered relevant if *any* part of it is relevant—but documents are typically longer than the lengths of text BERT was designed to handle. The surprising finding of Yilmaz et al. (2019) is that BERT models fine-tuned with the Microblog test collections in Section 4.1 can be *directly* applied to rank newswire documents, despite the differences in domain (social media posts vs. news articles). Furthermore, it appears that out-of-domain passage-level relevance judgments *fortuitously* available, such as the MS MARCO passage dataset (Bajaj et al., 2018) and the TREC CAR dataset (Dietz et al., 2017), are also beneficial.

Thus, it appears that BERT is able to learn *cross-domain, sentence-level* notions of relevance that can be exploited for ranking newswire documents. Table 2 presents an extract of results from Yilmaz et al. (2019) for Robust04, where we find that the best results are achieved by first fine-tuning on MS MARCO and then on the Microblog data. Scores from BERT are then combined with document scores (BM25+RM3) based on Eq (1). The notation “1S”, “2S”, and “3S” refer to aggregating scores from the top one, two, and three sentences, respectively. Including more sentences doesn’t help and ranking is already quite good if we just consider the top-scoring sentence. This result, surprisingly, suggests that document ranking can be distilled into relevance prediction primarily at the sentence level. Based on the meta-analysis by Yang et al. (2019a), this is not only the highest known AP reported on the Robust04 dataset for neural models, but also exceeds the previous best known AP score of 0.3686, which is a non-neural method based on ensembles.

5 Demonstration

We demonstrate the integration of Birch with the search frontend from HiCAL (Abualsaud et al., 2018b) and interactive Google Colab notebooks.

5.1 HiCAL

The goal of the HiCAL system⁶ is to help human assessors efficiently find as many relevant documents as possible in a large document collection to achieve high recall on a search task. The system comprises two main components: a Continuous Active Learning (CAL) model (Cormack and Grossman, 2014) and a search model. In the CAL model, a machine-learned classifier selects the most likely relevant document or paragraph for the assessor to judge; judgments are then fed back to retrain the classifier. In the current HiCAL implementation, Anserini provides the back-end search capabilities.

For the TREC Common Core Tracks in 2017 and 2018, a small group of researchers used HiCAL to find and judge relevant documents. The runs generated based on their assessments achieved the highest AP scores among all the submitted runs for two consecutive years (Zhang et al., 2017; Abualsaud et al., 2018a). The effectiveness of the system was further demonstrated in Zhang et al. (2018a).

We further augment the Anserini backend for HiCAL with Birch in two ways: First, HiCAL can directly take advantage of improved rankings provided by BERT. Second, the top-scoring sentences can be highlighted in the document to aid in assessment. A sample screenshot is shown in Figure 2. For query 336 “black bear attacks” from Robust04, we show part of the highest-scoring document LA081689-0039 with one of the top three sentences (according to BERT) highlighted.

5.2 Interactive Colab Notebooks

We present Google Colab⁷ notebooks that make it possible for anyone to reproduce our end-to-end document retrieval pipeline in an interactive manner.⁸ We make all our data and pre-trained models available, although users may also opt to rebuild them from scratch; the Colab GPU backend enables fine-tuning BERT models directly in the notebook environment.

⁶<https://github.com/hical>

⁷<https://colab.research.google.com/>

⁸To ensure long-term availability, sample notebooks are linked from the main Birch repository.



Figure 2: Screenshot of HiCAL using Birch to identify the most relevant sentences in a document retrieved for the query “black bear attacks”.

Our notebooks are set up to allow relevance scores to be computed for an entire test collection in batch, and also to support interactive querying. When the user issues a query through the interactive notebook, candidate documents from the corpus are first retrieved using Anserini. A sentence-level dataset is created on the fly from the initial ranking by splitting each document into its constituent sentences. Each sentence is fed into our BERT model to obtain a relevance score. These relevance scores are then aggregated with document scores to rerank the candidate documents, per Eq (1). The notebook setting allows a user to step through each part of the process and examine intermediate results to gain a better understanding of our approach.

In addition, we have implemented two methods to visualize the relevant documents for a given query from a test collection, hopefully conveying even more insights. First, we generate a table that displays the document scores juxtaposed with the BERT scores of constituent sentences. Sentences with low document scores but high BERT scores (and vice versa) are highlighted, allowing the user to examine the relative contributions of exact term matching and semantic matching, as contributed by BERT. Second, we incorporate *bertviz*,⁹ an open-source tool for visualizing attention in transformer models, to explore the interaction between multiple attention heads. In Figure 3, we show a sentence with a high BERT score in a document retrieved for query 322 “international art crime” from Robust04. Note that this sentence does *not*

⁹<https://github.com/jessevig/bertviz>

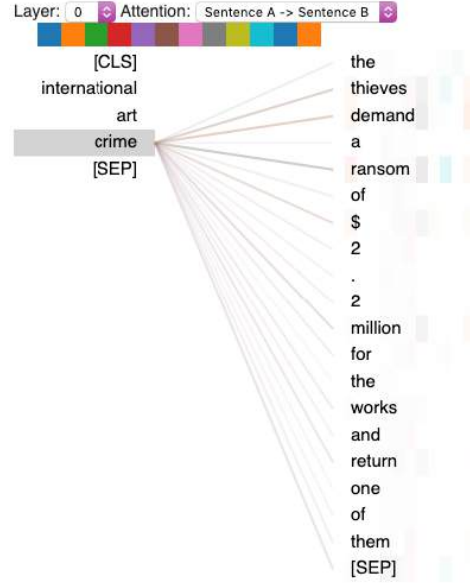


Figure 3: Screenshot of attention visualization for a sentence with a high BERT score from a document retrieved for the query “international art crime”. Note the lack of exact matches with query terms.

contain any of the query terms, but yet appears to be relevant. If we examine the attention visualization for the query term “crime”, we see that the model attends to obviously-related terms like “thieves”, “demand”, and “ransom”, illustrating the semantic knowledge that is captured in the BERT model.

6 Conclusions

This paper describes the system architecture and motivation behind a straightforward application of BERT to document ranking, via sentence-level inference and simple score aggregation. With the implementation of this system, we have also overcome the technical challenge of integrating a Lucene-based backend on the JVM with PyTorch to enable development in an environment NLP researchers and practitioners are already familiar with. The fruits of our labor are released in an open-source system for the community to continue explorations in search-related tasks with BERT.

Acknowledgments

This research was supported by the Natural Sciences and Engineering Research Council (NSERC) of Canada, and enabled by computational resources provided by Compute Ontario and Compute Canada.

CFO: A Framework for Building Production NLP Systems

Rishav Chakravarti¹ Cezar Pendus² Andrzej Sakrajda² Anthony Ferritto¹
Lin Pan¹ Michael Glass² Vittorio Castelli² J. William Murdock¹
Radu Florian² Salim Roukos² Avirup Sil^{2*}

¹IBM Watson, ²IBM Research AI

{rchakravarti, cpendus, ansa, panl, mrglass, vittorio, murdockj, raduf, roukos, avi}@us.ibm.com
aferritto@ibm.com

Abstract

This paper introduces a novel orchestration framework, called CFO (COMPUTATION FLOW ORCHESTRATOR), for building, experimenting with, and deploying interactive NLP (Natural Language Processing) and IR (Information Retrieval) systems to production environments. We then demonstrate a question answering system built using this framework which incorporates state-of-the-art BERT based MRC (Machine Reading Comprehension) with IR components to enable end-to-end answer retrieval. Results from the demo system are shown to be high quality in both academic and industry domain specific settings. Finally, we discuss best practices when (pre-)training BERT based MRC models for production systems.

1 Introduction

Production NLP (Natural Language Processing) and IR (information retrieval) applications often rely on a system flow consisting of multiple components that need to be woven together to build an end-to-end system (A. Ferrucci et al., 2010; Yang et al., 2019). This paper presents a novel approach for defining flow graphs and a toolkit for compiling those definitions into deploy-able production grade systems¹.

Though the framework, which we refer to as CFO (COMPUTATION FLOW ORCHESTRATOR), is well suited to a variety of use cases, we demonstrate it by creating an interactive QA (Question Answering) system that can be used both for academic benchmarking as well as industry specific use cases. The interactive system integrates SOTA (state-of-the-art) BERT-based MRC (Machine Reading Comprehension), an Elasticsearch

based document retrieval component, and a deduplication & sorting component to provide end-to-end answer retrieval. We will refer to this demonstration system as GAAMA (Go Ahead, Ask Me Anything). The key contributions of this work, therefore, are to (1) introduce a novel framework for stitching together deployable NLP components, (2) demonstrate the framework with an end-to-end QA system, and (3) discuss the training steps necessary for adapting a SOTA MRC model to a data set before plugging it into the QA system.

Section 2 provides the motivations and details of the CFO framework, section 3 discusses the specific model components integrated into GAAMA, section 4 discusses experimentation to adapt the BERT-based MRC component to this system, section 5 discusses related work, and, finally, section 6 provides a conclusion and discussion of future work.

In addition, a (private) screencast video demonstration of GAAMA has been uploaded at http://ibm.biz/gaama_demo (along with a supplementary presentation of the CFO framework at http://ibm.biz/gaama_cfo_demo).

2 CFO Architecture

The CFO framework relies on two sets of system specifications to define the computation flow graph. First, each node within the graph defines its service name, input message data fields, and output message data fields using Google’s Protocol Buffer Interface Definition Language². Second, the orchestrator is described using a custom specification format allowing the user to declare:

1. The set of nodes (provided as containerized gRPC³ microservices). Each node will have

* Corresponding author.

¹We are actively seeking to open source the toolkit and flow definition language. If successful, we will be releasing the code to http://ibm.biz/cfo_framework

²<https://developers.google.com/protocol-buffers/>

³<https://grpc.io/>

implemented a microservice according to the node’s declared interface specification.

2. The set of nodes to treat as entry points to the flow graph.
3. A mapping within the flow for each *data element* comprising the input and output message interfaces for nodes.
4. For ease of deployment, the specification also provides the ability to describe deployment specific configuration settings like service ports, docker registry location etc.

Given these specification files, CFO provides a compiler to auto-generate an orchestrator node which implements the computation flow graph, a (dockerized) launch script, and a simple REST interface / GUI which provide access to the defined entry points and debug information.

The resulting design allows data flows and dependencies to be described both concisely and transparently. The flow specification allows for easy debugging and modification of how data fields enter the computation flow, get transformed, and finally outputted. This is a significant departure from traditional orchestration systems which would require parsing through source code to determine and change the route taken by data fields through the computation flow. Furthermore, the use of Protocol Buffers to describe these data fields ensures a language and platform agnostic representation that does not compromise the system’s speed or ability to ensure data type correctness at compile time (as opposed to traditional JSON/XML representations which need to encode/decode from strings at run time).

Another benefit of the CFO toolkit is to auto-generate the serialization and connectivity code for each node as well as exposing the entry points via REST interfaces. Relinquishing this responsibility to the CFO toolkit frees the developer up from writing a significant amount of boilerplate code and worrying about distributed system best practices such as enforcing time outs, error propagation, latency logging, and parallelization through asynchronous calls. As an illustrative example, the auto-generated code for the GAAMA demo consists of 2,800 lines of C++ orchestration code.

Finally, the CFO toolkit generates a set of shell scripts, docker images, and config files for deploying and running the flow graph using either

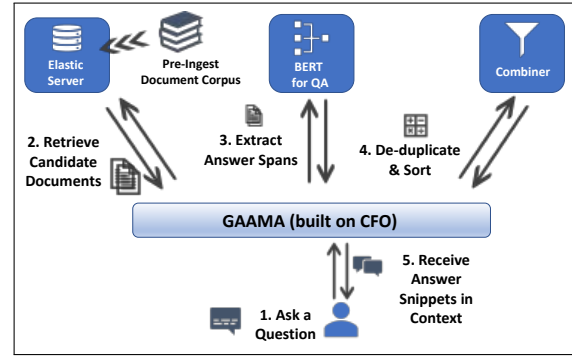


Figure 1: GAAMA System Architecture

docker-compose⁴ or kubernetes⁵. This allows the generated project to be deployed both locally for debugging as well as on modern cloud infrastructure.

3 GAAMA Architecture

As a simple case study for CFO, we create a demonstration QA system consisting of four nodes: (1) an Elasticsearch⁶ based IR node (2) a BERT based MRC node (3) an answer “de-duplication” node and (4) a final answer combiner node. See fig. 1 for an overview of the QA system.

As described in section 2, each component starts by declaring an interface specification allowing the underlying implementation to be swapped out without the need to modify the rest of the QA system. Next, a gRPC server is implemented with the core business logic. Auto-generated gRPC code stubs provide the core communication/serialization logic for the service layer of the server. We describe the IR and MRC nodes in further detail in the following two sections.

3.1 IR with Elasticsearch

The core business logic of the IR node uses Elasticsearch APIs to retrieve the top k documents from the appropriate corpus based on BM25, a popular variant of term frequency overlap between the query and document text (Robertson et al., 1976). Two document corpora are ingested using the standard English analyzer. The first corpus consists of Wikipedia paragraphs used for academic benchmarking and the second corpus consists of an industry dataset made up of IBM Technical Support Documentation. The user interface allows us to choose either corpus when asking

⁴<https://docs.docker.com/compose/>

⁵<https://kubernetes.io/>

⁶<https://www.elastic.co/products/elasticsearch>

questions to evaluate the system. We use “paragraphs” as the base unit of ingestion in line with (Yang et al., 2019) which shows this as an optimal pre-processing step for consumption by a MRC component.

The node’s input interface, therefore, accepts query text, a hyperparameter k , and a target corpus identifier. Its output interface produces a list of document texts accompanied by retrieval scores. As discussed earlier, these interfaces are defined using protocol buffer definitions, so we follow standard steps⁷ for auto-generating a gRPC server with placeholders for the custom business logic of retrieving documents. Similarly, with the server logic in place, we follow standard steps⁸ to create a docker image that CFO will need to launch the gRPC server. Though we are wrapping Elasticsearch’s index based retrieval implementation here, we can swap our implementation for more recent Neural IR based techniques (Craswell et al., 2017) without changing the exposed interface or the orchestration code.

3.2 MRC with BERT

The MRC node similarly wraps a BERT-for-QA model in a dockerized gRPC server which accepts a single query-document pair as its input and produces a span from the document along with a prediction score as its output. Note that CFO’s orchestrator automatically realizes that the IR node produces a list of documents, while the MRC node accepts a single document at a time. So CFO will take care of calling the MRC node for each of the k retrieved documents (using asynchronous calls to parallelize requests if a configuration flag is set).

The underlying BERT-for-QA model is based on (Alberti et al., 2019). BERT (Devlin et al., 2018) is one of a series of pre-trained neural models that can be fine tuned to provide state-of-the-art results in NLP (Peters et al., 2018; Howard and Ruder, 2018; Radford et al., 2019) including on the SQuAD (Rajpurkar et al., 2018) and NQ (Kwiatkowski et al., 2019) tasks that align with our MRC based QA.

We use the Huggingface PyTorch implementation of BERT⁹ which supports starting from a Base (a 12 layer, 768 hidden dimension, 12 atten-

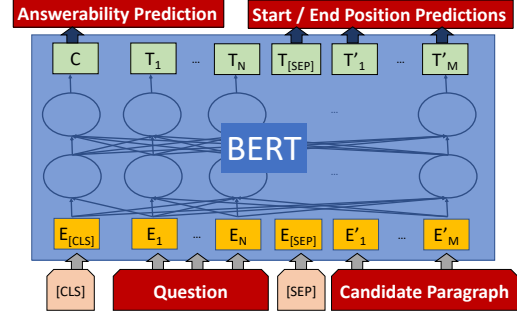


Figure 2: BERT for QA (Devlin et al., 2018)

tion head, 110M parameter transformer network) or a Large (a 24 layer, 1024 hidden dimension, 16 attention head, 340M parameter transformer network) model. An output feed forward layer is added on top of this to produce 3 sets of scores: (1) scores at each token offset marking the likelihood of an answer chunk starting at this offset (2) scores at each token offset marking the likelihood of an answer chunk ending at this offset (3) a score for the entire sequence marking the likelihood of the question being answerable given the current context.

The parameters of the entire network are fine tuned using a set of question and document pairs where annotators provide the correct start and end offsets or have marked the question-document pair as having no correct answer. Refer to fig. 2 for a visual depiction of the model and to (Alberti et al., 2019) for additional details about model architecture and implementation.

Section 4 includes practical pre-training, fine tuning, and hyperparameter optimization steps for building the final model deployed as part of GAAMA. At the time of writing, our BERT-based MRC model¹⁰ was the best performing submission (dated 7/31/2019), outperforming the next best system by 1% on F1 on the Natural Questions public leaderboard¹¹.

4 Experiments

Prior to integration of the MRC node into GAAMA, we first experiment with data preparation and training of the BERT MRC model as a standalone component using dev sets provided with the NQ and SQuAD data sets (see 5 for more on these data sets). NQ is preferred for evaluat-

⁷<https://grpc.io/docs/tutorials/>

⁸<https://github.com/grpc/grpc-docker-library/>

⁹<https://github.com/huggingface/pytorch-transformers>

¹⁰The best performing submission to the leaderboard uses an ensemble of models using different hyperparameters rather than a single model

¹¹<https://ai.google.com/research/NaturalQuestions/leaderboard>

Conundrums in Entity Coreference Resolution: Making Sense of the State of the Art

Jing Lu and Vincent Ng

Human Language Technology Research Institute

University of Texas at Dallas

Richardson, TX 75083-0688

{ljwinnie, vince}@hlt.utdallas.edu

Abstract

Despite the significant progress on entity coreference resolution observed in recent years, there is a general lack of understanding of what has been improved. We present an empirical analysis of state-of-the-art resolvers with the goal of providing the general NLP audience with a better understanding of the state of the art and coreference researchers with directions for future research.

1 Introduction

The advent of the neural NLP era has revolutionized virtually all areas of NLP research. For entity coreference, many issues that were once thought to be important no longer appear to be particularly relevant to the current research agenda. Specifically, while a decade ago coreference researchers have focused on developing computational models that are complex (e.g., structured models (Fernandes et al., 2012; Björkelund and Kuhn, 2014; Martschat and Strube, 2015)) and knowledge-rich (e.g., those that encode world knowledge (Ponzetto and Strube, 2007; Rahman and Ng, 2011a; Hajishirzi et al., 2013)), nowadays virtually all state-of-the-art resolvers employ a simple model (i.e., the mention-ranking model, which was developed more than a decade ago (Denis and Baldridge, 2008))¹ and a fairly simple input representation (i.e., contextualized word embeddings) in conjunction with a mechanism for learning representations of entity mention spans such that coreferent mentions have similar representations (Lee et al., 2017, 2018; Kantor and Globerson, 2019; Joshi et al., 2019).

Despite significant progress in the past few years in terms of performance numbers, what seems to be missing is an understanding of what has been

improved. The lack of understanding has long been a concern shared by coreference researchers, even before the neural revolution in NLP. This has led to several attempts to analyze coreference resolvers over the years (Stoyanov et al., 2009; Kummerfeld and Klein, 2013). With the development of neural resolvers, however, this concern has become more serious than ever: the fact that significant progress can be made via learning mention representations with a simple neural mention-ranking model that employs a fairly simple input representation for a task as challenging as coreference resolution (CR) is somewhat contrary to common wisdom.

In light of this apparent conundrum, we present an empirical analysis of state-of-the-art entity coreference resolvers through four major sets of experiments in this paper, with the goal of gaining insights into their behaviors. We believe that our analysis will not only provide the general NLP audience with a better understanding of the state of the art, but also provide coreference researchers with directions for future research.

2 Evaluation Setup

In this section, we describe the datasets, the evaluation metrics, the state-of-the-art resolvers and the hyperparameters used in our experiments.

Datasets. We report results on three coreference datasets. The NIST-sponsored ACE evaluations resulted in several datasets. We use ACE 2005 (Walker et al., 2006), the last one in the series. The ACE 2005 organizers have only made the official training set (but not the official test set) publicly available, so previous work defined different train-test splits over the official training set. We employ the same train-test split as Bansal and Klein (2012).

KBP is another series of NIST-sponsored evaluations in the mid 2010s. KBP does not have any evaluations on entity CR, but to support high-level

¹The first learning-based resolver is a pairwise ranker (Connolly et al., 1994), which was extended by Denis and Baldridge (2008) to rank more than two candidate antecedents at a time.

	ACE			OntoNotes			KBP		
	Train	Dev	Test	Train	Dev	Test	Train	Dev	Test
#docs	365	117	117	2802	343	348	360	97	168
#mentions	34481	11126	9261	155558	19155	19764	40628	10983	13860
#chains	11963	3798	3050	35142	4545	4532	14332	3942	5482

Table 1: Dataset statistics in terms of the number of documents, mentions, and coreference chains.

information extraction tasks (e.g., event extraction, event CR), the organizers have made available several corpora that include entity coreference annotations. For training, we use three such corpora (LDC2015E29, LDC2015E68, and LDC2016E64). For evaluation, the KBP 2017 organizers have made available the official test set for the event CR task (LDC2017E51), which also include entity coreference annotations. We use it as our test set.

OntoNotes (Hovy et al., 2006), which was developed circa 2006, is the most widely-used dataset for entity coreference evaluations. It has a standard train-dev-test split. Unlike in ACE and KBP, singleton clusters are not annotated in OntoNotes.

The key difference between these three corpora is that OntoNotes supports “unrestricted” CR, meaning that coreference links are annotated between entity mentions without regard to their entity types. In contrast, coreference links are only annotated between mentions belonging to one of the seven entity types in ACE and one of the five entity types in KBP. Statistics on these corpora are shown in Table 1.

Evaluation metrics. Following the convention established in the CoNLL 2011 and 2012 shared tasks (Pradhan et al., 2011, 2012), we use as our primary coreference evaluation measure the CoNLL score, which is the unweighted average of the F-scores provided by three popular metrics, the link-based MUC metric (Vilain et al., 1995), the mention-based B³ metric (Bagga and Baldwin, 1998), and the entity-based CEAF_e metric (Luo, 2005). We obtain these scores using the official CoNLL scorer (Pradhan et al., 2014).²

Mention detection (MD) is the task of extracting the mentions in a text needed for entity CR. A key observation made in the CoNLL shared tasks was that the performance of resolvers was limited by MD, so it is important to examine the extent to which MD performance has improved over the

years. We report performance in terms of recall, precision, and F-score, considering that a system mention is correctly detected if and only if it has an exact match in boundary with a gold mention.

Systems. We evaluate five variants of three state-of-the-art neural resolvers, all of which employ a ranking model where all candidate antecedents are ranked against each other for a given anaphor.

The first resolver, the Stanford neural resolver (Clark and Manning, 2016)³, takes as input a set of entity mentions identified for a given document by a rule-based MD system and trains using reinforcement learning a simple mention ranker consisting of three hidden layers of ReLU units and a final layer that is fully-connected.

The other two resolvers are developed by Lee et al. (2018)⁴ and Joshi et al. (2019)⁵. Both are *span-based* models, which have two key characteristics. First, mention spans are identified as part of CR, so this mitigates the propagation of errors from MD to CR. Second, *representations* of entity mention spans are learned so that coreferent mentions have similar representations. The key differences between these resolvers are: (1) in Lee et al. the input instances correspond to the sentences in the given document, whereas in Joshi et al. the input instances correspond to fixed-length non-overlapping segments of the input document⁶; (2) Lee et al. use a LSTM, whereas Joshi et al. use a transformer; and (3) the pretrained embeddings are different.

For each of these resolvers, we derive two variants. Specifically, Lee et al. (2018) employ GloVe+ELMo embeddings, but to better understand the effect of the contextual information provided by ELMo embeddings (Peters et al., 2018) on CR performance, we evaluate a version of Lee et al. using only GloVe embeddings (Pennington et al., 2014). We will henceforth refer to these two versions of Lee et al. as ELMo (i.e., ELMo+GloVe) and GloVe, respectively. Joshi

²LEA (Moosavi and Strube, 2016) is a coreference evaluation metric recently designed to address the shortcomings associated with B³ and CEAF_e, but we found no difference in the performance trends in our experiments according to CoNLL and LEA. See the Appendix for the LEA results.

³<https://github.com/clarkkev/deep-coref>

⁴<https://github.com/kentonl/e2e-coref>

⁵<https://github.com/mandarjoshi90/coref>

⁶This is the *independent* version in Joshi et al. (2019).

Hyperparameter	ACE			OntoNotes			KBP		
	GloVe/ELMo	SpanB-b	SpanB-l	GloVe/ELMo	SpanB-b	SpanB-l	GloVe/ELMo	SpanB-b	SpanB-l
Max span width	30	30	30	30	30	30	20	20	10
Max top antecedents	50	50	50	50	50	50	50	50	35
Max training segs/sents	50 sents	3 segs	3 segs	50 sents	3 segs	3 segs	50 sents	3 segs	3 segs
Top span ratio	0.4	0.35	0.4	0.4	0.4	0.4	0.35	0.35	0.35
Max segment length	—	384	512	—	384	512	—	384	512
SpanBERT learning rate	—	2e-5	1e-5	—	2e-5	1e-5	—	2e-5	2e-5
Task learning rate	0.001	1e-4	3e-4	0.001	1e-4	3e-4	0.001	1e-4	2e-4

Table 2: Best hyperparameters obtained on the development sets for each span-based resolver.

et al. (2019) employ embeddings pretrained using a new method called SpanBERT (Joshi et al., 2020), which is designed to better represent text *spans* than BERT. The two variants of Joshi et al. differ in the transformer. Specifically, SpanBERT-base (henceforth SpanBERT-b) employs a simple transformer while SpanBERT-large (henceforth SpanBERT-l) employs a more complex transformer.

We use the publicly-available implementation of each of these resolvers. There is one caveat, however. Recall that the span-based resolvers were all evaluated on OntoNotes. Since singleton clusters are not annotated in OntoNotes, all singleton clusters predicted by a resolver are removed from its output before it is sent to the scoring program. In contrast, singleton clusters that contain mentions belonging to one of the ACE/KBP entity types are annotated in ACE/KBP, so these mentions should not be removed from a resolver’s output. However, span-based resolvers cannot distinguish between spans that correspond to entity mentions and those that do not. To address this problem, we extend the span-based models so that they are jointly trained to predict entity mention spans and coreference links. Specifically, the feedforward neural network that is responsible for scoring a span in these models currently do not receive *direct* feedback on whether a span corresponds to an entity mention. We first turn it into a mention detector by training it in a supervised manner using the negative cross entropy loss, so that it predicts a positive mention score for a span if and only if the span corresponds to an entity mention. Then, to jointly learn MD and CR in the span-based resolvers, we employ a loss function that is the unweighted sum of the coreference loss and the MD loss.

Hyperparameter tuning. To ensure a fair comparison of the resolvers, we tune their hyperparameters to maximize the CoNLL score on development data. Note, however, that the authors of Stanford, ELMo, SpanBERT-b, and SpanBERT-l reported the best hyperparameter settings on OntoNotes in

the original papers (Lee et al., 2018; Joshi et al., 2019), so we simply use them in our experiments and focus on tuning the hyperparameters for the remaining cases. We adopt the set of hyperparameters to be tuned from the original papers.

For Stanford, there are three hyperparameters to tune: α_{WL} , α_{FA} , and α_{FN} . These are the weights associated with three different types of mistakes made by the coreference model. Following Clark and Manning (2016), we fix $\alpha_{WL} = 1.0$ and search for α_{FA} and α_{FN} out of $\{0.1, 0.2, \dots, 1.5\}$ using a variant of grid search. For ACE, $(\alpha_{WL}, \alpha_{FA}, \alpha_{FN}) = (1.0, 0.5, 1.0)$ is the best configuration, and for KBP, the best configuration is $(1.0, 0.5, 0.8)$. For OntoNotes, we use the configuration found by Clark and Manning, which is $(1.0, 0.5, 0.8)$.

For GloVe and ELMo, we have five hyperparameters to tune. Specifically, we search for: (1) max span width (i.e., maximum number of words in a candidate span) out of $\{10, 20, 30\}$; (2) max top antecedents (i.e., maximum number of candidate antecedents) out of $\{35, 40, 45, 50\}$; (3) max training sentences out of $\{25, 50, 75, 100\}$; (4) task learning rate out of $\{5e-4, 1e-3, 2e-3\}$; and (5) top span ratio (i.e., the fraction of top spans that survive the filtering) out of $\{0.3, 0.35, 0.4, 0.45, 0.5\}$. For the two SpanBERT resolvers, we have seven hyperparameters to tune. For three of the hyperparameters (max span width, max top antecedents, and top span ratio), the ranges are the same as those used in GloVe and ELMo. For the remaining four, we search for: (1) max training segments out of $\{3, 4, 5\}$; (2) max segment length out of $\{128, 256, 384, 512\}$; (3) SpanBERT learning rate out of $\{1e-5, 2e-5\}$; and (4) task learning rate out of $\{1e-4, 2e-4, 3e-4\}$. Table 2 shows the best hyperparameter setting of each span-based model on each dataset.

3 Performance across Datasets

We first provide the reader with a high-level understanding of the state of the art by analyzing the five

	System	ACE				OntoNotes				KBP			
		CoNLL MUC	Sing.	MD		CoNLL MUC	Sing.	MD		CoNLL MUC	Sing.	MD	
1	Stanford	38.6	57.9	56.8	50.6	65.5	74.3	—	80.1	26.2	48.8	42.8	26.4
2	GloVe	68.0	76.6	67.0	87.4	68.1	76.7	—	82.0	65.9	71.3	63.2	82.4
3	ELMo	71.8	79.5	70.1	90.4	73.0	80.5	—	85.1	69.7	73.8	67.5	85.3
4	SpanBERT-b	75.7	83.1	71.5	91.6	77.4	83.7	—	87.1	71.3	75.6	67.5	86.3
5	SpanBERT-l	78.9	85.4	75.1	92.3	79.6	85.3	—	88.2	75.8	80.1	71.1	88.5

Table 3: Results of the resolvers on the three coreference datasets.

resolvers’ performance on the three datasets.

Performance across datasets. Results on the three datasets, which are reported in terms of the CoNLL score, are shown in the CoNLL column in Table 3.⁷ Although the five resolvers have been evaluated solely on OntoNotes, their relative performances are consistent across the datasets. In particular, the use of ELMo embeddings enables ELMo to outperform GloVe by 3.8–4.9% points. SpanBERT-b outperforms ELMo by 1.6–4.4% points, and SpanBERT-l further outperforms SpanBERT-b by 2.2–4.5% points.

Source of performance improvements. Do the above improvements stem from improved recognition of coreference links, or improved recognition of singleton clusters, or both? To understand whether these resolvers have improved in terms of *link* prediction, we examine the MUC F-scores (see the MUC column), which are computed solely on coreference links. As we can see, the MUC scores are consistently increasing down the table across all datasets, meaning that later systems are indeed doing better at identifying coreference links. To understand whether later resolvers are also better at identifying singleton clusters, we show in the Singleton column the percentage of singleton clusters that are correctly *recalled*. Again, the scores are increasing down the table, and the degree of improvement is particularly large from GloVe to ELMo and from SpanBERT-b to SpanBERT-l.

Mention detection performance. First, MD performance has improved significantly over the years. SpanBERT-l achieves an F-score of 88.2 in MD on OntoNotes, which is significantly higher than the best MD F-score achieved in the CoNLL-2012 shared task (77.7). Note that Stanford’s mention detector performs substantially worse than those of the other resolvers, especially on ACE and KBP. The reason is that Stanford employs a rule-based MD system that was initially developed when the

Stanford NLP Group participated in the CoNLL-2011 shared task, whereas in the other resolvers MD is jointly trained with CR. Overall, MD performance appears to have a significant impact on CR performance. In particular, joint MD and CR in the span-based resolvers seems to be a driving force behind the rapid coreference performance improvements we have seen in recent years.

4 Using Oracles

Can the performance of coreference resolvers be further improved if we improve MD? Being able to answer this kind of questions is important: if further improvements in MD can result in significant gains in coreference performance, then future research efforts should perhaps be focused on MD.

To answer this kind of questions, we perform oracle experiments. Specifically, we provide a resolver with a particular type of perfect information (e.g., using gold mentions as input) and see how much performance improvement can be obtained.

4.1 Gold Mention Boundaries

Our first oracle experiment concerns training and testing our resolvers on gold mention boundaries. While this experiment has been conducted over the years by numerous researchers (e.g., Peng et al. (2015), Zhang et al. (2018)), we are primarily interested in understanding whether further improving an MD component that already has an F-score of more than 85% can improve coreference performance. For the four span-based models, we disable the component in the span representation layer that is responsible for proposing spans (i.e. mention boundaries) and instruct them to use gold mention spans instead. Note, however, that the representation of a span will be learned during training. In other words, although all resolvers are given gold mention spans, the span representations that will be used during resolution will still be different for different span-based resolvers.

Results, expressed in terms of the CoNLL score, are shown in the Gold Mention Boundaries col-

⁷Owing to space limitations, we show only the most important scores in Table 3. The detailed results (e.g., B³ and CEAF_s results) can be found in the Appendix.

	System	Gold Mention Boundaries			Perfect Anaphoricity			Gold Entity Types		
		ACE	Onto.	KBP	ACE	Onto.	KBP	ACE	Onto.	KBP
1	Stanford	76.3	83.7	80.0	40.3	73.4	29.2	39.5	71.8	27.7
2	GloVe	79.3	86.1	81.6	71.5	76.2	69.8	69.6	71.4	67.9
3	ELMo	81.8	87.2	83.5	76.1	82.0	73.8	73.3	76.7	71.6
4	SpanBERT-b	84.9	90.5	85.7	79.3	84.4	76.3	77.4	79.3	73.8
5	SpanBERT-l	87.3	91.9	88.0	82.8	86.9	80.1	80.4	81.4	77.6

Table 4: CoNLL scores of the resolvers using gold mention boundaries, perfect anaphoricity, and gold entity types.

umn in Table 4. First, despite recent significant improvement in MD, these results suggest that coreference performance can still be significantly improved just by improving MD: for the best resolver (SpanBERT-l), the CoNLL score can be improved by 8.4–12.3% points. Second, the relative performances of the resolvers are consistent across the three datasets: the CoNLL scores increase as we go down the table. Since the four span-based resolvers use essentially the same (mention-ranking) model for resolution and the same algorithm for weight updates, their performance differences can be attributed largely to differences in the pretrained embeddings and the encoder. In addition, these results suggest that the coreference performance improvements we observed in recent years can be attributed to not only improved mention (boundary) detection but also improved resolution accuracy presumably as a result of better span representations.

4.2 Perfect Anaphoricity

Anaphoricity determination, a.k.a. discourse-new detection (Poesio et al., 2004), is the task of determining whether a mention is coreferent with another mention that appears earlier in the text. Being able to identify non-anaphoric mentions could improve the precision of coreference resolvers, as any antecedent chosen for them is erroneous.

In this oracle experiment, we provide a resolver with perfect anaphoricity information, meaning that we know for every entity mention whether it is anaphoric or not. We use this perfect anaphoricity information during resolution: we will resolve all and only those mentions that are anaphoric.

Results are shown in the Perfect Anaphoricity column of Table 4. A few points deserve mention. First, all resolvers improved on all datasets when provided with perfect anaphoricity information. These results imply that anaphoricity determination remains an important issue in CR research, and further improvements in anaphoricity can improve CR. However, the gains that state-of-the-art resolvers can achieve by improving anaphoricity

determination are generally smaller than those by improving MD: the CoNLL scores of the span-based resolvers increase by 3.5–4.3% points on ACE, 4.4–9% points on OntoNotes, and 3.9–5% points on KBP. This is understandable, as MD is likely to improve both coreference precision and recall, whereas anaphoricity determination can only improve precision. Note that Stanford’s poor performance on ACE and KBP is due to poor MD.

4.3 Gold Entity Types

In this experiment, we assume that a resolver is given gold entity types (i.e., semantic classes) such as PERSON, ORGANIZATION, and LOCATION. The set of entity types to be provided is corpus-dependent. As mentioned before, ACE and KBP only have seven and five entity types respectively. In OntoNotes, however, only named entities are annotated with (one of 18) entity types. Consequently, we automatically derive entity types for pronouns and nominals using gold coreference chains: if a pronoun or a nominal appears in a coreference cluster that contains a name, we derive its entity type from that of the name. This method allows us to derive the entity type of 36.4% of the nominals and 70% of the pronouns. Any pronoun or nominal whose entity type cannot be derived using this method will be assigned the entity type UNKNOWN. While this method does not provide full coverage, we will still be able to examine whether having access to perfect entity types on a subset of the mentions will enable us to improve the performance of a resolver on OntoNotes.

We use entity types during resolution. We disallow a candidate antecedent to be selected as the antecedent for a given anaphor if they have different entity types. Results are shown in the Gold Entity Types column in Table 4. As we can see, all resolvers improved on all datasets when provided with gold entity types. Compared with the gains achieved using gold mention spans or perfect anaphoricity, the gains that come with the use of gold entity types are smaller: for the span-based

resolvers, the CoNLL scores increase by 1.5–1.7% points on ACE, 1.8–3.7% points on OntoNotes, and 1.9–2.5% points on KBP. In other words, state-of-the-art resolvers can be improved by improving the determination of entity types.

These results are particularly interesting in light of a conundrum in entity CR: while some researchers have reported successes with improving entity CR using automatically computed semantic information (Ng, 2007), there have also been numerous failed attempts (Kehler et al., 2004; Durrett and Klein, 2013; Sapena et al., 2013). Although the semantic information we use in this paper is restricted to gold entity types, our results suggest that hand-annotated semantic information is indeed useful, and the (non-)utility of semantics for CR reported in earlier work could be attributed to the noise inherent in computing semantic information.

5 Results on Resolution Classes

To gain additional insights into the state-of-the-art resolvers, we analyze their performance on different types of entity mentions. More specifically, motivated by Stoyanov et al. (2009), we partition the *gold* mentions into different *resolution classes*. While previous work has focused mainly on three coarse-grained resolution classes (namely, pronouns, names, and nominal mentions), we employ the 13 fine-grained resolution classes defined by Rahman and Ng (2011b), as discussed below.

Names. Four classes are defined for gold names. (1) **e**: a name is assigned to this *exact string match* class if there is a preceding mention such that the two are coreferent and are the same string; (2) **p**: a name is assigned to this *partial string match* class if there is a preceding mention such that the two are coreferent and have some content words in common; (3) **n**: a name is assigned to this *no string match* class if there is no preceding mention such that the two are coreferent and have some content words in common; and (4) **na**: a name is assigned to this *non-anaphor* class if it is not coreferent with any preceding mention.

Nominal mentions. Four analogous resolution classes are defined for gold mentions whose head is a nominal: (5) **e**; (6) **p**; (7) **n**; and (8) **na**.

Pronouns. We have three pronoun classes. (9) **1/2**: 1st and 2nd person pronouns; (10) **G3**: gendered 3rd person pronouns (e.g., *she*); (11) **U3**: ungendered 3rd person pronouns; (12) **oa**: any anaphoric pronouns that do not belong to (9), (10),

and (11) (e.g., relative pronouns); and (13) **na**: non-anaphoric pronouns (e.g., pleonastic pronouns).

Table 5 shows the performance of each resolver on each resolution class. To avoid overwhelming the reader, we only show the results of ELMo and SpanBERT-l, which will allow us to gain insights into what made SpanBERT-l better. Specifically, for each resolution class *C*, we show each resolver’s MD recall (percentage of gold mentions in *C* that are correctly recalled) under **MD** and its resolution accuracy (percentage of correctly identified anaphors in *C* that are correctly resolved)⁸ under **RA**. Under **Size** we show the percentage of gold mentions belonging to each resolution class.

First, if we consider only the three coarse-grained resolution classes, the results are perhaps not surprising: name resolution is the easiest and nominal resolution is the hardest.

Second, consider the 13 fine-grained resolution classes. By design, the names and the nominals in the ‘e’ class should be easier to resolve than those in ‘p’, which in turn should be easier to resolve than those in ‘n’. The results are consistent with this intuition. Results on the anaphoric pronoun classes are also consistent with our intuition: 3rd person gendered pronouns are the easiest to resolve, followed by 1st/2nd person gendered pronouns and then ungendered 3rd person pronouns.

Third, these results reveal that the difficulty of anaphoricity determination stems primarily from pronouns: while resolution accuracies on non-anaphoric names and nominal mentions are above 89%, those on non-anaphoric pronouns are only between 65.9% and 77.6%. Note that we consider a non-anaphoric mention correctly “resolved” if it is resolved to the dummy antecedent.

Finally, SpanBERT-l has better resolution accuracies than ELMo for all resolution classes on all datasets. Encouragingly, the harder a resolution class is, the bigger the improvement is. These results clearly show that we are making progress on resolving anaphors that are traditionally considered difficult to resolve. Note that part of this improvement can be attributed to improved MD, which increases the likelihood that the correct antecedent of an anaphor is present in its list of candidate antecedents. Additional experiments are needed to determine the impact of improved MD on improvement in resolution accuracies, however.

⁸In other words, the resolution accuracy does not depend on anaphor recall and precision.

	Class	ACE					OntoNotes					KBP				
		Size %	ELMo		SpanB-1		Size %	ELMo		SpanB-1		Size %	ELMo		SpanB-1	
			RA	MD	RA	MD		RA	MD	RA	MD		RA	MD	RA	MD
1	NAM-e	16.8	94.6	96.0	97.4	96.6	14.0	95.3	93.2	95.2	91.8	16.5	94.0	95.0	95.3	96.5
2	NAM-p	4.2	71.5	86.6	81.5	88.2	7.3	82.7	83.2	88.5	83.7	2.0	66.1	90.8	74.6	93.8
3	NAM-n	2.0	52.5	87.6	69.8	91.4	1.9	56.6	60.6	73.6	68.1	2.9	56.8	88.4	64.2	92.0
4	NAM-na	10.8	93.3	88.4	94.8	91.0	11.5	93.5	74.8	94.4	79.0	16.8	95.8	91.6	96.1	92.8
5	NOM-e	3.4	77.0	91.0	84.3	92.3	3.7	92.9	89.6	95.7	90.0	3.4	86.2	85.7	88.1	86.7
6	NOM-p	4.3	51.1	82.8	61.5	89.4	5.3	78.1	78.6	84.3	83.1	4.2	43.7	83.3	52.5	80.3
7	NOM-n	4.2	48.5	84.0	66.3	89.4	3.3	58.6	61.9	78.4	71.6	5.8	42.1	77.0	50.6	76.5
8	NOM-na	16.8	92.0	83.1	93.1	86.1	8.4	89.2	66.7	92.5	77.1	19.9	92.8	78.2	93.4	78.3
9	PRO-I/2	16.4	81.8	99.8	88.6	99.8	16.1	90.1	93.2	93.6	96.1	13.5	82.1	100	88.2	99.9
10	PRO-G3	5.7	88.5	99.8	94.1	100	10.7	91.6	99.6	95.9	99.3	6.5	88.3	100	94.7	99.7
11	PRO-U3	6.2	68.8	91.5	85.2	96.8	13.8	84.1	93.4	91.2	96.0	4.6	70.5	93.4	79.5	97.8
12	PRO-oa	3.9	52.9	80.3	70.3	86.1	2.2	57.0	57.3	69.1	70.0	1.2	68.3	69.8	72.0	72.7
13	PRO-na	5.3	73.9	87.3	77.6	90.6	1.8	65.9	80.5	69.6	84.3	2.8	66.5	93.5	72.6	92.7

Table 5: Results on resolution classes.

6 Sensitivity to Perturbed Inputs

Next, we conduct a series of experiments that involve perturbing the input. In each experiment, we (1) replace a certain kind of words/phrases in each training document with other words/phrases, (2) train a coreference model on these perturbed training documents, and (3) evaluate the output. Our goal is to gain insights into the behavior of state-of-the-art resolvers by examining how sensitive their performance is to perturbations in the input. Specifically, if performance drops significantly when a particular kind of words/phrases is replaced, that means the replaced words/phrases are important in the model learning process. Note that perturbations are only applied to the training documents; no changes are made to the test documents.

We divide the different kinds of perturbations into two broad categories, mention-internal perturbations and mention-external perturbations.

6.1 Mention-internal Perturbations

Mention-internal perturbations involve making changes to the words within an entity mention.

6.1.1 Perturbations to Names

We consider two kinds of perturbations to names.

Unseen names. We replace each name in a training document with a name that will highly unlikely appear in any test set. With this replacement, all the names in the test set will be unseen w.r.t. the training set. When trained on this perturbed training set, we can determine the extent to which the algorithms for learning coreference resolvers rely on memorizing seen names (as opposed to generalizing from their contexts) when performing MD and CR. Specifically, if a learner memorizes a lot,

Perturbation Type	Example
Unseen names	Mr. Smith → Mr. Htims
Names of a different type	John Smith → New York
Unseen nominals	activist → tsivitca
Nominals of a different type	actor → plane
Nominals of the same type	wife → grandmother
Unseen verbs	support → troppus
Seen verbs	acquire → believe
Unseen adj/adv	directly → yltcerid
Seen adj/adv	organic → shredded

Table 6: Perturbation examples.

it will likely perform poorly on MD (i.e., its recall will suffer) and subsequently CR.

We perform name replacement in a deterministic manner: we replace each word in a name with another word in which the order of its characters is reversed. Note that person prefixes (e.g., “Mr.”), organization words and suffixes (“Airlines”, “Inc.”), and location nouns (e.g., “River”) will not be replaced, as the goal is to introduce unseen names rather than change the type of a name.⁹ In addition, any word in a name that appears in a nominal mention in the training set will not be replaced. For instance, the word “Church” in “Baptist Church” appears in a nominal in the training set and therefore will not be replaced. This is done to ensure that only the “name” part of a mention will be changed to something that is not previously seen.¹⁰

Names of a different type. In this experiment, we replace each name, ne_1 , in a training document with another name, ne_2 , that satisfies two conditions. First, ne_2 , like ne_1 , should appear in the training set. This ensures that the number of names in the test set that will be unseen w.r.t. the train-

⁹These lists are available in the Appendix.

¹⁰Examples of this and other kinds of perturbations are shown in Table 6.

ing set will not change. Second, the two names should have different entity types. Importantly, the replacement is deterministic, meaning that (1) all occurrences of ne_1 will be replaced with the same name (i.e., ne_2), and (2) any name coreferent with ne_1 (but are not lexically identical to ne_1 , such as “Trump” and “President Trump”) will be replaced with a name coreferent with ne_2 . These conditions together ensure that only the names and their types will change, but their coreference relationships will not. Note that the choice of ne_2 is random subject to these conditions. Due to the randomness involved in the selection of ne_2 , we repeat the experiment three times and report the average result.

With this replacement, the resulting training documents may no longer make sense to a human reader, as a PERSON name may appear in a context for an ORGANIZATION name. In particular, the contexts in which a certain type of names (e.g., PERSON) appear in the training set will be different from those in which these names appear in the test set. This experiment will allow us to determine the extent to which a resolver makes use of contextual information when identifying coreference links involving names: if it makes heavy use of contextual information, we should see a considerable drop in resolver performance.

6.1.2 Perturbations to Nominal Mentions

We consider three kinds of perturbations to nominal mentions to determine the roles they play.

Unseen nominals. This experiment has the same setup as the “Unseen names” experiment above, except that we replace each nominal mention in the training set with another mention in which we reverse the order of the characters of each of its words. Note that this is a mention-internal perturbation, meaning that we replace all and only those nominals that are annotated as entity mentions, not all nominals in the training set.

Nominals of a different type. This experiment has the same setup as the “Names of a different type” experiment above, except that we replace nominal mentions rather than names. As in the previous experiment, we replace each nominal that is annotated as an entity mention in the training set.

Nominals of the same type. This experiment has the same setup as the “Nominals of a different type” experiment above, except that the nominal mention being replaced must have the same entity type as its replacement. This kind of perturbation

is “milder” than the previous kind of perturbation, as a PERSON mention will continue to appear in a PERSON context after the replacement. In other words, if a machine learner does not pay attention to the semantic compatibility between a nominal mention and its context, then we should see little performance difference when a resolver is trained on this training set vs. the previous training set (i.e., the one from “Nominals of a different type”).

6.2 Mention-external Perturbations

Mention-external perturbations involve making changes to the words outside a mention.

6.2.1 Perturbations to Verbs

We consider two kinds of perturbations to verbs to determine the role they play in resolution.

Unseen verbs. This experiment has the same setup as the two “Unseen” experiments above, except that we replace each verb in the training set that is not part of an entity mention.

Seen verbs. This experiment has the same setup as the “Names of a different type” experiment above, except that we replace verbs outside of entity mentions rather than names. In particular, the new verb is not constrained to have the same type as the verb being replaced: it can be any verb taken from the training set. Nevertheless, the replacement is deterministic: all occurrences of a given verb will be replaced with the same verb.

6.2.2 Perturbations to Adjectives & Adverbs

We consider two kinds of perturbations to adjectives and adverbs to determine the roles they play.

Unseen adjectives and adverbs. This experiment has the same setup as the three “Unseen” experiments above, except that we replace each adjective and adverb in the training set that is not part of an entity mention.

Seen adjectives and adverbs. This experiment has the same setup as the “Seen verbs” experiment above, except that we replace adjectives and adverbs outside of entity mentions rather than verbs.

6.3 Perturbation Results

Results of these experiments on the three datasets are shown in Table 7. As in Table 5, we only show the results of ELMo and SpanBERT-1. For each resolver, we show its CR CoNLL score and its MD F-score. To facilitate comparison, we show in row 1 the performance of the resolvers when the input is not perturbed.

Perturbation Type	ACE				OntoNotes				KBP			
	ELMo		SpanB-l		ELMo		SpanB-l		ELMo		SpanB-l	
	CR	MD	CR	MD	CR	MD	CR	MD	CR	MD	CR	MD
1 No Perturbation	71.8	90.4	78.9	92.3	73.0	85.1	79.6	88.2	69.7	85.3	75.8	88.5
2 NAM-Unseen	68.4	88.6	74.9	90.8	46.2	63.3	65.7	77.2	62.1	79.7	65.7	79.7
3 NAM-DiffType	69.5	89.5	75.7	91.1	58.8	75.2	72.7	83.5	64.3	82.5	67.2	81.4
4 NOM-Unseen	67.6	86.5	73.8	88.7	57.4	73.6	70.7	81.9	60.6	76.4	67.8	81.7
5 NOM-DiffType	69.7	88.6	74.7	89.6	59.0	75.4	71.2	82.1	66.0	82.5	70.7	83.6
6 NOM-SameType	69.9	89.2	75.3	90.1	62.5	79.2	72.8	83.4	66.8	83.4	72.1	84.9
7 Verb-Unseen	70.6	89.4	76.6	91.1	64.7	79.7	75.3	86.2	68.2	84.2	74.0	86.9
8 Verb-Seen	71.6	90.1	77.1	91.1	67.2	81.1	77.0	87.0	68.8	84.7	74.4	87.1
9 Adj/Adv-Unseen	71.7	89.9	76.8	91.0	68.9	82.3	77.4	87.4	69.3	85.3	73.5	86.4
10 Adj/Adv-Seen	69.8	88.8	76.9	91.3	70.1	83.0	76.3	86.7	68.5	84.8	74.0	86.6

Table 7: Perturbation results.

A few points deserve mention. First, mention-internal perturbations (rows 2–6) triggered larger deterioration in CR performance than mention-external perturbations. These results suggest that the resolvers rely more on the mentions themselves than their contexts for resolution, which should not be surprising. Among the mention-internal perturbations, the biggest CR performance drops occur with the Unseen perturbations (rows 2 and 4), particularly those involving unseen names, followed by perturbations involving the replacement of a seen name or nominal with a different type. A closer inspection of the results reveals that there is a strong correlation between CR performance and MD performance: larger drops in CR performance are always accompanied by larger drops in MD performance. This sheds light on why the Unseen perturbations triggered the largest drop in CR performance: when all the names or nominal mentions in the test set are not seen in the training set, the mention detector is likely to perform poorly on the test set. In contrast, when they are replaced by mentions of a different entity type, the percentage of unseen mentions in the test set doesn’t change, thus posing fewer problems for the mention detector.

As for the mention-external perturbations, no clear patterns emerged: while verb replacement (rows 7–8) has a greater impact than adjective and adverb replacement (rows 9–10) for OntoNotes, the same observation cannot be made for the other datasets. Moreover, while replacing a word with another seen word is generally expected to cause less harm to MD (and thus CR) performance than replacing a word with an unseen word, these experiments show that this is not necessarily the case. These results seem to suggest that the mention span learner is not particularly sensitive to the verbs, adjectives and adverbs that appear in the context.

Third, it is not easy to conclude which resolver

is more robust to perturbations. While the drops in CR performance on ACE are fairly mild for both resolvers, we see bigger CR performance drops on the other two datasets. In particular, ELMo suffers from a bigger drop in performance than SpanBERT-l on OntoNotes, whereas the reverse is true on KBP.

Finally, while the two resolvers’ MD performances are similar, SpanBERT-l’s CR performance is always superior to ELMo’s. These results reveal once again that the mention representations learned by SpanBERT-l are indeed better than those by ELMo as far as resolution is concerned.

7 Conclusions

While space limitations preclude a reiteration of all the observations we have made, we believe the key conclusions are: (1) the relative performances of the resolvers are consistent across datasets; (2) for each resolver, higher mention detection performance always yields better coreference performance; (3) the newest resolvers perform better because of not only improved mention detection, but also improved mention span representations, and they improved the resolution of both easy-and difficult-to-resolve anaphors; (4) all resolvers can be improved by improving mention detection, anaphoricity determination, and entity type detection; and (5) our perturbation results suggest that coreference performance is most sensitive to those words/phrases in the input that have the greatest impact on mention detection performance.

Acknowledgments

We thank the three anonymous reviewers for their comments. The paper’s title was motivated by that of Stoyanov et al.’s ACL 2009 paper — the credit goes to them. This work was supported in part by NSF Grants IIS-1528037 and CCF-1848608.

MUSTARD: MASTERING UNIFORM SYNTHESIS OF THEOREM AND PROOF DATA

Yinya Huang^{1,6} Xiaohan Lin² Zhengying Liu^{3†} Qingxing Cao^{2†} Huajian Xin²

Haiming Wang² Zhenguo Li³ Linqi Song^{1,6†} Xiaodan Liang^{2,4,5†}

¹City University of Hong Kong ²Shenzhen Campus of Sun Yat-sen University

³Huawei Noah’s Ark Lab ⁴DarkMatter AI Research ⁵MBZUAI ⁶CityUSRI

yinya.huang@hotmail.com, linxh55@mail2.sysu.edu.cn,
{liuzhengying2, Li.Zhenguo}@huawei.com, caoqx8@sysu.edu.cn,
linqi.song@cityu.edu.hk, xdliang328@gmail.com

ABSTRACT

Recent large language models (LLMs) have witnessed significant advancement in various tasks, including mathematical reasoning and theorem proving. As these two tasks require strict and formal multi-step inference, they are appealing domains for exploring the reasoning ability of LLMs but still face important challenges. Previous studies such as Chain-of-Thought (CoT) have revealed the effectiveness of intermediate steps guidance. However, such step-wise annotation requires heavy labor, leading to insufficient training steps for current benchmarks. To fill this gap, this work introduces MUSTARD, a data generation framework that masters uniform synthesis of theorem and proof data of high quality and diversity. MUSTARD synthesizes data in three stages: (1) It samples a few mathematical concept seeds as the problem category. (2) Then, it prompts a generative language model with the sampled concepts to obtain both the problems and their step-wise formal solutions. (3) Lastly, the framework utilizes a proof assistant (e.g., Lean Prover) to filter the valid proofs. With the proposed MUSTARD, we present a theorem-and-proof benchmark MUSTARDSAUCE with 5,866 valid data points. Each data point contains an informal statement, an informal proof, and a translated formal proof that passes the prover validation. We perform extensive analysis and demonstrate that MUSTARD generates validated high-quality step-by-step data. We further apply the MUSTARDSAUCE for fine-tuning smaller language models. The fine-tuned Llama 2-7B achieves a 15.41% average relative performance gain in automated theorem proving, and 8.18% in math word problems¹.

1 INTRODUCTION

Large language models (LLMs) (OpenAI, 2023; 2022) have shown promising reasoning capabilities in various domains, including math word problem and theorem proving (Cobbe et al., 2021; Hendrycks et al., 2021; Zheng et al., 2022; Wu et al., 2021). These two tasks, which require strictly and successively multi-step inference, have become appeal domains to evaluate and develop LLMs’ ability in complex reasoning. Recent works progress LLMs in solving math problems mainly through two techniques. The first is the chain-of-thoughts (CoT) prompting (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023b), which provides step-by-step solutions to the LLMs. The second is to leverage the LLMs’ ability in code generation to generate formalized languages and utilize external solvers to obtain strict inference results (Wu et al., 2022; Jiang et al., 2023; Polu & Sutskever, 2020; Han et al., 2022; Polu et al., 2023). Both techniques rely on step-wise annotation to improve LLMs’ performance and interpretability on the math problem.

Correct intermediate steps are crucial for LLMs to perform complex reasoning. However, high-quality step-wise annotations are hard to obtain, and Figure 1 demonstrates a few representative

[†]Corresponding author.

¹Codes and data are available at: <https://github.com/Eleanor-H/MUSTARD>.

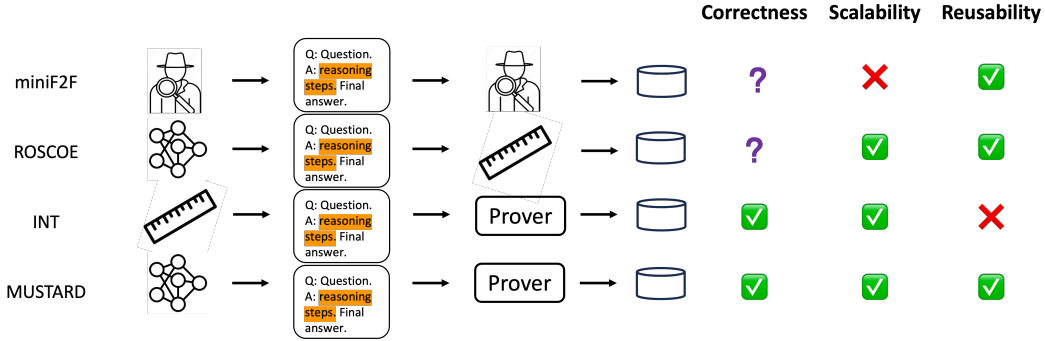


Figure 1: A comparison of methods of synthesizing and validating intermediate reasoning steps.

works. Previous works such as miniF2F (Zheng et al., 2022) resorts to manual annotation and validation to obtain high-quality step-wise labels. However, manual annotation requires heavy labor of knowledgeable experts, resulting in an extremely small-scale dataset. Manual checking also does not guarantee the correctness of data as the labelers would make mistakes in labeling. On the other hand, generating data with rule-based checking such as ROSCOE (Golovneva et al., 2023) can produce large-scale reasoning data. Given that the generated data are more friendly and readable for human beings, the correctness of the reasoning is not guaranteed by those rules. Moreover, another line of work such as INT (Wu et al., 2021) performs rule-based synthesis to generate validated proofs, which are both correct and large-scale. However, the data are brutally synthesized so that many generated proofs lack actual meaning. Therefore, we need a more efficient way to generate mathematical data that are large-scale, with accurate intermediate steps, and also meaningful mathematical knowledge to human beings.

To fill this gap, we propose MUSTARD, a data generation framework that uniformly synthesizes large-scale and high-quality mathematical data by combining the advantages of LLMs in verbalization and formal theorem provers in rigorous data validation. Specifically, MUSTARD first samples a few mathematical concepts from a predefined list and prompts an LLM to generate a related question described in natural language. Then, it applies the LLM to generate the corresponding solution in both natural and formal language. Given the generated solution, MUSTARD further validates them using a theorem prover. The passed one is considered to be correct and is a high-quality data point. The invalid one on the other hand is considered to be a challenging sample, which will be further combined with the error messages to prompt the LLM for a solution revision, and added as a challenging data point.

By applying the proposed MUSTARD one can obtain large amounts of problems and theorems with desired mathematical concepts and domains. Eventually, we build a mathematical dataset with validated informal and formal solutions, named MUSTARDSAUCE (MUSTARD resource).

We conduct extensive data analysis and experiments on the generated MUSTARDSAUCE. Through deep inspection of the data, we find that MUSTARD generates interesting and reasonable math problems by creatively combining two mathematical concepts, and MUSTARDSAUCE is diverse and has a high proportion of difficult data. We also observe that the prover is consistent with human evaluation, where humans usually consider a validated solution to have a higher quality than those without a formal validation process. Lastly, we fine-tune smaller-scale language models on MUSTARDSAUCE. The fine-tuned Llama 2-7B achieves improvements by 20.9% on zero-shot inference on GSM8K and achieves 8.7 of pass@1 on mathlib. These results demonstrate the effectiveness of MUSTARDSAUCE in improving the mathematical reasoning capabilities of language models.

The contributions of this paper are summarized as follows:

1. We propose a novel framework MUSTARD that can generate high-quality mathematical data (both informal and formal) with an interplay between generative language model and theorem prover assistants.
2. We release the MUSTARDSAUCE, which contains both math word problems and theorem-proving problems spanning over four educational levels. Each sample has corresponding informal and formal solutions.
3. We conduct extensive analysis and experiments on the generated data, demonstrating their quality, diversity, and effectiveness in improving language models’ mathematical reasoning performance.

2 RELATED WORKS

Large Language Models for Mathematical Reasoning The growing generative language models (Brown et al., 2020; OpenAI, 2022; 2023) show compelling potential for solving mathematical problems both in natural language proofs (OpenAI, 2023; Zheng et al., 2023a; Xiong et al., 2024) and in formal languages with theorem provers (Polu & Sutskever, 2020; Han et al., 2022; Zheng et al., 2023b; Polu et al., 2023). On the other hand, some works explore using language models to automatically translate natural language proofs into formal ones given few-shot demonstrations (Wu et al., 2022; Jiang et al., 2023; Liu et al., 2023; Wang et al., 2024). Chain-of-thought reasoning (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023b) is demonstrated beneficial for the LLMs to derive correct answers. However, some recent works (Saparov & He, 2023; Golovneva et al., 2023) observe that the intermediate reasoning steps can be inconsistent. This paper proposes a data generation framework that taps the comprehensive mathematical reasoning capabilities of large language models. It generates mathematical reasoning problems with informal and formal solutions that are step-wise validated by a formal theorem prover. With the framework, we obtain high-quality mathematical data.

Synthesizing Mathematical Data Obtaining large-scale high-quality mathematical data is a long-standing challenge. Previous data relies on well-trained annotators to hand-craft and review the formal proofs (Zheng et al., 2022), which is time and labour-consuming and results in a small data scale. Wang & Deng (2020) and Xiong et al. (2023) construct a neural generator for data synthesis, but it still requires the intervention of human-written data. Besides, Wu et al. (2021) explore using a theorem generator to automatically generate formal proofs with rules. However, the rule-based generation depends on given axioms in specified orders. As a result, the generated data is restricted to a few domains. On the other hand, recent works demonstrate the effectiveness of distilling knowledge from large language models (West et al., 2022; Yuan et al., 2023; Li et al., 2023), and some of them (Wang et al., 2023c; Xu et al., 2023) explore data evolution by properly prompting the language models. The proposed framework explores eliciting mathematical knowledge from large language models to achieve diverse and large-scale mathematical data. In this framework, an interplay between the language model and a formal proof assistant controls the quality and difficulties of data. Using the proposed framework, we collect a large-scale mathematical dataset that contains diverse and multiple-difficulty math questions with high-quality solutions.

3 MUSTARD

In this work, we aim to obtain large-scale mathematical data with multi-step annotations and propose MUSTARD to generate diverse and high-quality math and theorem-proving problems with multi-step informal and formal solutions. As shown in Figure 2, MUSTARD consists of three stages. In the first concept seeding stage, MUSTARD samples a set of math concepts as the problem domain. Then in the second solution generation stage, it generates the concept-related problem and solution by prompting an LLM. In the third stage, a theorem prover is used to validate the generated solution. If the solution can not pass the prover, the error message is returned to the second stage for another turn of solution generation. Through interaction between the LLM and a formal proof assistant, MUSTARD can generate diverse and high-quality data that contains both informal and formal solutions. We describe the details of each stage in this section.

3.1 CONCEPT SEEDING

We first define and build a mathematical concept pool that covers as complete sub-subjects in mathematics and educational levels as possible. Specifically, we collect all math courses on the Khan Academy website², the large-scale online educational platform. The resulting pool includes concepts in four educational levels: elementary school, middle school, high school, and higher education. Each educational level has 5 to 9 math domains, covering different types of math problems such as algebra and geometry. Each domain contains subdivided mathematical concepts to inspect different mathematical abilities like polynomial arithmetic or factorization. Concept statistics and detailed concepts in each domain are demonstrated in Appendix B.

Given the concept pool, for each educational level, MUSTARD uniformly samples 1 or 2 concepts from all domains as seeds, and then generates mathematical problems that cover the concepts. In particular, given an educational level, taking 2 concepts from different subjects challenges the model to generate problems that join diverse domains while keeping the problems reasonable.

²<https://www.khanacademy.org/math>

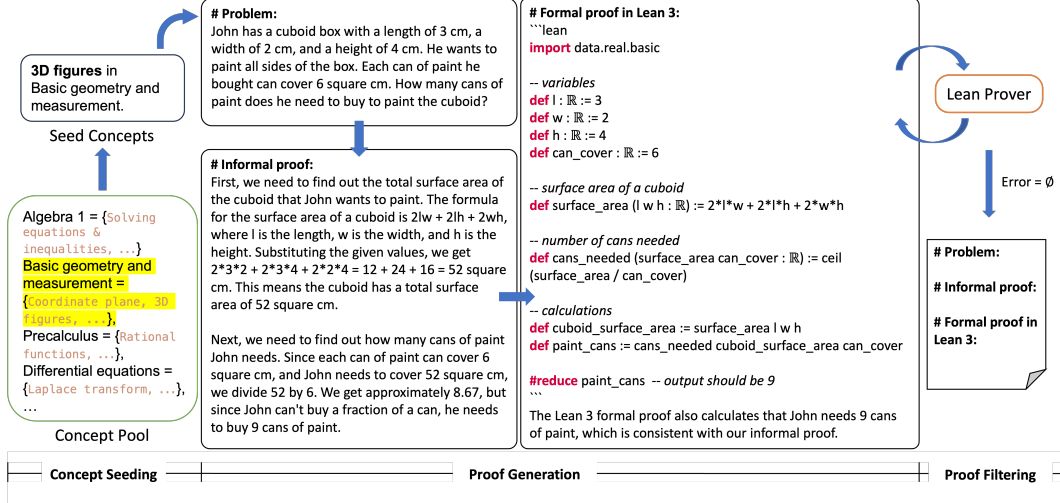


Figure 2: Overview of MUSTARD. The mathematical theorem (indicated by “# Problem:”) is created according to the randomly selected seed concepts. Following the theorem, a corresponding informal proof is generated, which is then translated into a formal proof in Lean 3. The formal proof is passed into Lean Prover for quality feedback, and according to the error messages the formal proof is revised and improved. The proofs that pass the Lean Prover are regarded as high-quality data points and collected.

3.2 PROOF GENERATION

Given the sampled mathematical concepts, MUSTARD generates math problems and their corresponding solutions. Specifically, MUSTARD leverages the capability of LLMs in generating natural language and code and prompts an LLM to generate the problem statement, its natural language solution, and a formal solution written in Lean. As a result, the LLM needs to complete the following three tasks: (T1) Generating a math problem that relates to the given concepts; (T2) Solving the math problem with a natural language proof; (T3) Performing auto-formalization to translate the written natural language proof into a formalized proof. In this work, we use GPT-4 [OpenAI \(2023\)](#) as the LLM for proof generation.

We intend to generate a problem based on educational level, math domains, and concepts. Considering that mathematical problems include proof and calculation, we also introduce the question types into the prompt for generating theorem proving and word problems respectively. Moreover, we do not include any exemplars or other manual interventions except for the sampled concepts. We intend to avoid potential biases brought by the concepts inside the exemplars and achieve a more diverse generation. The prompt template is shown as the following:

You are a math expert. Now please come up with a math problem according to the following requirements. The math problem should contain a question part (indicated by “# Problem:”), a corresponding solution in natural language (indicated by “# Informal proof:”), and a translated formal solution in Lean 3 (indicated by “# Formal proof in Lean 3:”). Please note that the informal proof and the formal proof need to be identical.

Please create a [QUESTION TYPE] in the level of [EDUCATIONAL LEVEL] based on the following knowledge point(s): [CONCEPT] in [DOMAIN]; [CONCEPT] in [DOMAIN].

You must respond in the following format:

```
# Problem: ...  
  
# Informal proof: ...  
  
# Formal proof in Lean 3: ...
```

The “[]” indicates the placeholders for the corresponding question type, educational level, concepts, and domains. Multiple concepts are separated by “;”. We retrieved the text after the “Problem:”, “Informal proof:” and “Formal proof in Lean 3:” as the generated sample.

3.3 PROOF FILTERING

In the proof-filtering stage, MUSTARD interacts with the Lean Prover (de Moura et al., 2015) to obtain validation messages of the proof steps, which guides data revision and filtering. Specifically, after a formal solution is passed to the Lean Prover and if the prover returns no error message, the corresponding data point is collected into the valid dataset. Otherwise, MUSTARD collects the error messages from the prover and prompts the language model to revise the invalid solution. To help the language model locate the incorrect lines described in the error messages, we also add a line number at the beginning of each line in the formal solutions. The verification and self-refinement are performed in multiple rounds until LLM generates a valid solution. We use the number of rounds to measure the difficulty of the generated sample, assuming a difficult problem is hard to solve by an LLM and requires more rounds of correction. The prompt template of a single round of correction is demonstrated as follows, and the complete prompt template is shown in Table I3 in Appendix C.1:

```
# Formal proof (c) in Lean 3:
```lean
line 1 <code>
line 2 <code>
line 3 <code>
...
```

# Error messages for Formal proof (c) from Lean Prover:
<error messages>
```

4 EXPERIMENTS

4.1 CASE STUDY

We first inspect the data points generated by MUSTARD. Table 1 shows a generated math problem in which MUSTARD creatively combines two mathematical concepts and constructs a reasonable question. The generated question includes knowledge from both concepts. It is suggested that MUSTARD can join the concepts and construct a reasonable question. Furthermore, Table 2 demonstrates a case that MUSTARD provides solid and comprehensive solutions in both natural language and Lean. Although the formal proof is long, it is consistent and passes the prover’s validation. It is demonstrated that MUSTARD can generate long valid solutions.

4.2 HUMAN EVALUATION

To further explore the quality of the data generated by MUSTARD, we recruit professionals who have expertise in mathematics and the Lean language to perform a sanity check on the data points. We randomly select 200 data points from the generated data, 100 of which pass the Lean Prover (Group Valid) and 100 of which do not (Group Invalid). The sanity check covers the four sections in each data point (i.e., informal statement, informal proof, formal statement, and formal proof), and includes factuality check and consistency check. Specifically, a high-quality data point should have a factually correct informal statement (D1) and a correct solution (D4). The formal statement and proof should be aligned with the informal descriptions (D5, D6). Moreover, the desired data point should meet the specified seed concepts (D2) and question type (D3). The six inspection dimensions and their requirements are demonstrated in Table 3. A data point is scored 1 in a dimension if it meets the requirement, otherwise, it gets 0.

The accuracies of Group Valid and Group Invalid in each dimension are demonstrated in Table 3. We also report the corresponding p -value in each dimension. (D4) and (D6) show significant differences in accuracy between the two groups. The results indicate that high-quality data points have significantly better auto-formalization results. As a result, given the validation of formal proofs by the Lean Prover, the data points have guaranteed high-quality informal proofs. Moreover, (D1) also shows significance with the inspected data scaled up. The differences in statement alignment (D5) and informal statement relevance (D2) of the two groups are less significant. Furthermore, no significant differences are observed in question type classification (D3), which indicates that Lean Prover validation in MUSTARD does not significantly influence the classification. Overall, the human evaluation results suggest that formally validated data have significantly higher quality.

4.3 DATA QUALITY BY DOWNSTREAM APPLICATION

To evaluate the impact of MUSTARDSAUCE on enhancing mathematical reasoning abilities, we use the data to fine-tune smaller-scale language models and evaluate them on math word problems (MWP) and automated theorem proving (ATP). Specifically, given all the generated data, we first